

Bachelor's Thesis

Importance Sampling with Adapted Exponential Families

Department of Mathematics

Giovanni Petrarolo



Supervisor:	Priv.-Doz. Dr. Ari-Pekka Perkkiö
Degree Program:	Wirtschaftsmathematik (2021)
Matriculation No.:	12624451
Date:	October 10, 2025

Declaration of Originality

I confirm that the submitted thesis is original work and was written by me without further assistance. Appropriate credit has been given where reference has been made to the work of others. The thesis was not examined before, nor has it been published.

Giovanni Petrarolo

Location/Date

Abstract

This work aims to conduct Importance Sampling with respect to a index parameter dependent sample value $f(X_1, X_2, \dots, X_S) \in \mathbb{R}$ by adapting exponential families to consider each random vector X_s , $s = 1, \dots, S$. To achieve this, a practical Python program is written to efficiently find the most fitting exponential family to perform Importance Sampling with. The foundation to discovering optimal exponential families lies within the commonly known problem of Variance Reduction. As the implementation process' difficulty is directly linked to the number of variables, a theoretical framework has been designed with practicability in mind. Hence, the framework aims to restrict the underlying optimization problem whilst giving more leniency to find the most suitable exponential family considering the situation. In order to evaluate the proposed code, famous models, like the Black-Scholes model, are employed to analyze the performance of the proposed framework and to find out if it improves on the traditional way to finding the expectation of $f(X_1, X_2, \dots, X_S)$ which can be a path-dependent option in the context of Black-Scholes.

Contents

1	Introduction	4
2	Monte Carlo and Importance Sampling	6
2.1	Introduction to the Monte Carlo Method	6
2.2	Importance Sampling	10
2.2.1	Defining Basics of Importance Sampling	10
2.2.2	Basics of Variance Reduction	12
3	Exponential Families	14
3.1	Introducing Exponential Families	14
3.2	Variations of Exponential Families	17
3.3	Joint and Adapted Exponential Families	19
4	Introducing Adapted Exponential Families in IS	23
4.1	Creating a Convex Optimization Problem	24
4.2	Choosing the Type of Exponential Family	26
4.3	Creating an Adapted Exponential Family for IS	27
5	Implementation of Adapted Exponential Families in IS	34
5.1	Construction of the Natural Parameter: Code Implementation	34
5.1.1	Natural Parameter Method: Exponential Distribution	36
5.1.2	Natural Parameter Method: Multivariate Normal Distribution	39
5.2	Optimization the Variance: Code Implementation	41
5.2.1	Optimization Method: Exponential Distribution	42
5.2.2	Optimization Method: Multivariate Normal Distribution	47
6	Demonstrating the Code Implementation	53
6.1	Model 1: Black-Scholes Model with Asian Spread Call Option	55
6.1.1	Practical Implementation of Model 1	56
6.1.2	Evaluating IS for Model 1	59
6.1.3	Summary of the Evaluation	65
6.2	Model 2: Poisson Process with Exponential Distribution	66
6.2.1	Practical Implementation of Model 2	67
6.2.2	Evaluating IS for Model 2	67
6.2.3	Summary of Evaluation	70
7	Conclusion	71
A	Appendix	72
A.1	Mathematical Preliminaries	72
A.2	Tables and Graphics: Model 1	75
A.3	Tables and Graphics: Model 2	83
A.4	Full Code	92
A.4.1	Code class_natural_parameter_components.py	92
A.4.2	Code class_IS_optimizer.py	96
A.4.3	Example Code: Model 1	102
A.4.4	Example Code: Model 2	104
A.4.5	Example Code: Monte Carlo Circle	105
	References	106

1 Introduction

The usage of expectations in the probability theory sense has been broadly established throughout many branches of science. Fields like Machine Learning, Finance Mathematics, Physics or even Casinos do take advantage of the expectation to estimate outcomes or profits. Naturally, all of these branches try to find this value with the data available to them. However, what if certain data is missing or the commonly known definition of the expectation

$$E_P[X] = \int_{\Omega} X dP$$

is too complicated to evaluate? Many methods and theories have been developed to deal with such problems, most notably, the Monte Carlo method.

In this thesis, the reader is introduced to such methods, however, the main topic throughout is the so called Importance Sampling process. Importance Sampling is already a well known method used in the context of Monte Carlo to improve the output of the approximations. Since the most basic variation of Importance Sampling is already well researched this work tries to practically implement a lesser-known approach. To be more particular, this study will utilize something this thesis refers to as adapted exponential families to conduct Importance Sampling.

As an introduction, sections 2 will present the reader to the basics needed to understand the topic at hand. With chapter 3 the focus will shift from the already well-known exponential families to the adapted exponential families. After both Importance Sampling and adapted exponential families were explored separately throughout these two chapters, section 4 will try to explain on how to combine those components, creating the foundation for the practical implementation.

By chapter 5 the theory is being converted to the practical code. Via Python this work tries to build a structure which can be used by a third party to conduct Importance Sampling with adapted exponential families accordingly. The idea is here to allow the reader full access to the code, which is shown in the Appendix, and to explore the thought process made while the program was written. Another important aspect of this study was to make the created code as simple as possible in order to make it easily expandable for future projects. At the end of this thesis the created code is presented by using well-established models in order to convey the intended use of the program to the reader.

As the topics discussed here are mainly founded in probability theory, an effort is made to maintain consistent notation. However, with the in-

introduction of new terminology from different fields and own creation, this thesis cannot limit itself to use commonly recognized notation throughout. Nonetheless, most of the terminology is cross-chapter, meaning that the notations used generally remain constant throughout, allowing for a better reader experience. The same can be said about indexation. Although most of the topics regarding Importance Sampling with adapted exponential families are well motivated, this thesis will take advantage of definitions, lemmas or theorems that are outside the scope of this study. If such knowledge is used, the reader will in most cases be referred to the Appendix where all topics are listed, however, without proof.

It should be noted that all throughout this thesis most operations will take place in the underlying probability space with $(\Omega, \mathcal{F}, P)$ where P is probability measure and $\mathcal{F}$ is the σ -algebra over a set Ω . Also for readability, this work will take the liberty to simplify a few expressions, for example some functions will be written with the argument and at some point without or the expectation $E_P[X]$ will be referred to as $E[X]$ instead.

2 Monte Carlo and Importance Sampling

In this chapter the foundation for this thesis is laid. In particular, already established methods, like the Monte Carlo method and Importance Sampling, are introduced and discussed. To ensure continuity, this chapter will also include many necessary definitions and theorems which will be needed in future chapters.

2.1 Introduction to the Monte Carlo Method

The Monte Carlo method is one of the most popular classes of computational algorithms. Its origin reaches back to the 20th century and often describes the method which relies on random sampling to acquire a practical result. This means that the idea of Monte Carlo method is based in statistics and an end result is obtained by repeating an experiment continuously by acquiring random samples; see [18] [22].

Therefore, the Monte Carlo method is based on the strong law of large numbers, also known as Kolmogorov's law, which will play an important role throughout this thesis.

Theorem 2.1 (Strong law of large numbers [2]).

Let $(X^{(m)})_{m=1}^{\infty}$ be a sequence of i.i.d. random variables with expectation $\mu := E[X^{(m)}]$ with $E[|X^{(m)}|] < \infty$ and $\sigma^2 := Var(X^{(m)}) \forall m \in \mathbb{N}$. Then for $M \in \mathbb{N}$

$$\frac{1}{M} \sum_{m=1}^M X^{(m)} \xrightarrow{M \rightarrow \infty} \mu$$

holds almost surely.

MCE and sample variance

Informally the strong law of large numbers states that average of a large sample size of i.i.d. random variables converges (almost surely) to their common expectation. For each $M \in \mathbb{N}$ the average is still a random variable and is called the "sample mean" or in the context of Monte Carlo the "Monte Carlo estimator", MCE for short; see [16].

For future references, the MCE will appear in different contexts throughout this thesis. In most cases each sample $X^{(m)}$, which can be a random vector, is being concatenated with a function f . Nevertheless, the meaning behind the MCE still holds, this is why from now on it will be denoted as

$$\mu^{(M)} := \frac{1}{M} \sum_{m=1}^M X^{(m)} \text{ or } \mu_f^{(M)} := \frac{1}{M} \sum_{m=1}^M f(X^{(m)}) \quad (1)$$

depending on the context.

It also important to note that $\mu^{(M)}$ maintains its expectation, meaning

$$E[\mu^{(M)}] = E\left[\frac{1}{M} \sum_{m=1}^M X^{(m)}\right] = \frac{1}{M} \sum_{m=1}^M \underbrace{E[X^{(m)}]}_{=\mu \ \forall m} = \frac{1}{M} M\mu = \mu \quad (2)$$

holds. Whereas due to independency (see A.1) of the random variables $(X^{(m)})_{m=1}^\infty$ the variance satisfies

$$\text{Var}\left(\mu^{(M)}\right) = \frac{1}{M^2} \text{Var}\left(\sum_{m=1}^M X^{(m)}\right) = \frac{1}{M^2} \sum_{m=1}^M \text{Var}\left(X^{(m)}\right) = \frac{1}{M} \sigma^2. \quad (3)$$

Although the strong law of large numbers can be used for normal distributed random variables, resulting in a normal distributed MCE, one might use an unspecified sequence of i.i.d. random variables. However, by extending $\mu^{(M)}$, using equations (2),(3) and the Central Limit theorem (see A.4), one can "make" the MCE normal for large enough M . Formally, if $\text{Var}(X^{(m)}) < \infty$ then

$$\sqrt{M}(\mu^{(M)} - \mu) \xrightarrow{d} \eta, \quad (4)$$

holds for some random variable $\eta \sim \mathcal{N}(0, \sigma^2)$; see [16].

Confidence intervals

As the Monte Carlo method is based on repeating an experiment as often as one can, resulting practically in only a finite number of random values $X^{(m)}$, one might ask how accurate each result is. To answer this question a "confidence interval", ci for short, around $\mu^{(M)}$ has to be created. With context from theorem A.4, one can define the so called "sample variance" via

$$(\sigma^{(M)})^2 := \frac{1}{M-1} \sum_{m=1}^M (X^{(m)} - \mu^{(M)})^2; \text{ see [16].}$$

Indeed, $(\sigma^{(M)})^2$ represents an estimation to σ^2 . More accurately,

$$\begin{aligned}
(\sigma^{(M)})^2 &= \frac{1}{M-1} \sum_{m=1}^M (X^{(m)} - \mu^{(M)})^2 \\
&= \frac{M}{M-1} \left(\sum_{m=1}^M \frac{(X^{(m)})^2}{M} - 2\mu^{(M)} \underbrace{\frac{1}{M} \sum_{m=1}^M X^{(m)}}_{=\mu^{(M)}} + \underbrace{\frac{1}{M} \sum_{m=1}^M (\mu^{(M)})^2}_{=(\mu^{(M)})^2} \right) \\
&= \frac{M}{M-1} \left(\sum_{m=1}^M \frac{(X^{(m)})^2}{M} - (\mu^{(M)})^2 \right) \rightarrow E[X^2] - (\mu)^2 \\
&= \sigma^2
\end{aligned} \tag{5}$$

holds by applying theorem 2.1. Here X is a random variable with $X \sim X^{(m)} \forall m$. Given the sample variance one can create a confidence interval upon using the error $\sigma^{(M)}/M$ after M experiments; see [26]. Thus, the confidence interval is given via

$$\left[\mu^{(M)} - \frac{c\sigma^{(M)}}{\sqrt{M}}, \mu^{(M)} + \frac{c\sigma^{(M)}}{\sqrt{M}} \right]$$

where c is the so called "critical value". Choosing $c = 1.96$ and given a large enough sample set of M experiments would yield a 95% probability that μ is within the given confidence interval; see [16].

It is noteworthy that the sample variance is *not* the same as $\text{Var}(\mu^{(M)})$, however, by applying (3) both are related through

$$\begin{aligned}
E[(\sigma^{(M)})^2] &= E \left[\frac{1}{M-1} \sum_{m=1}^M (X^{(m)} - \mu^{(M)})^2 \right] \\
&= \frac{1}{M-1} E \left[\sum_{m=1}^M ((X^{(m)} - \mu) - (\mu^{(M)} - \mu))^2 \right] \\
&= \frac{1}{M-1} \sum_{m=1}^M E \left[(X^{(m)} - \mu)^2 \right] - 2ME \left[(\mu^{(M)} - \mu)^2 \right] + ME \left[(\mu^{(M)} - \mu)^2 \right] \\
&= \frac{1}{M-1} \left(M \text{Var}(X) - 2M \frac{\text{Var}(X)}{M} + M \frac{\text{Var}(X)}{M} \right) \\
&= \frac{1}{M-1} ((M-1) \text{Var}(X)) = \sigma^2 = M \text{Var}(\mu^{(M)})
\end{aligned}$$

where again $X \sim X^{(m)} \forall m$.

To better understand the Monte Carlo method consider the following simple example.

Example 2.2 (The Monte Carlo Circle [16]).

The following problem is given: One must approximate $\pi \approx 3,14159\dots$ by only using the Monte Carlo method.

To simplify the problem one can place a circle with radius $r = 1$ into a two by two square. Formally, let

$$\overline{B}_1(0) := \left\{ (x_1, x_2) \in \mathbb{R}^2 \mid \sqrt{x_1^2 + x_2^2} \leq 1 \right\} \subset [-1, 1]^2$$

be the closed unit circle. Given by the area formula of a circle one gets

$$r^2 \pi \stackrel{r=1}{=} \pi$$

whereas the area of the square is 4.

To use Monte Carlo a sequence of random vector is defined via $X^{(m)} := (X_1^{(m)}, X_2^{(m)})$ with $X_i^{(m)} \sim \text{Unif}([-1, 1])$, $i = 1, 2$ representing the first or second coordinate of a point. The idea is to throw points randomly onto the square and marking whether the point is within the circle or not. $X^{(m)}$ would represent the position of the m -th point thrown. After tossing a number of $M \in \mathbb{N}$ points, one can simply count the amount of throws which landed within the circle and calculate the relative amount. The resulting MCE would

$$\mu_f^{(M)} := \frac{1}{M} \sum_{m=1}^M f(X^{(m)})$$

with $f(x) := \mathbf{1}_{\overline{B}_1(0)}(x)$ being the indicator function of $\overline{B}_1(0)(x)$.

Since the points where tossed randomly onto the square, the relative amount of points within the circle should be equivalent to the relative area of the circle within the square. In particular, one can approximate π (almost surely) by applying the strong law of large numbers (see theorem 2.1)

$$4\mu_f^{(M)} \xrightarrow{M \rightarrow \infty} \pi$$

as $(f(X^{(m)}))_{m=1}^M$ is i.i.d. by construction. The following Python code simulates this.

```

5 sample_size = 10000
6 samples = np.random.uniform(-1, 1, size=(sample_size, 2))
7
8 def f(XX):
9     if XX[0]**2 + XX[1]**2 <= 1:
10         return 1
11     else:
12         return 0
13
14 f_val = [f(XX) for XX in samples]
15 f_arr = np.asarray(f_val)
16
17 mu_f = np.mean(f_arr)
18 MCE = 4*mu_f
19
20 c = 1.96
21
22 std = np.std(f_arr)
23 se_pi = 4 * std / np.sqrt(sample_size)
24 low_pi = MCE - c * se_pi
25 up_pi = MCE + c * se_pi

```

Python Code 1: Monte Carlo Circle

Using the code above and choosing `sample_size = 100` instead of 10000 results in an approximation of 3.04 with a 95% ci of [2.705167, 3.374833]. On the other hand `sample_size = 10000` gives a much better output of 3.14684 with a much tighter ci of [3.136684, 3.156996], demonstrating the operating principle of Monte Carlo and the power of a large sample set.

2.2 Importance Sampling

While the Monte Carlo method provides a powerful tool for estimating expectations, it is also prone to giving false impressions about the expected value. Reason behind this is that Monte Carlo is based on the strong law of large numbers which depends on a large sample set. However, what is "large"? An indicator for a proper sample size would be the confidence interval. Roughly speaking, a tighter ci indicates less fluctuation and thus a more likely accurate approximation. Here Importance Sampling comes into play.

2.2.1 Defining Basics of Importance Sampling

Formally, Importance Sampling (IS) describes a selection of Monte Carlo methods where an approximation of the expectation with respect to one distribution is constructed by using another distribution. In particular, let p be the probability density function (pdf) for a random vector X with respect to the underlying measure P . Then the expectation μ_f of $f(X)$, with f being

a real-valued outer function, is calculated via

$$\mu_f := E_p[f(X)] := E[f(X)] = \int p(x)f(x)dx.$$

Here the subscripted p highlights that the expectation is calculated with respect to p , whereas the integral symbol does not refer to the indefinite integral but rather symbolizes the integral over a proper set.

By introducing another probability density function q which fulfills $q(x) > 0$ as long as $p(x)f(x) \neq 0$ holds, one can rewrite μ_f by multiplying and dividing the integrand with q , resulting in

$$\mu_f = \int q(x) \left(\frac{p(x)}{q(x)} f(x) \right) dx = E_q \left[\frac{p(X)}{q(X)} f(X) \right]. \quad (6)$$

Introducing a sequence of i.i.d. random vectors $(X^{(m)})_{m=1}^M$ sampled from the probability density function q and applying the definition of the MCE from (1), one can modify the MCE by using (6) via

$$\mu_{f,I}^{(M)} := \frac{1}{M} \sum_{m=1}^M I(X^{(m)}) f(X^{(m)})$$

where $I(x) := \frac{p(x)}{q(x)}$; see [25]. Looking at the resulting equation

$$E_p[f(X)] = E_q \left[\frac{p(X)}{q(X)} f(X) \right]$$

one could also interpret IS as an application of the Radon-Nikodym theorem A.7. Indeed, one could write $I(X)$ as the Radon-Nikodym derivative $\frac{dP}{dP_q}$ for which

$$E_{P_q} \left[\frac{dP}{dP_q} \right] = 1 \quad (7)$$

holds for some arbitrary probability measure P_q .

To ensure that IS can be conducted appropriately one must consider that the used pdf q must fulfill various requirements.

Rules 2.3 (Rules for Importance Sampling [20]).

Let $(X^{(m)})_{m=1}^M$ be samples. Then q can be used for IS w.r.t. to p if points 1), 2) and 3) from below hold for all m .

1) *There must exist a procedure to draw $X^{(m)} \sim q$.*

2) *$q(X^{(m)})$ must be possible to calculate.*

3) *$q(X^{(m)}) > 0$ holds if $p(X^{(m)})f(X^{(m)}) \neq 0$ holds.*

It is important to keep in mind that $I(X^{(m)})f(X^{(m)})$ can be evaluated $\forall m = 1, \dots, M$ as this thesis assumes that $p(X^{(m)})$ and $f(X^{(m)})$ are well-defined $\forall m \in \mathbb{N}$; see [20].

Moreover, the first point from 2.3 is equivalent to the existence of a probability measure P_q such that $X^{(m)}$ has density q for all m . The following definition will take advantage of these rules and introduces notation created for this work.

Definition 2.4.

Let $\mathcal{Q}_p$ be the space of density functions q which are compatible conduct IS under w.r.t. p , i.e. rules 2.3 apply.

Note that $\mathcal{Q}_p$ is dependent on the underlying density p , however, as this thesis will always establish what the underlying density is, $\mathcal{Q}_p$ will simply be denoted by $\mathcal{Q}$.

While in general $\mu_f^{(M)}$ and $\mu_{f,I}^{(M)}$ are *not* equal, due to the shift in densities, both provide an estimation of the expectation μ_f and therefore both are MCEs and fulfill their respected properties. Indeed, due to the strong rule of large numbers from theorem 2.1 one gets

$$\mu_{f,I}^{(M)} - \mu_f^{(M)} \xrightarrow{M \rightarrow \infty} \mu_{f,I} - \mu_f = 0 \quad \text{a.s.}$$

and $\mu_{f,I} = \mu_{f,I} = \mu_f$ holds because of equation (6), meaning that both estimations result in the same expectations almost surely for $M \rightarrow \infty$.

2.2.2 Basics of Variance Reduction

Like in equation (4) one can find that $\sqrt{M}(\mu_f^{(M)} - \mu_f)$ and $\sqrt{M}(\mu_{f,I}^{(M)} - \mu_f)$ convert in distribution to normal distributed random variables η_f and $\eta_{f,I}$, respectively. While both η_f and $\eta_{f,I}$ have same mean 0, their respected variances are *not* generally equal; see [25]. Indeed, analogous to (5) one gets

$$(\sigma_f^{(M)})^2 \longrightarrow E_p[f(X)^2] - (\mu_f)^2 = (\sigma_f)^2$$

and

$$(\sigma_{f,I}^{(M)})^2 \longrightarrow E_q[(I(X)f(X))^2] - (\mu_f)^2 = (\sigma_{f,I})^2$$

where

$$E_p[f(X)^2] = \int p(x)f(x)^2 dx \stackrel{!}{=} \int q(x)(I(x)f(x))^2 dx = E_q[(I(X)f(X))^2]$$

does *not* generally hold, implying that $(\sigma_f)^2 = (\sigma_{f,I})^2$ does *not* generally hold either. Due to (3)

$$\text{Var}(\mu_f^{(M)}) = \frac{1}{M}(\sigma_f)^2 \stackrel{!}{=} \frac{1}{M}(\sigma_{f,I})^2 = \text{Var}(\mu_{f,I}^{(M)})$$

does also generally *not* hold.

This means that through IS a new MCE can be produced which approximates the same expectation as the traditional MCE, however, the accuracy of the approximation itself can vary. Caused by the changes in the fluctuations, the resulting confidence interval from the previous chapter 2.1 would also change, giving the chance to tighten the space within the interval.

Naturally, to use IS best, one would seek a q for which the variance is optimal, meaning

$$\text{minimize } \text{Var}_q(\mu_{f,I}^{(M)}) \quad \text{over } q \in \mathcal{Q}$$

where the subscripted q once again highlights that the variance is calculated with respect to q ; see [20].

At this point it is important to remark that if $f \geq 0$ and $\mu_f \neq 0$ the optimal density function is given by

$$q^*(x) = \frac{f(x)p(x)}{\mu_f}; \text{ see [20].}$$

Yet, the calculation of $q^*(x)$ would require the value μ_f which begs the question why to approximate the expectation in the first place, making q^* somewhat obsolete. Nonetheless, choosing the right $q \in \mathcal{Q}$ can lead to efficient variance reduction, meaning

$$\text{Var}_q(\mu_{f,I}^{(M)}) \leq \text{Var}_p(\mu_f^{(M)})$$

for a proper q ; see [20].

3 Exponential Families

This section will elaborate on all necessary definitions used in the context of exponential families, also known as Koopman-Darmois families, and discuss certain characteristics used throughout this thesis. As the term "exponential family" is often used loosely and mostly refers to a specific set of densities depending on some parameters, this chapter is divided into various subsections introducing the reader to exponential families step by step. Aim is that by the end of this chapter the reader gets familiarized with something this thesis refers to as adapted exponential families which will be more prominent in the parts that follows.

3.1 Introducing Exponential Families

Let X be a random vector taking values $x \in \mathbb{R}^d$, $d \in \mathbb{N}$. Additionally assume that X takes its values with jointly continuous components.

Definition 3.1 (Single-parameter Exponential family [8] [6]).

A single-parameter exponential family, denoted by $\mathcal{G}_1$, is a set of densities of the form

$$g(x|z) := g_0(x) \exp(z \cdot T(x) - A(z))$$

where g_0 , A and T are proper real-valued known functions and z is the real-valued "natural parameter" from the natural parameter space

$$\mathcal{Z}_1 := \left\{ z \mid \int g_0(x) \exp(z \cdot T(x)) dx < \infty \right\}.$$

For clarification $z \in \mathcal{Z}_1$ is in most cases a function which takes a single parameter $\theta \in \Theta \subset \mathbb{R}$ and transforms it accordingly. With this context the density function $g(\cdot|z)$ is then also depended on the parameters in the parameter space Θ , however, to simplify notation the parameters θ are not directly used but are implicitly expressed by z ; see [6].

This kind of notation will be used throughout this thesis, as the underlying parameter θ will not have any important role over the next chapters. For better understanding this can also be seen in example 3.2.

While g_0 , A and T are not specified in definition 3.1 every one of these functions must implicitly fulfill certain conditions. To uphold the defining requirements such that g is a density, g_0 must be non-negative while A is chosen such that

$$\int g(x|z) dx = \int g_0(x) \exp(z \cdot T(x)) \frac{dx}{\exp(A(z))} = 1$$

holds. This means $\exp(A(z))$ normalizes the remaining expression of $g(x|z)$. Therefore, A is referred to as the "normalizing function", while naturally A will be expressed through the "log-partition" function

$$z \mapsto \log \left(\int g_0(x) \exp(z \cdot T(x)) dx \right). \quad (8)$$

At last $T(x)$ will be called the "sufficient statistic" which can take various forms throughout this thesis; see [8] [6].

The construction of densities $g \in \mathcal{G}_1$ allows to express various new and already familiar densities by only defining the g_0 , T and z properly, while A is then given via (8) by plugging in accordingly. For example one can construct a normal density function of the form $\mathcal{N}(\mu, \sigma^2)$ by using the right functions and assuming either μ or σ as constant. The following example discusses this in more detail.

Example 3.2 (Normal distributions in $\mathcal{G}_1$ [6]).

Let σ take values in $\mathbb{R}_+$ and

$$f(x) := \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

be the density function for a real-valued random variable $X \sim \mathcal{N}(\mu, \sigma^2)$ with given $\mu \in \mathbb{R}$. Then by choosing

$$z := z(\sigma) = -1/(2\sigma^2), \quad T(x) := (x - \mu)^2 \quad \text{and} \quad g_0(x) := 1/\sqrt{2\pi}$$

one gets

$$A(z) = \log \left(\int \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) dx \right) = \log(\sigma)$$

by the definition of the log-partition. Thus,

$$g(x|z) = g_0(x) \exp(z \cdot T(x) - A(z)) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Recall that μ must be constant so that $g(x|z(\sigma)) \in \mathcal{G}_1$ holds, meaning z is constructed with only one parameter. Analogous to this, holding $\sigma > 0$ constant and letting μ be a real-valued parameter, one can choose

$$z := z(\mu) = \mu/\sigma^2, \quad T(x) := x \quad \text{and} \quad g_0(x) := \frac{1}{\sqrt{2\sigma^2\pi}} \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

resulting in

$$A(z) = \log \left(\int \frac{1}{\sqrt{2\sigma^2\pi}} \exp\left(-\frac{x^2}{2\sigma^2}\right) \exp\left(\frac{\mu}{\sigma^2}x\right) dx \right) = \frac{\mu^2}{2\sigma^2}$$

and furthermore in

$$g(x|z) = g_0(x) \exp(z \cdot T(x) - A(z)) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

The previous example showed that a single-parameter exponential family can reproduce some already known densities, however, one also recognizes that the usage of only one parameter might appear more restricting than necessary. To extend the term of the exponential family, the next definition introduces multi-parameter exponential families.

Definition 3.3 (Multi-parameter Exponential family [6]).

A k -parameter exponential family $\mathcal{G}_k$ is a set of densities of the form

$$g(x|z) := g_0(x) \exp\left(\sum_{j=1}^k z_j \cdot T_j(x) - A(z)\right) \quad (9)$$

where g_0 , A and $T_j \forall j \in \{1, \dots, k\}$ are real-valued known functions and z is the $\mathbb{R}^k$ -valued natural parameter from the natural parameter space

$$\mathcal{Z}_k := \left\{ z \mid \int g_0(x) \exp\left(\sum_{j=1}^k z_j \cdot T_j(x)\right) dx < \infty \right\}.$$

Definition 3.3 is very similar to definition 3.1 in the sense that the sufficient statistic $T := (T_1, \dots, T_k)^T$ and the natural parameter $z := (z_1, \dots, z_k)^T$ are now vectors with k components. This means also that the term $\sum_{j=1}^k z_j \cdot T_j(x)$ in (9) can be expressed via the scalar product $z^T \cdot T$. Also for readability purposes this thesis will often write g instead of $g(\cdot|z)$ where additionally the density g will at times be loosely referred to as the exponential family.

It is noteworthy that A still remains the log-partition function from (8) with $z \cdot T(x)$ now being the scalar product and where analogous to chapter 3.1 z still transforms parameters $\theta \in \Theta \subset \mathbb{R}^k$ accordingly; see [6]. To maintain continuity this thesis will simply write z instead of $z(\theta)$ like established before.

Example 3.2 established that one can write the density function for a normal distributed random variable as an element of $\mathcal{G}_1$ by holding either μ or σ fixed and using the other as a dependent parameter. Naturally, the multi-parameter approach for exponential families allows one to express the density in such a way so that neither μ and σ need to be constant.

Example 3.4 (Normal distributions in $\mathcal{G}_2$ [6]).

Let $X \sim \mathcal{N}(\mu, \sigma^2)$ with μ and σ^2 being real-valued parameters. Then by choosing

$$z := z(\mu, \sigma) = \begin{pmatrix} \mu/(\sigma^2) \\ -1/(2\sigma^2) \end{pmatrix}, \quad T(x) := \begin{pmatrix} x \\ x^2 \end{pmatrix} \quad \text{and} \quad g_0(x) := 1/\sqrt{2\pi}$$

one gets by definition of the log-partition function

$$A(z) = \log \left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{\mu^2}{2\sigma^2}\right) \int \sigma \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) dx \right) = \log(\sigma) + \frac{\mu^2}{2\sigma^2}$$

resulting in

$$g(x|z) := \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2\sigma^2} + \frac{x\mu}{\sigma^2} - \log(\sigma) - \frac{\mu^2}{2\sigma^2}\right) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

This means that the density of the normal distribution $g(x|z)$ with parameters μ and σ^2 is in $\mathcal{G}_2$, showing again the diverse usability of exponential families.

Note, that the assumption that θ and z have the same dimension does not always hold throughout literature, in most cases within this thesis the parameter space Θ is simply irrelevant or is not used at all. It is here important to recall that Θ was never part of definition 3.3 and is simply a vessel to use z accordingly, thus further details like the dimension of Θ will be omitted. The same can be said about the dimension of the natural parameter z as in practical uses later on, an exponential family will just be considered as a k -parameter multi-parameter exponential family, $\mathcal{G}$ for short, without considering the specific value of k . Analogous, the natural parameter space for k -dimensional natural parameter $\mathcal{Z}_k$ will be simply denoted by $\mathcal{Z}$. Nonetheless, while talking about exponential families formally the notation will maintain the same as established in definition 3.3.

3.2 Variations of Exponential Families

As exponential families mostly depend on the scalar product within the exponent, the definitions may vary throughout literature. Another popular way to express elements of $\mathcal{G}$ is by using a combination of matrices and vectors. In more detail, let z and $T(x)$ be of the following forms

$$z = (\lambda, \Lambda) \in \mathbb{R}^l \times \mathbb{R}^{w \times s} \quad \text{and} \quad T(x) = (\tau(x), \mathcal{T}(x)) \in \mathbb{R}^l \times \mathbb{R}^{w \times s} \quad (10)$$

where $l, w, s \in \mathbb{N}$. Then the inner product $z^T T(x)$ becomes

$$z^T T(x) := \lambda^T \tau(x) + \langle \Lambda, \mathcal{T} \rangle_F \quad (11)$$

with $\langle \cdot, \cdot \rangle_F$ being the Frobenius inner product. However, this notation is equivalent to the one used in definition 3.3 as both matrices Λ and $\mathcal{T}(x)$

can be simply flattened, resulting in $(w \cdot s)$ -dimensional vectors. Therefore, by replacing Λ and $\mathcal{T}(x)$ with their vectors respectively, the equation (11) will be reduced back down to the standard scalar product of two $(l + w \cdot s)$ -dimensional vectors. Considering this, although the natural parameter z may appear in different forms, the underlying definitions do not change. That said, practically the shape of the natural parameter does change the handling of the exponential family. This is why throughout this thesis the word *shape* may appear in context of natural parameters which loosely refers to their rough form or structure.

For demonstration, the following two examples, which will appear in practical uses later on, will show how the shape of the natural parameter can be used differently in already established densities.

Example 3.5 (Exponential distributions in $\mathcal{G}$ [19]).

Let $\theta > 0$ be a parameter. Then the pdf of the exponential distributed random variable $X \sim \text{Exp}(\theta)$ is defined by

$$p(x) = \mathbf{1}_{\{x \geq 0\}}(x) \theta e^{-\theta x}.$$

Naturally, $\text{Exp}(\theta)$ can be easily expressed by an exponential family by choosing

$$z := -\theta, T(x) := x \text{ and } g_0(x) := \mathbf{1}_{\{x \geq 0\}}(x)$$

while the resulting log-partition will be $A(z) = -\log(-z)$. By simply filling in one would get by construction

$$g(x|z) = \mathbf{1}_{\{x \geq 0\}}(x) \exp(zT(x) + \log(-z)) = p(x).$$

which also shows that the natural parameter z has to be smaller than 0.

Example 3.6 (Multivariate Normal distributions $\mathcal{G}$ [19]).

Let $\mu \in \mathbb{R}^d$ be the mean vector and $\Sigma \in \mathcal{S}_+^d$ the symmetric, positive semi-definite covariance matrix. Then the pdf of the d -dimensional random vector $X \sim \mathcal{N}(\mu, \Sigma)$ is defined by

$$p(x) = \left((2\pi)^d \det(\Sigma) \right)^{-\frac{1}{2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right).$$

Now using the notation from (10) one can choose

$$\lambda := \Sigma^{-1} \mu, \Lambda := -\frac{1}{2} \Sigma^{-1}, \tau(x) := x, \mathcal{T}(x) := x x^T \text{ and } g_0(x) := (2\pi)^{-\frac{d}{2}}$$

where the log-partition is defined via $A(z) = -\frac{1}{4} \lambda^T \Lambda^{-1} \lambda - \frac{1}{2} \log(\det(-2\Lambda))$. It can be seen that Λ is also symmetric, however, not positive semidefinite

but negative semidefinite. Still, by plugging into the exponential family and completing the square one would get

$$\begin{aligned}
g(x|z) &= (2\pi)^{-\frac{d}{2}} \exp \left(\lambda^T \tau(x) + \langle \Lambda, \mathcal{T}(x) \rangle_F + \frac{1}{4} \lambda^T \Lambda^{-1} \lambda + \frac{1}{2} \log(\det(-2\Lambda)) \right) \\
&= \left((2\pi)^d \det(\Sigma) \right)^{-1/2} \exp \left(\lambda^T \tau(x) + \langle \Lambda, \mathcal{T}(x) \rangle_F + \frac{1}{4} \lambda^T \Lambda^{-1} \lambda \right) \\
&= \left((2\pi)^d \det(\Sigma) \right)^{-1/2} \exp \left((\Sigma^{-1} \mu)^T x - \frac{1}{2} x^T \Sigma^{-1} x - \frac{1}{2} (\Sigma^{-1} \mu)^T \Sigma (\Sigma^{-1} \mu) \right) \\
&= \left((2\pi)^d \det(\Sigma) \right)^{-1/2} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right) = p(x)
\end{aligned}$$

which shows that expressing the natural parameter and the sufficient statistics with a combination of vectors and matrices can make the handling of more complex structures easier. Looking back at example 3.4 one could see that the proposed definitions there are just special cases of the setup above.

This chapter should have given a small insight on how versatile the natural parameter can be used. Although the rough structure seems different, both previous examples show that the underlying definitions for an exponential family still hold. It is also worth remembering that these examples not only show that one can recreate already known densities given the underlying parameters, like θ , μ and Σ , but also demonstrates that creating a new density can be achieved by only considering the properties and the shape of the natural parameter and sufficient statistic. This means that for example 3.6 one could simply define λ , Λ , τ , $\mathcal{T}$ and g_0 without considering μ and Σ and still get a multivariate normal distribution of some kind.

One can also expand the complexity of exponential families by introducing more than one random vector X . Intuitively one can imagine instead of just using one random vector, a sequence of random vectors is constructed each representing the state of an individual time point. The following section discusses this further in more detail and will show on how to expand exponential families using an adapted process.

3.3 Joint and Adapted Exponential Families

The aim of this section is to use equation (12) from below to create an adapted exponential family in such a way that a random vector $\mathbb{X} := (X_s)_{s=1}^S$ where $X_s := (X_{s,i})_{i=1}^d$ can be sampled from it.

Like the introduction of definitions 3.1 and 3.3 suggest one can also use an X with various components, meaning $X = (X_i)_{i=1}^d$. Still, what would happen if more than one observation was sampled? Let $\mathbb{X} := (X_s)_{s=1}^S$ for now be a sequence of S d -dimensional random vectors. Additionally, let

$g^{(s)}(x_s|z^{(s)}) \in \mathcal{G}_{k^{(s)}}$ be the for X_s constructed density of the exponential family such that $g^{(s)}(X_s|z^{(s)})$ is well defined. Then one can define a multi-parameter exponential family $g(x|z)$ for $\mathbb{X}$ via

$$\begin{aligned} g(x|z) &:= \prod_{s=1}^S g^{(s)}(x_s|z^{(s)}) \\ &= \left(\underbrace{\prod_{s=1}^S g_0^{(s)}(x_s)}_{=:g_0(x)} \right) \exp \left(\underbrace{\sum_{s=1}^S z^{(s)T} T^{(s)}(x_s)}_{=:z^T T(x)} - \underbrace{\sum_{s=1}^S A^{(s)}(z^{(s)})}_{=:A(z)} \right) \quad (12) \\ &= g_0(x) \exp(z^T T(x) - A(z)) \end{aligned}$$

where the components of $x := (x_s)_{s=1}^S$, $z := (z^{(s)})_{s=1}^S$ and $T(x) := (T^{(s)}(x_s))_{s=1}^S$ are stacked into vectors and with $A(z)$ still normalizing the rest of the expression. Indeed via Fubini (see A.8),

$$\begin{aligned} A(z) &= \log \left(\int g_0(x) \exp(z^T T(x)) dx \right) \\ &= \log \left(\int \dots \int \prod_{s=1}^S g_0^{(s)}(x_s) \exp(z^{(s)T} T^{(s)}(x_s)) dx_1 \dots dx_S \right) \\ &= \log \left(\prod_{s=1}^S \int g_0^{(s)}(x_s) \exp(z^{(s)T} T^{(s)}(x_s)) dx_s \right) \\ &= \sum_{s=1}^S \log \left(\int g_0^{(s)}(x_s) \exp(z^{(s)T} T^{(s)}(x_s)) dx_s \right) = \sum_{s=1}^S A^{(s)}(z^{(s)}) \end{aligned}$$

holds. This means, by following the definitions in (12) one gets functions $x \mapsto g_0(x)$, $x \mapsto T(x)$ and $z \mapsto A(z)$ for x and a natural parameter z . Thus, the constructed $g(x|z)$ for $\mathbb{X}$ w.r.t. $d(x_1 \otimes \dots \otimes x_S)$ is in the multi-parameter exponential family $\mathcal{G}$ according to definition 3.3. Here z is at maximum $\left(\sum_{s=1}^S k^{(s)}\right)$ -dimensional if the natural parameters $z^{(s)}$ and the sufficient statistics $T^{(s)}$ are different enough over all s .

It is important to mention that if $\mathbb{X} := (X_s)_{s=1}^S$ was sampled i.i.d. from $g^{(s)}(\cdot|z^{(s)}) \equiv \tilde{g}(\cdot|\tilde{z}) \in \mathcal{G}_{\tilde{k}}$, meaning

$$X_1, \dots, X_S \stackrel{\text{i.i.d.}}{\sim} \tilde{g}(\cdot|\tilde{z})$$

with $z^{(s)} \equiv \tilde{z}$ *not* changing over s , the resulting exponential family would still be $\mathcal{G}_{\tilde{k}}$. The reason is that $\sum_{s=1}^S z^{(s)T} T^{(s)}(x_s)$ would boil down to $\sum_{s=1}^S \tilde{z}^T T(x_s) = \tilde{z}^T \sum_{s=1}^S T(x_s)$. Thus, the scalar product would still be

conducted with $\tilde{k}$ -dimensional vectors, underlining that $g(\cdot|z)$ is not always $\left(\sum_{s=1}^S k^{(s)}\right)$ -dimensional; see [8].

The construction of such density in (12) will be important in coming chapters as one can reverse the equation and split the "greater" exponential family into "smaller" ones, given that the underlying random vector does satisfy the proper conditions.

Creating an adapted exponential family

Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space where the filtration $\mathbb{F}$ is the natural filtration of $\mathbb{X}$. Meaning, $\mathbb{F} := (\mathcal{F}_s)_{s=0}^S := (\sigma(X_1, \dots, X_s))_{s=1}^S$ with the terminal σ -algebra being given by $\mathcal{F} = \mathcal{F}_S$ and $\mathcal{F}_0$ by the trivial σ -algebra $\{\emptyset, \Omega\}$. Also let

$$g(x|z) = \prod_{s=1}^S g^{(s)}(x_s|z^{(s)})$$

be the exponential family from equation (12). However this time, let the natural parameters $z^{(s)}$ be predictable over all s , formally $z^{(s)}$ is $\mathcal{F}_{s-1}$ -measurable for all $s = 1, \dots, S$; see [9]. This thesis will refer to such densities g as *adapted exponential families*.

Intuitively one can also imagine that each $z^{(s)}$ is a deterministic function taking $(X_1, \dots, X_{s-1})$ as input where $z^{(1)}$ is trivial, meaning a constant value. Loosely speaking, this also means that the parameter space Θ gets replaced by the sample space with respect to previous sampled values. This intuition is also supported by Doobs Functional Representation Lemma A.9. Thus, from now on $z^{(s)}$ will be referred to as deterministic function

$$z^{(s)} : (X_1, \dots, X_{s-1}) \mapsto z^{(s)}(X_1, \dots, X_{s-1})$$

as well as the $\mathcal{F}_{s-1}$ -measurable random vector. Given this information one can show that g is still a proper density w.r.t. the Lebesgue-measure.

Proposition 3.7.

The adapted exponential family $g(\cdot|z)$ described above is a density w.r.t. to the Lebesgue-measure.

Proof. It is sufficient to show that g fulfills the normalization argument as $g \geq 0$ clearly holds (see A.2). As $z^{(s)}$ can be written as $z^{(s)}(x_1, \dots, x_{s-1})$ and the integral is independent of $x_1, \dots, x_{s-1}$

$$\int g^{(S)}(x_S|z^{(S)}(x_1, \dots, x_{S-1})) dx_S = 1$$

holds. Via Fubini (see A.8) the following equation holds as well

$$\begin{aligned}
\int g(x|z)dx &= \int_{\mathbb{R}^S} \prod_{s=1}^S g^{(s)}(x_s|z^{(s)}(x_1, \dots, x_{s-1}))dx \\
&= \int_{\mathbb{R}^{S-1}} \left(\int_{\mathbb{R}} g^{(S)}(x_S|z^{(S)}(x_1, \dots, x_{S-1}))dx_S \right) \\
&\quad \cdot \prod_{s=1}^{S-1} g^{(s)}(x_s|z^{(s)}(x_1, \dots, x_{s-1}))d(x_1, \dots, x_{S-1}) \\
&= \int_{\mathbb{R}^{S-1}} \prod_{s=1}^{S-1} g^{(s)}(x_s|z^{(s)}(x_1, \dots, x_{s-1}))d(x_1, \dots, x_{S-1}).
\end{aligned}$$

Given that, the resulting setup for $S - 1$ is analogous to the one for S . Repeating the same argument by using induction, the whole equation reduces down to 1, proofing $\int g(x|z)dx = 1$ and, therefore, normalization. $\square$

It is here important to note that adapted exponential families are still exponential families according to definition 3.3, as the natural parameters $z^{(s)}$ must still fulfill

$$\int g_0^{(s)}(x_s) \exp\left(z^{(s)T} T(x_s)\right)dx < \infty.$$

As g can now be considered a density, one could also sample from it meaning $\mathbb{X} = (X_1, \dots, X_S) \sim g(\cdot|z)$. To do so one must simply follow this procedure inspired by [5].

Algorithm 3.8.

Let g be given by (12) where $z^{(s)} := z^{(s)}(X_1, \dots, X_{s-1}) \forall s$. Then $\mathbb{X}$ can be sampled by using the following steps.

- 1) Sample X_1 from $g^{(1)}(\cdot|z^{(1)})$
- 2) For $s = 2, \dots, S$ do:
 - Calculate $z^{(s)}$ given the previous sampled values $X_1, \dots, X_{s-1}$
 - Sample X_s from $g^{(s)}(\cdot|z^{(s)})$
- 3) Construct $\mathbb{X} := (X_1, \dots, X_S)$

4 Introducing Adapted Exponential Families in IS

Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space generated by the random vector $\mathbb{X}$. In particular, let $\mathbb{X} := (X_s)_{s=1}^S$ be a random vector, where $X_s := (X_{s,i})_{i=1}^d \sim p$ is a d -dimensional random vector distributed according to p , and $\mathbb{F}$ be the natural filtration of $\mathbb{X}$. Furthermore, assume that $\mathbb{X}$ has i.i.d. components under P and $\mathbb{X} \sim \bar{p} := \prod_{s=1}^S p(x_s)$.

Choosing this setup the aim of this chapter will be to explore IS for $\mathbb{X}$ using adapted exponential families. More specifically, let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a real-valued outer function for $\mathbb{X}$ and $\mu_f := E[f(\mathbb{X})]$ the expectation to be approximated. Then by chapter 2 the MCE which approximates the expectation is given by

$$\mu_f^{(M)} := \sum_{m=1}^M f(\mathbb{X}^{(m)})$$

for samples $(\mathbb{X}^{(m)})_{m=1}^M$, $\mathbb{X}^{(m)} \sim \mathbb{X}$. Combining previous chapters 2 and 3, one can conduct Importance Sampling over the conditional exponential family. Formally, this chapter aims to find a $q \in \mathcal{G} \cap \mathcal{Q}$ such that for $I = \bar{p}/q$

$$\text{Var}_q(\mu_{f,I}^{(M)}) \leq \text{Var}_{\bar{p}}(\mu_f^{(M)})$$

holds or even such that q minimizes $\text{Var}_q(\mu_{f,I}^{(M)})$ fully. Note that for now q can be considered as being not adapted.

As $\text{Var}_q(\mu_{f,I}^{(M)}) = \frac{1}{M} (E_q[(I(\mathbb{X})f(\mathbb{X}))^2] - (\mu_f)^2)$ holds by (3) where μ_f is constant and M is assumed to be a fixed sample size, the optimization problem can be equivalently written as

$$\text{minimize} \quad E_{\bar{p}} \left[\frac{\bar{p}(\mathbb{X})}{q(\mathbb{X}|z)} f(\mathbb{X})^2 \right] \quad \text{over } q \in \mathcal{G} \cap \mathcal{Q}. \quad (13)$$

by shifting the densities back to $\bar{p}$.

Plugging $q(x|z) := q_0(x) \exp(z^T T(x) - A(z))$ in above one gets

$$E_{\bar{p}} \left[\frac{\bar{p}(\mathbb{X})}{q(\mathbb{X}|z)} f(\mathbb{X})^2 \right] = E_{\bar{p}} \left[\bar{p}(\mathbb{X}) q_0(\mathbb{X})^{-1} e^{(A(z) - z^T T(\mathbb{X}))} f(\mathbb{X})^2 \right].$$

Naturally, one could minimize over q_0 , T , z and Θ accordingly. However, as $q \in \mathcal{G} \cap \mathcal{Q}$ does not necessarily give much restrains on how to choose these functions and sets, optimizing over them would be quite ambiguous and could possibly require a lot of computing time. This could make IS

obsolete, as simply using the same time to generate more samples and conduct Monte Carlo with the extra information would possibly yield similar or better results.

Another problem is the calculation of the integral. For one, it could also take a lot of computing time, especially when optimizing is involved. Whereas the reason of the calculation might also be questionable, as the aim was to find μ_f which itself is an integral, possibly one that is easier to acquire as the one described in (13).

Thus, another goal will be to conduct IS more efficiently and to solve the aforementioned problems in greater detail. The first step would be rewriting the optimization problems, most notably by using Monte Carlo on (13) and minimizing

$$\frac{1}{M} \sum_{m=1}^M \bar{p}(\mathbb{X}^{(m)}) (q_0(\mathbb{X}^{(m)}))^{-1} e^{(A(z) - z^T T(\mathbb{X}^{(m)}))} (f(\mathbb{X}^{(m)}))^2 \quad (14)$$

over $q \in \mathcal{G} \cap \mathcal{Q}$ for $\mathbb{X}^{(m)} \sim \bar{p}$. It is important to note that M in (14) is *not* necessarily the same used for $\mu_{f,I}^{(M)}$, but rather is another sample size generated for this specific Monte Carlo scenario.

4.1 Creating a Convex Optimization Problem

Having convexity in an optimization problem is always an advantage as solving the problem becomes much more efficient and reliable. Various algorithms and methods to find the solutions to such problems have already been established and are not scarce in the world of mathematics; see [4].

Having said this, throughout this section this thesis will discuss a way to reformulate the problem (14) in such a way that it becomes convex. To achieve this the following proposition will be useful.

Proposition 4.1.

Let $q \in \mathcal{G}$, $z \in \mathcal{Z}$ and p be a density. Then the function defined by

$$z \mapsto p(x) q_0(x)^{-1} e^{(A(z) - z^T T(x))} f(x)^2$$

is convex.

Proof. Since $f(x)^2$, $p(x)$ and $q_0(x)$ are non-negative it is sufficient to show that $z \mapsto e^{(A(z) - z^T T(x))}$ is convex.

First, one can show via the Hölder inequality (see section A.10) that $A(z)$ is convex in z . Therefore, let $\xi \in [0, 1]$ and $z, \tilde{z} \in \mathcal{Z}$. By defining

$F(z) := \int \exp(z^T T(x)) q_0(x) dx$, $A(z) = \log(F(z))$ holds. Furthermore, by applying the Hölder inequality w.r.t. to measure $q_0(x) dx$ one gets

$$\begin{aligned} F(\xi z + (1 - \xi)\tilde{z}) &= \int e^{\xi z^T T(x)} e^{(1-\xi)\tilde{z}^T T(x)} q_0(x) dx \\ &\leq \left(\int |e^{\xi z^T T(x)}|^{\frac{1}{\xi}} q_0(x) dx \right)^\xi \left(\int |e^{(1-\xi)\tilde{z}^T T(x)}|^{\frac{1}{1-\xi}} q_0(x) dx \right)^{(1-\xi)} \\ &= F(z)^\xi F(\tilde{z})^{(1-\xi)}. \end{aligned}$$

Thus, by the rules of the logarithm

$$A(\xi z + (1 - \xi)\tilde{z}) = \log(F(\xi z + (1 - \xi)\tilde{z})) \leq \xi A(z) + (1 - \xi)A(\tilde{z}).$$

As $-z^T T(x)$ is a linear term and the log-partition $A(z)$ is convex, $A(z) - z^T T(x)$ is also convex in z . The resulting $e^{(A(z) - z^T T(x))}$ is therefore convex in z as $y \mapsto e^y$ is a convex and strictly monotonically increasing function. Thus, proposition 4.1 is shown. Note that the convexity of $A(z)$ is also shown in [8]. $\square$

Using proposition 4.1 one could formulate the optimization problem (14) such that minimizing the expression would not be conducted over the parameter space Θ , but instead would take place over another set of parameters which would be multiplied with z component-wise. It should be noted in the case of z being of constant value this would be equivalent to optimizing over the natural parameter space. However, if the optimization process is being conducted over a separate parameter space, the defining components of q , meaning q_0 , T and z , must be found/defined beforehand. The basis on how to choose these components will be discussed further in chapter 4.2, for now it can be assumed that all three q_0 , T and z are given and are unchangeable.

As Θ is no longer part of the optimization problem and z , T and q_0 are assumed to be chosen accordingly before minimizing, one could define a few vectors of parameters $\alpha^{(s)} := (\alpha_j^{(s)})_{j=1}^{k^{(s)}} \in \mathcal{A}^{(s)} \subset \mathbb{R}^{k^{(s)}}$ accordingly to the $k^{(s)}$ -dimensional natural parameter $z^{(s)}$ for all $s = 1, \dots, S$. By using the component-wise product for each s , one could define new natural parameters

$$z_\alpha^{(s)} := \alpha^{(s)} \odot z^{(s)} := (\alpha_1^{(s)} z_1^{(s)}, \dots, \alpha_{k^{(s)}}^{(s)} z_{k^{(s)}}^{(s)}) \text{ and } z_\alpha := (z_\alpha^{(s)})_{s=1}^S,$$

as well as the whole parameter $\alpha := (\alpha^{(s)})_{s=1}^S \in \mathcal{A}^{(1)} \times \dots \times \mathcal{A}^{(S)} =: \mathcal{A}$ where $\mathcal{A}$ needs to be convex. Here $\odot$ describes the component-wise multiplication between two vectors of the same dimension.

With these definitions, proposition 4.1, (14) and by getting rid of the constant fraction, one gets the convex optimization problem

$$\text{minimize} \quad \sum_{m=1}^M \bar{p}(\mathbb{X}^{(m)}) \left(q(\mathbb{X}^{(m)} | z_\alpha) \right)^{-1} (f(\mathbb{X}^{(m)}))^2 \quad \text{over } \alpha \in \mathcal{A} \quad (15)$$

where $\mathbb{X}^{(m)} \sim \bar{p}$. Also by using this notation $q(\mathbb{X}^{(m)}|z_\alpha)$ can be written as

$$q_0(\mathbb{X}^{(m)}) \exp \left(\sum_{s=1}^S z_\alpha^{(s)T} T^{(s)}(\mathbb{X}_s^{(m)}) - A^{(s)}(z_\alpha^{(s)}) \right).$$

4.2 Choosing the Type of Exponential Family

While problem (15) seems well motivated by choosing a convex model to optimize, the choices of T , q_0 and z are restricted by practical application. The first problem arises from the endless possibilities on how to choose them, while the second problem is based on the practical application of integration algorithms. While q must not be integrated, the log-partition function A within q is defined by the integral. As the integral is commonly known to be linear and therefore does not influence the convexity of the problem in theory, however, the practical methods and algorithms which approximate the integral might not be linear at all. This could result in longer computing time, an error while running the code or violating the convexity all together.

Yet, while restricting the outcome this issue can simply be avoided by choosing already established densities which the exponential family can represent and are in $\mathcal{G} \cap \mathcal{Q}$. There is already a lot of literature describing the choices one could make for T , q_0 , z and the log-partition function A , f.e. [19]. This was already seen in the previous chapters where example 3.2 and 3.4 show the particular solution of $A(z)$ for the normal distribution with respect to the given parameters. As proposition 4.1 already proved the convexity of the problem (15) still holds when simply plugging in the particular solution of $A(z)$, thus, eliminating the practical integration problems.

Although this somewhat restricts the choices of T , q_0 and z , the amount of available densities which can be represented by exponential families is far too large. To even attempt optimizing over a reasonable number of these densities, one would require a library which includes the final forms of the log-partition function A .

As building such a library would require a lot of work and would exceed the purpose of this thesis, this work proposes choosing the density q as follows.

Rules 4.2.

In order to choose the right q for the IS w.r.t. $\bar{p}$ one can follow points 1), 2) and 3) from below.

- 1) q must be in $\mathcal{G} \cap \mathcal{Q}$.
- 2) q must be of similiar type to the density $\bar{p}$, meaning if for example $\bar{p}$ is $\mathcal{N}(0, 1)$, then q is to be chosen normally distributed.
- 3) Point 2) can be disregarded if another choice for q is clearly a much better fit.

Although 2) and 3) from rules 4.2 are both subjectively created specifically for this study, point 3) is purely subjective and exists only to allow more leniency while trying new densities. Note that 2) does restrict the choices of T , q_0 and z a lot and sometimes even results in an easier application, as dividing $\bar{p}$ by q occasionally allows for terms to cancel out. This would not necessarily enhance the optimization, however, in many cases it would not make it any worse, because if the whole term $I = \bar{p}/q$ would cancel out to 1, the resulting MCE would simply be $(1/M) \sum_{m=1}^M f(\mathbb{X}^{(m)})$ which was the original setting. Also such a structure can be seen as a "safe switch", because even though a really bad sample $\mathbb{X}^{(m)}$ in (15) could corrupt the whole solution, some cancellations might aid to prevent such occasions. On how effective rules 4.2 really are stand to be explored.

It is also noteworthy that when talking about the choices of T , q_0 and z , that T and q_0 are the only functions which will be defined explicitly. The shape and general rough structure of natural parameter z still given by the density, however, what each component explicitly does is not given at all.

For example, in example 3.2 z was given by $z(\sigma) = -1/(2\sigma^2)$, but choosing $z(\sigma) = \sigma^2$ would still have been a normal distribution. Indeed, a normal distribution of another kind, however, in the context IS and point 2) of rules 4.2, such conversion would still be valid. As the shape of the natural parameter is more important than the actual individual values of the components, rules 4.2 would still apply if one would choose the exponential family as adapted.

4.3 Creating an Adapted Exponential Family for IS

As the previous chapter 4.2 established optimizing over $q \in \mathcal{G} \cap \mathcal{Q}$ is counterproductive, therefore one must define q_0 , T , z and Θ (if z was not chosen as a constant function) beforehand to permit a well-defined problem.

Throughout this section it is assumed that the kind of exponential family was already chosen. Thus, q_0 and T are fully defined while z is partially given

by the shape. Now the natural parameter must be defined component-wise to enable a fully functional exponential family.

This thesis proposes the usage of adapted exponential families from chapter 3.3 which would make the Θ obsolete and would "only" require to define deterministic functions for $z^{(s)}$ such that $z^{(s)} := z^{(s)}(X_1, \dots, X_{s-1})$ becomes a predictable random vector within $\mathcal{Z}_{k^{(s)}}$ for all s .

To recap, section 3.3 showed that q can be broken down into smaller densities

$$\begin{aligned} q(x|z) &:= \prod_{s=1}^S q^{(s)}(x_s|z^{(s)}) \\ &= \left(\prod_{s=1}^S q_0^{(s)}(x_s) \right) \exp \left(\sum_{s=1}^S (z^{(s)T} T^{(s)}(x_s)) - \sum_{s=1}^S A^{(s)}(z^{(s)}) \right). \end{aligned} \quad (16)$$

where each $z^{(s)}$ is $\mathcal{F}_{s-1}$ -measurable or by applying Doobs Functional Representation Lemma each $z^{(s)}$ is deterministic taking $(X_1, \dots, X_{s-1})$ as input. One should note that for all s , $z^{(s)} : \Omega \mapsto \mathbb{R}^{k^{(s)}}$ is now considered to be a random vector where the natural parameter space $\mathcal{Z}_{k^{(s)}}$ does allow a well-defined normalization for all feasible situations in Ω .

However, as z becomes a random vector the density property of q might collapse, resulting in IS not being feasible anymore. Although proposition 3.7 already suggested that the adapted exponential family is a density, the following proposition will also show in further detail that the proposed structure allows the usage of IS in the context of Radon-Nikodym.

Proposition 4.3.

If $q \in \mathcal{G}$ is an adapted exponential family, meaning it has the proposed form from (16) and $z := (z^{(s)})_{s=1}^S$ is an element of

$$\left\{ z | z^{(s)} \in \mathcal{Z} \text{ and } z^{(s)} \text{ } \mathcal{F}_{s-1}\text{-measurable } \forall s \in \{1, \dots, S\} \right\}$$

then $\bar{p}(\mathbb{X})/q(\mathbb{X}|z)$ is still a Radon-Nikodym density w.r.t. $P_q := P_{q(\cdot|z)}$.

Proof. To show $\bar{p}(\mathbb{X})/q(\mathbb{X}|z)$ that is a Radon-Nikodym density it is sufficient to show that $\bar{p}(\mathbb{X})/q(\mathbb{X}|z) \geq 0$ and $E_{P_q} \left[\frac{\bar{p}(\mathbb{X})}{q(\mathbb{X}|z)} \right] = 1$. The first aspect is clearly given as $\bar{p}$ and q are densities with respect to Lebesgue measure. The second position will be proven by using mathematical induction under consideration that $x_s \mapsto q^{(s)}(x_s|z^{(s)}), p(x_s)$ are densities respectively.

base case $S = 1$: If $S = 1$ then $z = z^{(1)}$. Therefore, by construction z is $\mathcal{F}_0 = \{\emptyset, \Omega\}$ -measurable which further means z is constant. This case is

completely equivalent to the initial exponential family density used in section 3, therefore by (7) the base case is proven.

induction step $S - 1 \rightarrow S$: Due to the tower property (see A.5) and $z^{(s)}$, X_s being $\mathcal{F}_{S-1}$ -measurable $\forall s \leq S - 1$ the following holds:

$$\begin{aligned}
E_{P_q} \left[\frac{\bar{p}(\mathbb{X})}{q(\mathbb{X}|z)} \right] &= E_{P_q} \left[\prod_{s=1}^S \frac{p(X_s)}{q^{(s)}(X_s | z^{(s)})} \right] \\
&= E_{P_q} \left[\left(\prod_{s=1}^S p(X_s) \left(q_0^{(s)}(X_s) \right)^{-1} \right) \exp \left(\sum_{s=1}^S \left(A^{(s)}(z^{(s)}) - z^{(s)} T^{(s)}(X_s) \right) \right) \right] \\
&= E_{P_q} \left[\left(\prod_{s=1}^{S-1} p(X_s) \left(q_0^{(s)}(X_s) \right)^{-1} \right) \exp \left(\sum_{s=1}^{S-1} \left(A^{(s)}(z^{(s)}) - z^{(s)} T^{(s)}(X_s) \right) \right) \right. \\
&\quad \cdot E_{P_q} \left[p(X_S) \left(q_0^{(S)}(X_S) \right)^{-1} \exp \left(A^{(S)}(z^{(S)}) - z^{(S)} T^{(S)}(X_S) \right) \middle| \mathcal{F}_{S-1} \right] \Bigg].
\end{aligned} \tag{17}$$

Looking at conditional expectation in the above equation one could see that

$$\begin{aligned}
&E_{P_q} \left[p(X_S) \left(q_0^{(S)}(X_S) \right)^{-1} \exp \left(A^{(S)}(z^{(S)}) - z^{(S)} T^{(S)}(X_S) \right) \middle| \mathcal{F}_{S-1} \right] \\
&= \int \frac{p(x_S)}{q(x_S | z^{(S)})} q(x_S | z^{(S)}) dx_S = 1 \text{ a.s.}
\end{aligned} \tag{18}$$

holds, due the assumption that under P_q the conditional distribution of $X_S | \mathcal{F}_{S-1}$ is given by $q^{(S)}(\cdot | z^{(S)})$ which holds as q can be factorized accordingly and $z^{(S)}$ is $\mathcal{F}_{S-1}$ -measurable. In other words as the law of $\mathbb{X} = (X_1, \dots, X_S)$ is given by the adapted exponential family $q(\cdot | z)$ under P_q , the resulting law of $X_S | \mathcal{F}_{S-1}$ is given via $q^{(S)}(\cdot | z^{(S)})$. Intuitively one can think back to algorithm 3.8.

Thus, plugging the result of (18) in (17) the proposition has been proven, because the case of $S - 1$ is assumed to be true by induction assumption. $\square$

Choosing and defining the natural parameter

Consequently, the current aim is to find fitting functions to implement them to the components of the natural parameter as the inputs are already given by $\mathbb{X}$. To define the natural parameter z correctly one must consider the shape and the underlying necessary functions.

This study proposes the usage of two functions to define a single $z_i^{(s)}$ for $i = 1, \dots, k^{(s)}$ and $s = 2, \dots, S$. Consider beforehand that both those functions were subjectively defined specifically for this thesis as they will be applied in practice later on.

Definition 4.4 (Type and Combination function).

A function

$$\text{type} : \mathbb{R} \rightarrow \mathbb{R}, \quad x \mapsto \text{type}(x).$$

which specifies the structural form of the component $z_i^{(s)}$ is called the type function.

For $s = 2, \dots, S$, a function

$$\text{comb}^{(s)} : \mathbb{R}^{(s-1) \times d} \rightarrow \mathbb{R}, \quad (x_1, \dots, x_{s-1}) \mapsto \text{comb}^{(s)}(x_1, \dots, x_{s-1}).$$

which combines $X_1, \dots, X_{s-1}$ into the components of $z^{(s)}$ in such a way that the result is $\mathcal{F}_{s-1}$ -measurable, is called the (s -th) combination function.

Although the combination function alone would be enough to define each $z_i^{(s)}$, this thesis proposes to keep $\text{comb}^{(s)}$ as simple as possible to avoid complex structures. Instead, the idea is merging type and combination functions in such a way that the resulting $z_i^{(s)}$ inherits the complex structures from the type functions and $\mathcal{F}_{s-1}$ -measurability properties from the combination functions. It is noteworthy that "merging" does *not* explicitly refer to the concatenation " $\text{type} \circ \text{comb}^{(s)}$ " but rather means that several type and combination functions can be used in any constructive way so that the resulting $z_i^{(s)}$ can be considered well-defined and proper for its use.

Furthermore, the above criteria were not given for $z^{(1)}$ as it is considered as trivial. This means that as long as the density properties are not violated one can choose some fitting constant components. In some cases simply choosing $z^{(1)} = (1, \dots, 1)^T$ would be enough, as $\alpha_i^{(1)} \odot z_i^{(1)} \propto \alpha_i^{(1)}$ holds for all i .

The following examples will show some type functions which will be used later on in practical applications. This thesis proposes the usage of already established functions in fields like Machine Learning and functions which are suited for the models. Still, which type functions really fit the models has to be found out via deeper analysis. The following examples are therefore chosen purely subjectively and do not follow any specific criteria.

Example 4.5 (type: Absolute value, identity and constant function).

The most basic versions of a type function are the simple identity function

$$\text{id}(x) := x,$$

the absolute value

$$\text{abs}(x) := |x|$$

and the constant function

$$\text{const}(x) := \beta$$

for $\beta \in \mathbb{R}$.

Example 4.6 (type: ReLU, capped ReLU Leaky ReLU and shifted ReLU).

The first proposed function is known as ReLU (Rectified Linear Unit) which can be written as

$$\text{ReLU}(x) := \max(0, x)$$

where

$$\text{cappedReLU}(x) := \min(\max(0, x), \beta), \beta \in \mathbb{R}$$

is the capped version of it. While ReLU does not output negative values, the second proposed function

$$\text{LReLU}(x) := \begin{cases} x\beta & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases},$$

given some β does allow to do so. Such function will be called the Leaky ReLU; see [17]. Some literatures like [17] suggest $\beta = 0.01$ which slightly restricts the impact of negative values.

While both Leaky ReLU and ReLU allow to take 0 as a value, some situations might not allow such conversion, f.e. a normal distribution $\mathcal{N}(\mu, \sigma^2)$ does not consider $\sigma = 0$ to be feasible. This thesis, therefore, suggests to shift the ReLU function simply to the right by a small enough value $\beta > 0$ which is still feasible for some calculations but is close enough to 0, formally

$$\text{ReLU}_\beta(x) := \begin{cases} \beta & \text{if } x < \beta \\ x & \text{if } x \geq \beta \end{cases}.$$

Example 4.7 (type: Clipped density).

In case of normal distributions $\mathcal{N}(\mu, \sigma^2)$, this works suggests to use a slightly altered version of the pdf p . As $p > 0$ holds, one could choose some $\beta > 0$ and then clip the density via

$$p_\beta(x) := \begin{cases} p(x) & \text{if } x \in [\mu - \beta\sigma, \mu + \beta\sigma] \\ p(\mu + \beta\sigma) & \text{else} \end{cases}$$

which considers values x that are within some feasible interval $[\mu - \beta\sigma, \mu + \beta\sigma]$ much more than values that are not.

However, in practical cases the implementation must be done with caution as $p_\beta \xrightarrow{\beta \rightarrow \infty} p$ holds. Some bad samples could corrupt the function to output numbers indistinguishable to 0, which may trigger errors. Due to this, p_β does take the constant $p(\mu + \beta\sigma)$ outside of $[\mu - \beta\sigma, \mu + \beta\sigma]$.

Indeed, such implementation can be conducted with various densities, for example $\text{Exp}(\theta)$, however, it must be altered with caution to prevent similar problems as discussed before.

For simplicity reasons the only combination function used in this thesis would be the mean. However, one must consider that the choice on how to use the mean is directly dependent on the shape of the natural parameter. Thus, the desired combination function must be chosen with caution. The following example will demonstrate a specific case on how to use the mean properly. Note that the proposed method will be used later on in practical application.

Example 4.8 (comb: mean).

Let $z^{(s)}$ be a d -dimensional random vector for all s , i.e. $z^{(s)}$ has the same dimension as X_s . Then the combination function can be chosen as

$$\text{comb}_i^{(s)}(x_1, \dots, x_{s-1}) := \text{mean}(x_{1,i}, \dots, x_{(s-1),i}) := \frac{1}{s-1} \sum_{\bar{s}=1}^{s-1} x_{\bar{s},i} \quad (19)$$

where $\text{comb}_i^{(s)}$ is the combination function responsible for the i -th component of $z^{(s)}$ for $s = 2, \dots, S$.

Moreover, the above equation (19) is *not* strictly referring to $x_{\bar{s},i}$ being a value of $X_{\bar{s},i}$, but rather should demonstrate a more general case. The next example will demonstrate what this thesis understands by "combining" type and combination functions.

Example 4.9.

Let $z^{(s)}$ be a d -dimensional random vector for all s . Also let $\text{type}_i^{(s)}$ be type functions and let $\text{comb}_i^{(s)}$ be the combination function given by the mean according to (19) for $s = 2, \dots, S$ and $i = 1, \dots, d$. Then the s -th natural parameter $z^{(s)}$ given by

$$z_i^{(s)} := \text{type}_i^{(s)} \left(\text{comb}_i^{(s)}(X_1, \dots, X_{s-1}) \right) \quad (20)$$

is $\mathcal{F}_{s-1}$ -measurable. On the other hand let $\text{type}_i^{(s)}$ be type functions responsible $X_{s,i}$ and let

$$X_s^{(\text{type})} := \left(\text{type}_i^{(s)}(X_{s,i}) \right)_{i=1}^d$$

for $s = 2, \dots, d$. Then the s -th natural parameter $z^{(s)}$ given by

$$z_i^{(s)} := \text{comb}_i^{(s)}(X_1^{(\text{type})}, \dots, X_{s-1}^{(\text{type})}) \quad (21)$$

is also $\mathcal{F}_{s-1}$ -measurable where $\text{comb}_i^{(s)}$ is again given by (19).

The examples above illustrate that every type and combination function can be chosen differently, which results in an indexation nightmare. Consequently, practical applications in later chapters will stay more consistent

with the selection of type and combination functions. This means that in most cases there exists one type function type for which holds

$$\text{type} = \text{type}_i^{(s)}$$

for all $s = 1, \dots, S$, $i = 1, \dots, d$ whereas the combination function will always be chosen as the mean.

To summarize, throughout chapter 4.1 the optimization problem (14) was reduced down to the convex problem (15). Furthermore, in chapter 4.2 the choice for a feasible $q \in \mathcal{G} \cap \mathcal{Q}$ was also restricted by applying rules 4.2, allowing to specify q_0 and T fully while z was only partially defined by its shape. Lastly, section 4.3 constructed an adapted exponential family with respect to the previous chapter 4.2, which proposed methods to define each natural parameter specifically. Yet, this thesis recognizes that these choices for z were only one of many ways on how to approach adaptability. In theory as well as in reality one would have considerably more functions available to define the natural parameter. However, considering the extent of this work, the future chapters will only test and use the proposed examples or at least a slight variation of them. Further it is notable that the whole section 4 will be highly relevant in the coming chapters, as every program and practical application will be implemented according to its contents.

5 Implementation of Adapted Exponential Families in IS

Given the results in the previous chapter 4, this section will construct a model in Python which conducts IS accordingly. To develop such a model this thesis proposes a structure which includes two components, each being mainly represented by a class.

The first component manages the construction and form of the natural parameter z according to chapter 4.2 and 4.3, whereas the second component utilizes z including other inputs to find the optimal α , α_{opt} , solving the optimization problem (15). If the α_{opt} was successfully calculated, the second component would return a sampler and the desired density function. The IS process can then be implemented in a third program constructed by the user according to their needs.

Another goal of this study is to construct the proposed classes with extensibility in mind. Thus both classes are modeled so that they are compatible for future projects. In the best possible case, new types of exponential families can be added by simply copying already established exponential families and changing them slightly according to the intended use.

The following two subsections will explain the code in more detail by taking the multivariate normal and the exponential distribution from examples 3.5 and 3.6. To enhance readability, parts of the code which this thesis deems as insignificant will be omitted in the following chapters. For more details regarding certain parts of code see section A.4 in the Appendix. Moreover, to allow a more intuitive reading comprehension this thesis will use a mixture of code and mathematical notations, trying to convey the thought behind the code to the reader. Thus, the notation from the previous chapter will be continued over next few sections.

5.1 Construction of the Natural Parameter: Code Implementation

In this section the construction of the natural parameter z according to chapter 4.3 will be elaborated further.

As previously established, when defining the natural parameter in practice, the most important part is to find the right shape. The file "class_natural_parameter_components.py" (see section A.4.1), which includes a class called `natural_parameter_definer`, was designed to consider such aspects and will act as the first component of the whole model. The core libraries used within this file include `numpy`, `scipy` and `random` from the

Python standard library; see [23] [24] [11]. However, the importance of random is questionable as the code would still work without it.

The following code shows the beginning of class `natural_parameter_definer`

```

40 class natural_parameter_definer(object):
41     def __init__(self, typ=None, beta=None):
42         def shiftReLUbeta(x): return shiftReLU(x, beta)
43         def LReLUbeta(x): return LReLU(x, beta)
44         def id(x): return x
45         def const(x): return beta
46         def cappedReLUbeta(x): return cappedReLU(x, beta)
47         def stdNormal(x): return standardNormal(x, beta)
48         def stdExp_dist(x): return standardExp_dist(x, beta)
49
50         if typ == "ReLU":
51             self.typ = ReLU
52
53         elif typ == "LReLU":
54             self.typ = LReLUbeta
55
56         elif typ == "shiftReLU":
57             self.typ = shiftReLUbeta
58
59         elif typ == "id":
60             self.typ = id
61
62         elif typ == "constant":
63             self.typ = const
64
65         elif typ == "cappedReLU":
66             self.typ = cappedReLUbeta
67
68         elif typ == "stdNormal":
69             self.typ = stdNormal
70
71         elif typ == "abs":
72             self.typ = abs
73
74         elif typ == "stdExp_dist":
75             self.typ = stdExp_dist
76
77         else:
78             self.typ = random.choice([ReLU, const,
                                     shiftReLUbeta, LReLUbeta, cappedReLUbeta, id,
                                     abs])

```

Python Code 2: `class_natural_parameter_components` part 1

Code 2 shows the dunder method `__init__` which takes advantage of the functions defined outside the class. However, as these functions are very simple in nature, they will be excluded from this code demonstration. To keep this class as simple as possible `natural_parameter_definer` does only take two inputs. The first input, the variable `typ`, is the name of the type function from definition 4.4 as a string. By conversion `typ` is set to `None`

per default. If `typ` was initialized with the right string the `typ` function of the natural parameter will be set to the aforementioned function. In this case the resulting type functions are all the same or using the notation from example 4.9

$$\text{type}_i^{(s)} = \text{typ}$$

holds for all $s = 2, \dots, S$, $i = 1, \dots, d$. Consider that code 2 allows only to choose type functions from examples 4.5, 4.6 and 4.7, however, one could simply extend these options by following the scheme and implement a real-valued function. On the other hand if `typ` is initialized with an incorrect string or stays `None` a type function will be drawn randomly from the list as seen within the `else` case.

The second input `beta` fulfills the same purpose as the β in the examples 4.5, 4.6 and 4.7 and was intended to be used as a float.

This thesis recognizes that the proposed implementation does not allow to specify each type function individually but rather defines all type functions the same way or makes the process of defining random without giving further insight. However, this implementation was done by choice to ensure easier usage within the scope of this study. An extension to the code which allows to specify each type function individually should be possible, although the implementation might not be so easy.

After establishing the `natural_parameter_definer` within the project, the user is able to choose the kind of q needed to conduct IS. This part is the most essential one as the kind of q does impact the shape of the natural parameter directly. Within the scope of this thesis q can be chosen as a multivariate normal distribution or an exponential distribution. Both of them are implemented as methods within the class.

5.1.1 Natural Parameter Method: Exponential Distribution

The method `create_z_exp_dist` constructs the necessary natural parameter such that $q^{(s)}$ is the pdf of an exponential distribution. Formally, let $d = 1$ and $\mathbb{X}$ be a random vector of length S . By following example 3.5 one could simply choose

$$z^{(s)} < 0, T^{(s)}(x_s) = x_s \text{ and } A^{(s)}(z^{(s)}) = -\log(-z^{(s)})$$

for all $s = 1, \dots, S$, meaning by (16)

$$q(x|z) := \left(\prod_{s=1}^S \mathbf{1}_{\{x_s \geq 0\}}(x_s) \right) \exp \left(\sum_{s=1}^S (z^{(s)} x_s) + \sum_{s=1}^S \log(-z^{(s)}) \right) \quad (22)$$

holds. Here $z^{(s)} < 0$ means that the function $z^{(s)} : \mathbb{R}^{s-1} \rightarrow \mathbb{R}$ only takes negative values. Considering this, the following code shows the beginning of the method `create_z_exp_dist`.

```

80     def create_z_exp_dist(self, S, lamb = None,
81                           combination_typ="mean_out_of_typ"):
82         typ = self.typ
83
84         if lamb == None:
85             lamb = 1
86
87         def z0(list_empty):
88             return -lamb
89
90         list_of_z = [z0]
```

Python Code 3: `class_natural_parameter_components` part 2

As $d = 1$, S is the only necessary input while the positive float lamb can be chosen to define $z^{(1)}$ more precisely, meaning formally $z^{(1)} = -\text{lamb}$. If lamb is not chosen beforehand, $z^{(1)}$ will only take the value -1 per default. Note that $z^{(1)}$ gets denoted by `z0` as python lists start at index 0. After $z^{(1)}$ is created the set $\{z^{(s)} | s = 1, \dots, S\}$, or practical equivalently z , gets defined with initialization of `list_of_z` which gets set equal to `[z0]`, ensuring that `list_of_z` is not only a list but that `z0` is at index 0.

The last input `combination_typ` is a string which describes what kind of combination function is used. More precisely and using example 4.9, `"mean_out_of_typ"` refers to the method used in equation (21). Naturally, `"mean_in_typ"` would mean that the the code will do as (20) proposed.

```

93         if combination_typ == "mean_out_of_typ":
94
95             def combination(list_of_xs):
96                 list_of_xs = np.array(list_of_xs)
97                 sm1 = len(list_of_xs)
98                 typed_values = np.zeros(sm1)
99
100             for s in range(sm1):
101                 typed_values[s] = typ(list_of_xs[s])
102
103             output = np.mean(typed_values)
104
105             return -output
106
107         combination_typ_for_USE = combination
108
109         elif combination_typ == "mean_in_typ":
110
111             def combination(list_of_xs):
112                 list_of_xs = np.array(list_of_xs)
113                 this_is_mean = np.mean(list_of_xs)
114                 output = typ(this_is_mean)
```

```

116         return -output
117
118
119
120         combination_typ_for_USE = combination
121
122     else:
123         raise ValueError(f"Invalid combination_typ '{combination_typ}'. Use 'mean_out_of_typ' or 'mean_in_typ'.")

```

Python Code 4: class_natural_parameter_components part 3

Code 4 demonstrates how one could handle the different cases "mean_in_typ" and "mean_out_of_typ". Firstly, the string combination_typ is checked, where in the case that combination_typ is not equal to either "mean_in_typ" or "mean_out_of_typ", an error is raised. In other cases a function combination is created which takes a list list_of_xs := $(x_1, \dots, x_{s-1})$ of length sm1 := $s - 1$ as input and internally performs the calculations necessary according to example 4.9. Afterwards, the function combination will be set equal to combination_typ_for_USE which, as the name suggests, will be used to define each $z^{(s)}$.

```

125     def make_z_s(comb_typ):
126         def z_s(list_of_xs):
127             return comb_typ(list_of_xs)
128         return z_s
129
130     for _ in range(S - 1):
131         list_of_z.append(make_z_s(
132             combination_typ_for_USE))
133
134     return list_of_z

```

Python Code 5: class_natural_parameter_components part 4

At last list_of_z gets completed and output by the method create_z_exp_dist. This is achieved by using the helper function make_z_s which itself outputs z_s with the help of combination_typ_for_USE. By looping and appending the list_of_z accordingly it is ensured that the outputted result has the desired length and shape.

It is here to note that it is highly recommended to find a type function which only outputs positive values as the function combination multiplies the final result by -1 trying to output a negative value. If the type function was not chosen positive, the final result could be corrupted or the code as a whole could be erroneous.

5.1.2 Natural Parameter Method: Multivariate Normal Distribution

Similar to the method `create_z_exp_dist` the method `create_z_normal` creates the natural parameter for the multivariate normal distribution. Looking at example 3.6 one could define

$$z^{(s)} := (\lambda^{(s)}, \Lambda^{(s)}), T^{(s)}(x_s) := (\tau^{(s)}(x_s), \mathcal{T}^{(s)}(x_s)) \text{ and } q_0^{(s)}(x_s) := (2\pi)^{-\left(\frac{d}{2}\right)}$$

where $\lambda^{(s)}$ is a d -dimensional vector, $\Lambda^{(s)}$ is a symmetric negative semidefinite $(d \times d)$ -matrice, $\tau^{(s)}(x_s) = x_s$ and $\mathcal{T}^{(s)}(x_s) = x_s x_s^T$ for all $s = 1, \dots, S$. Then by using (16) the exponential family is given by

$$q(x|z) := (2\pi)^{-\left(\frac{d}{2}\right)S} \exp \left(\sum_{s=1}^S \left(\lambda^{(s)T} x_s + \langle \Lambda^{(s)}, x_s x_s^T \rangle_F \right) - \sum_{s=1}^S A^{(s)}(z^{(s)}) \right) \quad (23)$$

with the log-partition being given by

$$A^{(s)}(z^{(s)}) = -\frac{1}{4} \lambda^{(s)T} \Lambda^{(s)-1} \lambda^{(s)} - \frac{1}{2} \log(\det(-2\Lambda^{(s)})).$$

Note that here, in contrast to the case of the exponential distribution, a natural parameter $z^{(s)}$ does not possess the same dimension as the underlying d -dimensional random vector X_s . By flattening the matrix, ignoring symmetry and looking at it as if it was a vector the exponential family $q^{(s)}(\cdot|z^{(s)})$ can be considered being a $(d + d \cdot d)$ -parameter exponential family. Thus, the proposed methods in example 4.9 do not directly work in this case as there a d -dimensional natural parameter was proposed.

However, one could overcome this problem by only making the vector part $\lambda^{(s)}$ adapted and choosing the matrices $\Lambda^{(s)}$ as constant over all $s = 1, \dots, S$. Therefore, this work will only construct the $\lambda^{(s)}$ to be $\mathcal{F}_{s-1}$ -measurable. This would also reduce the problem back down to the case in example 4.9. Next to that, strictly speaking $z^{(s)} = (\lambda^{(s)}, \Lambda^{(s)})$ as a whole would still be $\mathcal{F}_{s-1}$ -measurable as the constant matrix part is trivial and the trivial σ -algebra is part of every σ -algebra.

Also, this thesis refrains from optimizing over $\Lambda^{(s)}$ as the log-partition includes the inverse as well as the determinant which could make optimizing practically a lot more complicated, exceeding the limits of this study. In this case, when referring to the natural parameter $z_\alpha^{(s)}$ with the component-wise multiplied α , this thesis indicates that only the vector part is affected by α , formally

$$z_\alpha^{(s)} := (\lambda_\alpha^{(s)}, \Lambda^{(s)}).$$

To define the natural parameter practically the method `create_z_normal` follows the same principles as `create_z_exp_dist`. In order to create all $z^{(s)}$ one

must input d , S , the combination_type and the variables for the exponential family at $s = 1$, $q^{(1)}(\cdot|z^{(1)})$. The following code demonstrates what happens in more detail.

```

136     def create_z_normal(self, d, S, mu=None, SIG=None,
137                          combination_type="mean_out_of_type"):
138         typ = self.typ
139
140         if mu is None:
141             mu = np.ones(d)
142         if SIG is None:
143             SIG = np.eye(d)
144         SIG_inv = np.linalg.inv(SIG)
145
146         def z0(list_empty):
147             return SIG_inv @ mu, -0.5 * SIG_inv
148
149         list_of_z = [z0]
```

Python Code 6: class_natural_parameter_components part 5

One can see combination_type works as in create_z_exp_dist, also d and S are equivalently used as in theory. The only inputs different are mu and SIG.

This thesis uses the conventional method of finding $z^{(1)}$ by using example 3.6 and setting

$$\mu = \text{mu and } \Sigma = \text{SIG}.$$

Hence, the resulting $\lambda^{(1)} = \Sigma^{-1}\mu$ and $\Lambda^{(1)} = -\frac{1}{2}\Sigma^{-1}$ are transforming $q^{(1)}(\cdot|z^{(1)})$ into a distribution of form $\mathcal{N}(\mu, \Sigma)$. One can also see that per default mu is set to the d -dimensional vector with only 1 as components and SIG is set to the identity matrix.

It is important to note that this kind of transforming does not hold for other natural parameters $z^{(s)}$ $s = 2, \dots, S$, more formally this thesis proposes to set

$$\Lambda^{(s)} = \Lambda^{(1)}$$

and $\lambda^{(s)}$ so that

$$\lambda_i^{(s)} = \text{type}_i^{(s)} \left(\text{comb}_i^{(s)}(X_1, \dots, X_{s-1}) \right) \text{ or } \text{comb}_i^{(s)}(X_1^{(\text{type})}, \dots, X_{s-1}^{(\text{type})})$$

holds for all $s = 2, \dots, S$, $i = 1, \dots, d$. Indeed, this is exactly what example 4.9 demonstrated.

The method create_z_normal follows exactly these guidelines and despite some small differences, the rest of the code is nearly identical to code 4 and 5. Therefore, the rest of create_z_normal will be excluded from the following paragraphs and can be found in the Appendix instead. The only major difference is that the resulting output list_of_z is a list where each element is a tuple, one including the vector and the other including the matrix part.

5.2 Optimization the Variance: Code Implementation

In this section optimization problem (15) gets further analyzed and practically implemented with a python class which solves (15) by only using numpy and cvxpy which is a open source modeling language for convex optimization problems in python; see [23] [7] [1].

The file "class_IS_optimizer.py" acts as the second component of the model and includes the class IS_optimizer as well as two helper functions outside the class, which are available to see in section A.4.2.

```

30 class IS_optimizer(object):
31     def __init__(self, list_of_z, samples, p_joint, f):
32         self.list_of_z = list_of_z
33         self.samples = samples
34         self.p_joint = p_joint
35         self.f = f
36
37         sample0 = self.samples[0]
38         assert sample0.ndim == 2, "Each sample must be a 2D
39             array of shape (S, d)"
39         self.S, self.d = sample0.shape

```

Python Code 7: class_IS_optimizer part 1

Code 7 shows that the initialization of the class requires four inputs. The first input would be list_of_z which can be generated by using the class natural_parameter_definer from the previous chapter.

In order to conduct the optimization process one must generate samples $(\mathbb{X}^{(m)})_{m=1}^M$ from the underlying density $\bar{p}$ as seen in (15). This thesis proposes that these samples are generated preemptively by the user and then are given to the class under the second input samples. These samples are recommended to be imbedded in a list where one sample must be of shape (S, d), even if $d = 1$, as all calculations within the class are mostly conducted by numpy. Next to that, one can also see that S and d within the code are automatically defined by the shape of the first sample in the sample set.

The third and fourth input, p_joint and f, must also be defined beforehand. Following the notation of this thesis

$$\bar{p} = p_{\text{joint}} \text{ and } f = f$$

hold, where both of these functions must be able to calculate numpy arrays of shape (S, d), thus, these functions must be compatible with elements of samples.

Although practically the method of defining the natural parameter practically is very similar for exponential and the multivariate normal distribution, the same can not be said about the optimizing process in both cases.

Indeed, the basic ideas are nearly identical and both methods try to stay as similar as possible, however, the complexity of the multivariate normal distribution far outweighs the complexity of the exponential distribution. One reason is that the exponential distribution works only in the case of $d = 1$ where as multivariate normal distribution theoretically works for all $d \in \mathbb{N}$. Another problem is precomputing. This thesis resolves to precomputing all terms which are not dependent on α to allow a much faster computation time. Therefore, the optimization problem (15) is being reformed by using equivalent reformulations. Theoretically those, changes do not impact the final result. Practically however, those changes infer a huge difference, leading to a multiplicative change in computing time. The following two subsections will explain the mathematical and practical approach for both exponential and the multivariate normal distribution in more detail.

5.2.1 Optimization Method: Exponential Distribution

In order to efficiently solve the optimization problem (15) has to be equivalently reformed. For q being the exponential density according to (22), this thesis proposes to divide the problem into a α dependent part and a weight part which is not dependent on α .

Let $(\mathbb{X}^{(m)})_{m=1}^M$ be a sequence of samples sampled from $\bar{p}$,

$$z_{\mathbb{X}^{(m)}}^{(s)} := z^{(s)}(X_1^{(m)}, \dots, X_{s-1}^{(m)})$$

the s -th natural parameter for the m -th sample and $q_0(x) = \prod_{s=1}^S \mathbf{1}_{\{x_s \geq 0\}}(x_s)$. By plugging in q accordingly one gets the following sequence of equations.

$$\begin{aligned} & \sum_{m=1}^M \bar{p}(\mathbb{X}^{(m)}) f(\mathbb{X}^{(m)})^2 q_0(\mathbb{X}^{(m)}) \exp \left(\sum_{s=1}^S -(\alpha^{(s)} z_{\mathbb{X}^{(m)}}^{(s)}) X_s^{(m)} - \log \left(-\alpha^{(s)} z_{\mathbb{X}^{(m)}}^{(s)} \right) \right) \\ &= \sum_{m=1}^M \bar{p}(\mathbb{X}^{(m)}) f(\mathbb{X}^{(m)})^2 q_0(\mathbb{X}^{(m)}) e^{-\sum_{s=1}^S (\alpha^{(s)} z_{\mathbb{X}^{(m)}}^{(s)}) X_s^{(m)}} e^{\sum_{s=1}^S -\log \left(-\alpha^{(s)} z_{\mathbb{X}^{(m)}}^{(s)} \right)} \end{aligned}$$

By vectorizing the problem, meaning setting

$$\alpha := (\alpha^{(1)}, \dots, \alpha^{(S)})^T, z_{\mathbb{X}^{(m)}} := (z_{\mathbb{X}^{(m)}}^{(1)}, \dots, z_{\mathbb{X}^{(m)}}^{(S)})^T \text{ and } \mathbb{X}^{(m)} := (X_1^{(m)}, \dots, X_S^{(m)})^T$$

the problem above equation is equivalent to

$$\sum_{m=1}^M \bar{p}(\mathbb{X}^{(m)}) f(\mathbb{X}^{(m)})^2 q_0(\mathbb{X}^{(m)}) e^{-(\alpha \odot z_{\mathbb{X}^{(m)}})^T \mathbb{X}^{(m)}} e^{\sum_{s=1}^S -\log \left(-\alpha^{(s)} z_{\mathbb{X}^{(m)}}^{(s)} \right)}.$$

Here, by the laws of logarithms and exponents

$$\exp \left(\sum_{s=1}^S -\log \left(-\alpha^{(s)} z_{\mathbb{X}^{(m)}}^{(s)} \right) \right) = e^{-\sum_{s=1}^S \log \left(\alpha^{(s)} \right)} e^{-\sum_{s=1}^S \log \left(-z_{\mathbb{X}^{(m)}}^{(s)} \right)}$$

holds. Therefore, defining the weights $w = (w_1, \dots, w_M)$ by

$$w_m := \bar{p}(\mathbb{X}^{(m)}) f(\mathbb{X}^{(m)})^2 q_0(\mathbb{X}^{(m)}) \exp \left(- \sum_{s=1}^S \log \left(-z_{\mathbb{X}^{(m)}}^{(s)} \right) \right)$$

and recognizing that

$$(\alpha \odot z_{\mathbb{X}^{(m)}})^T \mathbb{X}^{(m)} = \alpha^T (z_{\mathbb{X}^{(m)}} \odot \mathbb{X}^{(m)})$$

holds, the term in the optimization problem gets reduced down to

$$\exp \left(- \sum_{s=1}^S \log \left(\alpha^{(s)} \right) \right) \sum_{m=1}^M w_m \exp \left(- \alpha^T (z_{\mathbb{X}^{(m)}}^{(s)} \odot \mathbb{X}^{(m)}) \right).$$

Additionally, as the logarithm is strictly increasing and therefore is stable when searching the optimal α , the equivalent optimization problem for the exponential distribution can be formulated as minimizing

$$- \sum_{s=1}^S \log \left(\alpha^{(s)} \right) + \log \left(\sum_{m=1}^M \exp \left(\log(w_m) - \alpha^T (z_{\mathbb{X}^{(m)}}^{(s)} \odot \mathbb{X}^{(m)}) \right) \right) \quad (24)$$

over $\alpha \in \mathcal{A}$.

The proposed method `exp_dist` from the class `IS_optimizer` uses the approach described above. However, this work does not formally consider the indicator-function q_0 within the code. Instead it is assumed that each $q^{(s)}$ does not take values smaller or equal to 0 as input, as when IS is conducted the division would result in an ill-defined calculation approach. So instead of handling q_0 in any special way within the method `exp_dist` will simply set $q_0 = 1$ and raise an error if any samples have components smaller or equal to 0.

```

42 def exp_dist(self, bounds=None, cvxpy_solver=cp.SCS,
    cvxpy_verbose=False, cvxpy_max_iters=50000,
    cvxpy_eps=1e-5, cvxpy_acceleration_lookback=10,
    cvxpy_scale=1.0):
43
44     d, S = 1, self.S
45     f = self.f
46     p_joint = self.p_joint
47     list_of_z = self.list_of_z

```

Python Code 8: class `IS_optimizer` part 2

The code above shows the beginning of the method `exp_dist`. One can see that the possible inputs far outnumber the inputs from other methods found throughout this thesis. However, the only important for the end user

would be the bounds which are defined using "**lambda** x:" followed by the desired constraint. An example on how the bounds could look like would be

```
bounds = lambda x: [x>=0,x<=1].
```

Any other input will not be explained in this work as they are strictly used to operate the solver of cvxpy; see [7] [1]. Next to that, it was established that the exponential distribution is only working for $d = 1$. This can also be seen in the code where d is set preemptively to 1.

```

67         alpha = cp.Variable(S*d)
68
69         cons = []
70         eps_pos = 1e-5
71         if bounds is not None:
72             cons += list(bounds(alpha))
73         else:
74             cons += [alpha >= eps_pos]
75
76         z_T_all_XX = [None]*M
77         z_all_XX = [None]*M
78         sum_log_z_all_XX = [None]*M
79
80         for i, XX in enumerate(samples):
81             z_all_XX[i] = helping_z(XX)
82             z_T_all_XX[i] = z_all_XX[i] * XX
83             sum_log_z_all_XX[i] = np.sum(np.log(-
84                 z_all_XX[i]))
85
86         z_T_matrix = np.array(z_T_all_XX)
87
88         weight = np.array([p_joint(XX) * (f(XX)**2) * np
89             .exp(-sum_log_z_all_XX[i]) for XX in samples
90             ])
91
92         alpha_sum_log = cp.sum(cp.log(alpha))
93         log_weight = cp.Constant(np.log(np.maximum(
94             weight, 1e-300)))
95         exponent_vec = -(z_T_matrix @ alpha) +
96             log_weight
97         expression = -alpha_sum_log + cp.log_sum_exp(
98             exponent_vec)
99
100        obj = cp.Minimize(expression)
101        prob = cp.Problem(obj, cons)
102        prob.solve(solver=cvxpy_solver, verbose=
103            cvxpy_verbose, max_iters=cvxpy_max_iters,
104            eps=cvxpy_eps, acceleration_lookback=
105            cvxpy_acceleration_lookback, scale=
106            cvxpy_scale)

```

Python Code 9: class_IS_optimizer part 3

Code 4 shows the with the initialization of $\alpha = \text{alpha}$ the practical approach is very similar to the mathematical approach explained in the begin-

ning of this section. Ignoring the constraints `cons` which are given by the bounds, code 4 uses a lot of precomputing.

By using the notation from throughout this section one gets that

$$z_all_XX := (z_{\mathbb{X}(m)})_{m=1}^M, \quad z_T_all_XX := (z_{\mathbb{X}(m)} \odot \mathbb{X}^{(m)})_{m=1}^M,$$

$$weight := (w_m)_{m=1}^M \text{ and } sum_log_z_all_XX := \left(\sum_{s=1}^S \log(-z_{\mathbb{X}(m)}^{(s)}) \right)_{m=1}^M$$

all hold respectively. The code then vectorizes the problem by defining the matrix

$$z_T_matrix := \left(z_{\mathbb{X}(m)}^{(s)} X_s^{(m)} \right)_{m=1, \dots, M \ s=1, \dots, S} \in \mathbb{R}^{M \times S}$$

and the vector of log weights

$$log_weight := (\log(w_1), \dots, \log(w_M))^T \in \mathbb{R}^M.$$

One can then define a vector responsible for the exponent in problem (24) by

$$exponent_vec := (z_T_matrix @ alpha) + log_weight \in \mathbb{R}^M$$

where `@` simply indicates a matrix-vector multiplication. At the end optimization problem (24) is solved by plugging expression in the `cvxpy` solver where expression consists of the negative logarithm sum over the $\alpha^{(s)}$

$$-alpha_sum_log := - \sum_{s=1}^S \log(\alpha^{(s)})$$

and a appropriate transformation of `exponent_vec`

$$cp.log_sum_exp(exponent_vec) = \log \left(\sum_{s=1}^M \exp(exponent_vec_m) \right).$$

Note that `exponent_vecm` simply refers to the m -th component of the array `exponent_vec`.

Let $\alpha_{opt} = alpha_opt$ be the optimal α of proper shape which solves (24) according to `cvxpy`. The following code demonstrates how one can use α_{opt} to find all necessary components for IS and output them to the end user.

```

101         alpha_opt = alpha.value.reshape(S, d)
102
103     def create_list_of_z_alpha(alpha):
104         alpha = alpha.reshape(S, d)
105         list_of_z_alpha = []
106         for s in range(S):
107             alpha_s = alpha[s]
```

```

108         z_s = list_of_z[s]
109         def z_alpha_s_factory(z_s=z_s, alpha_s=
            alpha_s):
110             def z_alpha_s(list_of_x_s):
111                 eval_z_s = z_s(list_of_x_s)
112                 return alpha_s * eval_z_s
113             return z_alpha_s
114         list_of_z_alpha.append(z_alpha_s_factory
            ())
115         return list_of_z_alpha
116
117     list_of_z_alpha = create_list_of_z_alpha(
        alpha_opt)
118
119     def q_sampler(sample_size):
120         sample_set = [None]*sample_size
121         eps = 1e-8
122         for i in range(sample_size):
123             XX = []
124             for s in range(S):
125                 z_alpha_s = list_of_z_alpha[s]
126                 past_xs = [] if len(XX) == 0 else XX
127                 lam_s = abs(z_alpha_s(past_xs))
128                 lam_s = float(max(lam_s, eps))
129                 x_s = np.random.exponential(scale
                    =1.0/lam_s, size=d)
130                 XX.append(x_s)
131             XX_matrix = np.stack(XX, axis=0)
132             sample_set[i] = XX_matrix
133         return sample_set
134
135     def q_alpha(XX):
136         if np.any(np.array(XX) <= 0):
137             raise ValueError("This exponential pdf
                take no samples with values smaller
                or equal to 0")
138
139         list_of_x_s = []
140         eval_z = np.zeros(S)
141         eval_T = np.zeros(S)
142         for s in range(S):
143             x_s = XX[s]
144             eval_z[s] = list_of_z_alpha[s](
                list_of_x_s)
145             eval_T[s] = x_s
146             list_of_x_s.append(x_s)
147         return np.exp(np.clip(eval_z.T @ eval_T,
            -700, 700)) * np.prod(np.maximum(-eval_z
            , 1e-300))
148
149     return q_sampler, q_alpha, alpha_opt,
        list_of_z_alpha

```

Python Code 10: class_IS_optimizer part 4

The function `create_list_of_z_alpha` creates a new list of natural parameters, formally z_α , using the old `list_of_z` which was given as input by the user when calling `IS_optimizer`.

This z_α is then used to create the new optimized exponential family

$$q(\cdot|z_\alpha) = q_alpha$$

by simply using the definition of exponential families. Note that q_alpha takes the same kind of input as f and p_joint .

One can also see that the sampler $q_sampler$, which takes integers as input, is constructed with algorithm 3.8 in mind. Here `np.random.exponential` is a sampler for exponential distributed from the numpy library where the scaling parameter for each time point s is simply the inverse absolute value of the natural parameter at time s . Formally, given the samples $(X_1, \dots, X_{s-1})$ X_s is sampled from $\text{Exp}(|z^{(s)}|)$.

At the end the method `exp_dist` returns $q_sampler$, q_alpha , $alpha_opt$ and the new optimal natural parameter list `list_of_z_alpha` = $z_{\alpha_{opt}}$ where only the first two outputs are important to conduct IS in a third program.

5.2.2 Optimization Method: Multivariate Normal Distribution

Analogous to the previous section 5.2.1, this chapter addresses the practical implementation of the convex optimization problem (15) with respect to the multivariate normal distribution as the adapted exponential family.

Let $(\mathbb{X}^{(m)})_{m=1}^M$ be a sequence of samples sampled from $\bar{p}$,

$$z_{\mathbb{X}^{(m)}}^{(s)} := (\lambda_{\mathbb{X}^{(m)}}^{(s)}, \Lambda_{\mathbb{X}^{(m)}}^{(s)}) := z^{(s)}(X_1^{(m)}, \dots, X_{s-1}^{(m)})$$

the s -th natural parameter for the m -th sample and $q_0 = (2\pi)^{-\binom{d}{2}S}$. By combining (15) and (23) the problem becomes to minimize

$$\begin{aligned} & \sum_{m=1}^M \bar{p}(\mathbb{X}^{(m)}) f(\mathbb{X}^{(m)})^2 q_0^{-1} \\ & \cdot \exp \left(\sum_{s=1}^S -(\alpha^{(s)} \odot \lambda_{\mathbb{X}^{(m)}}^{(s)})^T X_s^{(m)} - \langle \Lambda_{\mathbb{X}^{(m)}}^{(s)}, X_s^{(m)} X_s^{(m)T} \rangle_F + A^{(s)}(z_{\mathbb{X}^{(m)}, \alpha}^{(s)}) \right) \end{aligned} \quad (25)$$

over $\alpha \in \mathcal{A}$, where

$$A^{(s)}(z_{\mathbb{X}^{(m)}, \alpha}^{(s)}) = -\frac{1}{4}(\alpha^{(s)} \odot \lambda_{\mathbb{X}^{(m)}}^{(s)})^T \Lambda_{\mathbb{X}^{(m)}}^{(s)-1} (\alpha^{(s)} \odot \lambda_{\mathbb{X}^{(m)}}^{(s)}) - \frac{1}{2} \log(\det(-2\Lambda_{\mathbb{X}^{(m)}}^{(s)})).$$

Note that by using $\Lambda_{\mathbb{X}^{(m)}}^{(s)}$ as notation this thesis assumes that the matrix part of the natural parameter is also adapted. However, as established before in

section 5.1.2 the class `natural_parameter_definer` assumes that the matrix part is being chosen as trivial and not adapted. Thus, within the scope of this thesis one can assume that

$$\Lambda_{\mathbb{X}(m)}^{(s)} = \Lambda_{\mathbb{X}(1)}^{(1)} =: \Lambda^{(1)}$$

holds for all $s = 2, \dots, S$. On the other hand the method normal for the class `IS_optimizer`, which will be introduced in this chapter, *could* take an adapted matrix part for the natural parameter as input, allowing possibly to extend `natural_parameter_definer` by introducing adapted matrices. Despite that, this study will *not* optimize over the matrix part resulting in problem (25) from above.

Like in the previous chapter 5.2.1, implementing this problem one to one into the python code can come with computational problems. Thus, reforming the problem with equivalent transformations seems natural. This thesis proposes once again to split the problem into an α dependent and weight part where the former gets vectorized as much as possible.

Via setting w_m equal to

$$\bar{p}(\mathbb{X}^{(m)}) f(\mathbb{X}^{(m)})^2 q_0^{-1} \exp \left(- \sum_{s=1}^S \langle \Lambda_{\mathbb{X}(m)}^{(s)}, X_s^{(m)} X_s^{(m)T} \rangle_F + \frac{1}{2} \log(\det(-2\Lambda_{\mathbb{X}(m)}^{(s)})) \right)$$

one can define the weight vector $w := (w_m)_{m=1}^M$, reducing the term in problem (25) down to

$$\sum_{m=1}^M w_m \exp \left(\sum_{s=1}^S -(\alpha^{(s)} \odot \lambda_{\mathbb{X}(m)}^{(s)})^T X_s^{(m)} - \frac{1}{4} (\alpha^{(s)} \odot \lambda_{\mathbb{X}(m)}^{(s)})^T \Lambda_{\mathbb{X}(m)}^{(s)-1} (\alpha^{(s)} \odot \lambda_{\mathbb{X}(m)}^{(s)}) \right). \quad (26)$$

To vectorize the problem let

$$D_\lambda^{(m)} := \text{blockdiag} \left(\text{diag}(\lambda_{\mathbb{X}(m)}^{(1)}), \dots, \text{diag}(\lambda_{\mathbb{X}(m)}^{(S)}) \right) \in \mathbb{R}^{dS \times dS}$$

be the block matrix with diagonal-matrices of $\lambda_{\mathbb{X}(m)}^{(s)}$ on its diagonal and

$$\Lambda_{inv}^{(m)} := \text{blockdiag} \left(\Lambda_{\mathbb{X}(m)}^{(1)-1}, \dots, \Lambda_{\mathbb{X}(m)}^{(S)-1} \right) \in \mathbb{R}^{dS \times dS}$$

be the block matrix with the inverse of $\Lambda_{\mathbb{X}(m)}^{(s)}$ on its diagonal. Then

$$Q^{(m)} := -\frac{1}{4} D_\lambda^{(m)} \Lambda_{inv}^{(m)} D_\lambda^{(m)}$$

can be used to describe the quadratic part of the log-partition over all s

$$\alpha^T Q^{(m)} \alpha := -\frac{1}{4} \sum_{s=1}^S (\alpha^{(s)} \odot \lambda_{\mathbb{X}(m)}^{(s)})^T \Lambda_{\mathbb{X}(m)}^{(s)-1} (\alpha^{(s)} \odot \lambda_{\mathbb{X}(m)}^{(s)})$$

as $\alpha \odot \lambda_{\mathbb{X}(m)} = D_{\lambda}^{(m)} \alpha$ holds for all $m = 1, \dots, M$. Remember that

$$\alpha := (\alpha^{(1)}, \dots, \alpha^{(S)})^T \text{ and } \lambda_{\mathbb{X}(m)} := (\lambda_{\mathbb{X}(m)}^{(1)}, \dots, \lambda_{\mathbb{X}(m)}^{(S)})^T$$

hold per construction analogous to the last chapter 5.2.1. Additionally, let

$$b^{(m)} := \left(\lambda_{\mathbb{X}(m)}^{(1)} \odot X_1^{(m)}, \dots, \lambda_{\mathbb{X}(m)}^{(S)} \odot X_S^{(m)} \right)^T \in \mathbb{R}^{dS}.$$

be the vector which describes the component-wise multiplication between the vector part of the natural parameter and the sufficient statistics $\tau^{(s)}(x_s) = x_s$.

Combining the above one gets that (26) is equivalent to

$$\sum_{m=1}^M w_m \exp \left(\alpha^T Q^{(m)} \alpha - b^{(m)} \alpha \right)$$

as again $(\alpha^{(s)} \odot \lambda_{\mathbb{X}(m)}^{(s)})^T X_s^{(m)} = \alpha^{(s)T} (\lambda_{\mathbb{X}(m)}^{(s)} \odot X_s^{(m)})$ holds respectively. One can also go further by applying the logarithm over the weights and multiplying by a constant $e^{-maxlog}$ which gives one the final term

$$\sum_{m=1}^M \exp \left(\alpha^T Q^{(m)} \alpha - b^{(m)} \alpha + \log(w_m) - maxlog \right) \quad (27)$$

for the optimization problem. Here $maxlog := \max_m \log(w_m)$ is the maximal logarithmic weight to make the problem practically and numerically more stable.

The beginning of the method normal is almost the same as the one of `exp_dist`.

```

151 def normal(self, bounds=None, cvxpy_solver=cp.SCS,
      cvxpy_verbose=False, cvxpy_max_iters=20000,
      cvxpy_eps=1e-5, cvxpy_acceleration_lookback=10,
      cvxpy_scale=1.0):
152     d, S = self.d, self.S
153     f = self.f
154     p_joint = self.p_joint
155     q_0_inv = (((2 * np.pi) ** (-d / 2)) ** S)**(-1)
156     list_of_z = self.list_of_z
157

```

Python Code 11: class `IS_optimizer` part 5

Therefore, this thesis will omit the description of initialization and continue directly with the optimization process.

```

166 alpha = cp.Variable(S*d)
167
168 cons = []
169 if bounds is not None:
170     cons += list(bounds(alpha))
171
172 frobenius_all_XX = [None]*M
173 quad_term_matrix_all_XX = [None]*M
174 log_det_all_XX = [None]*M
175 lam_times_tau_all_XX = [None]*M
176 exprs = [None]*M
177
178 for i, XX in enumerate(samples):
179     frob_sum = 0.0
180     logdet_sum = 0.0
181     Diag_lam_blocks = []
182     Lam_inv_blocks = []
183     lam_tau_segments = []
184
185     for s in range(S):
186         lam_s, Lam_s = list_of_z[s](list(XX[:, s]))
187         tau_s, Tau_s = T(XX[s])
188
189         frob_sum += np.vdot(Lam_s, Tau_s)
190
191         L = np.linalg.cholesky(-2.0 * Lam_s)
192         logdet_sum += np.sum(np.log(np.diag(L)))
193
194         Diag_lam_blocks.append(np.diag(lam_s))
195         Lam_inv_blocks.append(np.linalg.inv(Lam_s))
196         lam_tau_segments.append(lam_s * tau_s)
197
198     frobenius_all_XX[i] = frob_sum
199     log_det_all_XX[i] = logdet_sum
200
201     D_lambda = block_diag(Diag_lam_blocks)
202     Lambda_inv = block_diag(Lam_inv_blocks)
203     Q = (D_lambda @ Lambda_inv @ D_lambda)
204     quad_term_matrix_all_XX[i] = -0.125 * (Q + Q.T)
205     lam_times_tau_all_XX[i] = np.concatenate(
206         lam_tau_segments)
207
208     Q_i = quad_term_matrix_all_XX[i]
209     b_i = lam_times_tau_all_XX[i]
210     exprs[i] = cp.quad_form(alpha, Q_i) - (alpha @
211         b_i)
212
213     weight = np.array([p_joint(XX) * (f(XX)**2) *
214         q_0_inv * np.exp(-frobenius_all_XX[i] -
215         log_det_all_XX[i]) for i, XX in enumerate(
216         samples)])
217
218     log_w = np.log(weight + 1e-300)
219     maxlog = float(np.max(log_w))
220
221     exponent = cp.hstack(exprs)

```

```

217 |         shifted = exponent + log_w - maxlog
218 |         obj = cp.Minimize(cp.sum(cp.exp(shifted)))
219 |         prob = cp.Problem(obj, cons)
220 |         prob.solve(solver=cvxpy_solver, verbose=cvxpy_verbose
                    |         , max_iters=cvxpy_max_iters, eps=cvxpy_eps,
                    |         acceleration_lookback=
                    |         cvxpy_acceleration_lookback, scale=cvxpy_scale)

```

Python Code 12: class_IS_optimizer part 6

Code 12 introduces $\alpha = \alpha$ as an $(S \cdot d)$ -dimensional vector and follows exactly the above discribed procedure to solve (27) respectively.

Here `frobenuis_all_XX` represents a list of sums of the frobenius product over all samples, formally

$$\text{frobenuis_all_XX} := \left(\sum_{s=1}^S \langle \Lambda_{\mathbb{X}(m)}^{(s)}, X_s^{(m)} X_s^{(m)T} \rangle_F \right)_{m=1}^M$$

where `log_det_all_XX` describes the list of sums of log-determinant part over all samples

$$\text{log_det_all_XX} := \left(\frac{1}{2} \sum_{s=1}^S \log(\det(-2\Lambda_{\mathbb{X}(m)}^{(s)})) \right)_{m=1}^M.$$

Note that in the code the log-determinant is calculated via the Cholesky decomposition (see A.11)

$$\log(\det(-2\Lambda^{(s)})) = 2 \sum_{j=1}^d \log(L_{jj}^{(s)})$$

where $L^{(s)}$ is the lower-triangular Cholesky factor of an arbitrary matrix $-2\Lambda^{(s)}$. Those two components are important to calculate the exponents of the weight w . Naturally, within code 12 `weight = w` holds.

In code 12 `quad_term_matrix_all_XX` refers to a list of symmetric matrices $Q^{(m)}$ over all samples. However, as taking the inverse of $\Lambda_{\mathbb{X}(m)}^{(s)}$ might be numerically prone to inaccuracies resulting in a non-symmetric matrices, this thesis proposes to artificially make the matrix symmetric via setting

$$\text{quad_term_matrix_all_XX} := \frac{1}{2} \left(Q^{(m)} + Q^{(m)T} \right)_{m=1}^M.$$

Note that in code 12 `quad_term_matrix_all_XX` is constructed using the helping function `block_diag` which simply outputs $(dS \times dS)$ -dimensional block matrices respectively.

The last non on α dependent expression `lam_times_tau_all_XX` is set

$$\text{lam_times_tau_all_XX} := \left(b^{(m)} \right)_{m=1}^M.$$

It is here to highlight that as within the loop

$$\text{lam_s}, \text{Lam_s} = \text{list_of_z[s]}(\text{list}(\text{XX}[\text{s}]))$$

is individually evaluating $\lambda_{\mathbb{X}(m)}^{(s)}, \Lambda_{\mathbb{X}(m)}^{(s)}$ for every sample. Thus, as mentioned before, an adapted matrix part of the natural parameter would be theoretically possible if the class `natural_parameter_definer` would produce such.

In order to practically express problem (27) for `cvxpy`, `exprs` uses $Q^{(m)}$ and $b^{(m)}$ and expresses the exponents

$$\text{cp.quad_form}(\alpha, Q_i) - (\alpha @ b_i) := \alpha^T Q^{(m)} \alpha - b^{(m)} \alpha$$

for each sample. At the end

$$\text{maxlog} := \text{maxlog} \text{ and } \log_w := (\log(w_m))_{m=1}^M$$

are defined and every necessary component is plugged into the `cvxpy` solver accordingly, considering the given bounds which work analogous to the ones in `exp_dist`.

The resulting optimal $\alpha_{opt} = \alpha_opt$ is then used to find `list_of_z_alpha`, `q_alpha` and `q_sampler` analogous to chapter 5.2.1 code 25. At the end the method `normal` follows `exp_dist` by outputting the exact same functions and variables respectively.

Recall that `q_sampler` follows exactly algorithm 3.8. However, in order to use the already established multivariate normal sampler `np.random.multivariate_normal` from `numpy` one has to find the mean vector and covariance matrix for each time step s . By looking at example 3.6 one can see that by defining

$$\Sigma^{(s)} := -\frac{1}{2} \Lambda^{(s)-1} \text{ and } \mu^{(s)} := \Sigma^{(s)} \lambda^{(s)},$$

sampling X_s from $\mathcal{N}(\mu^{(s)}, \Sigma^{(s)})$ is the same as sampling from $q^{(s)}(\cdot | z^{(s)})$, given $X_1, \dots, X_{s-1}$.

To conclude both methods `normal` and `exp_dist` follow the exact same principles, however, making the optimization problem efficiently enough requires to reform the original problem (15) in such way so that weights are calculated separately. On how efficient the proposed processes really are will be demonstrated in the next chapters.

6 Demonstrating the Code Implementation

In this section this thesis explores the proposed practical approach to conduct IS with adapted exponential families. Throughout this chapter the classes `natural_parameter_definer` and `IS_optimizer` are tested on two models.

One of these models takes advantage of the well established Black-Scholes model from finance mathematical literatures. However, as this work aims to explain only IS with adapted exponential families and not the Black-Scholes models, this thesis omits various facts and definitions relevant for the Black-Scholes model. Although notations will be formally introduced, readers who are not familiar with the Black-Scholes model can consult in literature like [9] or [27].

As `natural_parameter_definer` and `IS_optimizer` were introduced as two classes which get initialized in a third program created by the user, this study will also refrain from showing the whole code used to demonstrate these examples. Nonetheless, some fundamental code parts will be explained in more detail and can be observed in the Appendix under sections A.4.3 and A.4.4.

The aim throughout this chapter is to analyze the "efficiency" of the proposed models. This is being achieved by comparing the regular Monte Carlo method with the results of the IS process with adapted exponential families. In the first model, a direct comparison to with "real" or "exact" expectation *cannot* be made as numerical or theoretical calculations might take too long, are too error prone or are simply too complex to estimate. This also underlines the very purpose of Monte Carlo and IS since the practical estimation of some expectations proves to be more beneficial than only relying on theory.

This thesis recognizes that the expression "efficiency" is subjectively defined and can very dependent on the circumstances given. For the purpose of this work the IS process is "efficient" if their 95% confidence interval gets smaller considering that $\mu_{f,I}^{(M)}$ seemingly converges to the true expectation μ_f .

More accurately, this thesis proposes to evaluate the data on three levels. The first level would be with graphics which show the progression of the approximations and the 95% confidence intervals over all positive integers $M \in \{M_{start}, \dots, M_{eval}\}$. Here M_{start} and M_{eval} are the smallest and largest integer where $\mu_f^{(M)}$ and $\mu_{f,I}^{(M)}$ are evaluated. It should be noted that for two samples sizes $M_1 \leq M_2$ from $\{M_{start}, \dots, M_{eval}\}$ the samples $(\mathbb{X}^{(m)})_{m=1}^{M_1}$ are

the equal to the first M_1 samples in $(\mathbb{X}^{(m)})_{m=1}^{M_2}$.

In order to show the individual progression for a few $M \in \{M_{start}, \dots, M_{eval}\}$, the second level includes generated tables considering only a few $M \in \mathcal{M} \subset \{M_{start}, \dots, M_{eval}\}$. What samples sizes $\mathcal{M}$ specifically consists of will be specified in the individual model evaluations. Tables of the second evaluation level show the traditional MCE (MCE) and the MCE for the IS (IS) at their respective sample amount M . Also visible are the lower and upper 95% confidence interval borders

$$[\text{low. MCE}_M, \text{up. MCE}_M] := \left[\mu_f^{(M)} - \frac{1.96\sigma_f^{(M)}}{\sqrt{M}}, \mu_f^{(M)} + \frac{1.96\sigma_f^{(M)}}{\sqrt{M}} \right]$$

$$[\text{low. IS}_M, \text{up. IS}_M] := \left[\mu_{f,I}^{(M)} - \frac{1.96\sigma_{f,I}^{(M)}}{\sqrt{M}}, \mu_{f,I}^{(M)} + \frac{1.96\sigma_{f,I}^{(M)}}{\sqrt{M}} \right]$$

for the MCE and IS cases respectfully as well as the width of the confidence intervals

$$\text{width ci MCE}_M := |\text{up. MCE}_M - \text{low. MCE}_M|$$

$$\text{width ci IS}_M := |\text{up. IS}_M - \text{low. IS}_M|.$$

The last two columns of the table show the absolute difference/error to real expectation μ_f , meaning

$$\text{abs. err. MCE}_M := |\mu_f^{(M)} - \mu_f|, \text{abs. err. IS}_M := |\mu_{f,I}^{(M)} - \mu_f|.$$

The third level analyzes the average error and average width of the ci between M_{min} and $M_{max} \in \{M_{start}, \dots, M_{eval}\}$ which will be presented on a separate table. Formally, let the average absolute difference be given by

$$\text{av. abs. diff. MCE}_{M_{min}, M_{max}} := \frac{1}{M_{max} - M_{min} + 1} \sum_{M=M_{min}}^{M_{max}} |\mu_f^{(M)} - \mu_f|$$

$$\text{av. abs. diff. IS}_{M_{min}, M_{max}} := \frac{1}{M_{max} - M_{min} + 1} \sum_{M=M_{min}}^{M_{max}} |\mu_{f,I}^{(M)} - \mu_f|$$

and the average width be

$$\text{av. ci width MCE}_{M_{min}, M_{max}} := \frac{1}{M_{max} - M_{min} + 1} \sum_{M=M_{min}}^{M_{max}} \text{width ci MCE}_M$$

$$\text{av. ci width IS}_{M_{min}, M_{max}} := \frac{1}{M_{max} - M_{min} + 1} \sum_{M=M_{min}}^{M_{max}} \text{width ci IS}_M.$$

Furthermore, to put the error and width into perspective, the last two columns analyze the relative differences from the MCE to the IS case, informally

$$(\text{IS case} - \text{MCE case})/\text{MCE case}.$$

This conversion will be continued throughout when relatively comparing the MCE to the IS.

All graphics and tables relating to the evaluation can be seen in sections A.2 and A.3 while the tables will make use of the notation introduced above. Each test consists of one graphic and two table, each shown under the other respectively. For readability purposes, each test case will start referencing each evaluation level via "Lvl 1", "Lvl 2" and "Lvl 3" and the corresponding tables/figures relevant to each "Lvl" at the introduction of the case. It is important to note that all tables and graphics were generated using matplotlib; see [13].

The evaluation process does *not* consider the computational time of finding the natural parameter with `natural_parameter_definer`, optimizing the variance with `IS_optimizer` and sampling with the new density $q(\cdot|z_{\alpha_{opt}})$, *unless* the processing is *not* completed within an "reasonable" time. Reason for that is that optimizing the code to allow for a faster computational time would exceed the purpose of this thesis since the aim was only the implementation of adapted exponential in IS. Another reason is that in order to analyze computational time, one must respect the underlying hardware and computational power of the processor. Nonetheless, the experiments will give a rough estimate on computational running time to give the reader an understanding what this thesis considers as "reasonable".

The following section will discuss the models and their respective experiments in more detail. As these explanations often compare the MCE and the MCE of the IS directly this work will in some parts use "IS" instead of "MCE of the IS" for readability purposes.

6.1 Model 1: Black-Scholes Model with Asian Spread Call Option

To remain consistent in notation this thesis will replace commonly used notations in the context of the Black-Scholes model by the ones used throughout this study. Moreover, the claims below can be verified via comparison to [9] and [27]. For simplicity reasons it is assumed that for the following model the probability measure Q is the to P equivalent martingale measure ensuring a well-behaved example.

Let $t_0, \dots, t_S$ be $S+1$ time points in the Black-Scholes model with $t_0 = 0$,

terminal time t_S and

$$\mathbb{S}_s := \mathbb{S}_{t_s} := (\mathbb{S}_{t_s}^i)_{i=1}^d$$

the vector of $d \in \mathbb{N}$ asset prices at time t_s for $s = 1, \dots, S$. Given the definitions from the Black-Scholes model the asset prices can be expressed via

$$\mathbb{S}_s^i := \mathbb{S}_{s-1}^i \exp \left(\left(r - \frac{1}{2} \sum_{j=1}^d |\sigma^{ij}|^2 \right) \Delta t_s + \sqrt{\Delta t_s} \sum_{j=1}^d \sigma^{ij} X_{s,j} \right) \quad (28)$$

for $s = 1, \dots, S$, $i = 1, \dots, d$ with $S_0 = (\mathbb{S}_0^i)_{i=1}^d$ being the initial positive prices and $X_s \sim \mathcal{N}(0, I_d)$. Here I_d denotes the $d \times d$ identity matrix. For further context $r > 0$ is the so called risk-free interest rate, $\Sigma = (\sigma^{ij})_{i,j=1}^d$ is the volatility matrix and $\Delta t_s := t_s - t_{s-1}$ describes the time increments. The so called risk-free asset is defined by

$$\mathbb{S}_s^0 := e^{rt_s}.$$

Consider that the above described model takes advantage of pre-established knowledge about the Black-Scholes model as normally assets are evaluated over continuous time and with the help of the brownian motion.

Let $G(\mathbb{S}_0, \dots, \mathbb{S}_S)$ be a path-dependent european claim with finite second moment $E_Q[|G(\mathbb{S}_0, \dots, \mathbb{S}_S)|^2] < \infty$ under measure Q . Now this work aims to approximate the expected payoff given by

$$\mu_f := e^{-rt_S} E_Q[G(\mathbb{S}_0, \dots, \mathbb{S}_S)].$$

To achieve this one can define $\mathbb{X} := (X_s)_{s=1}^S$ the vector of i.i.d. d -dimensional components and find a function f for which

$$f(X_1, \dots, X_S) = e^{-rt_S} G(\mathbb{S}_0, \dots, \mathbb{S}_S)$$

holds. Thus, the interest switches from defining G to defining f which takes the i.i.d. normal distributed random vectors X_s as input. This subtle reformulation allows one to apply the proposed approach from chapters 4 and 5, allowing one to approximate

$$\mu_f = E_Q[f(X_1, \dots, X_S)].$$

6.1.1 Practical Implementation of Model 1

In order to test the implementation of the code, this thesis created the following scenario. Given the information of the Black-Scholes model from chapter 6.1, let $d = 2$, $S \in \mathbb{N}$ and $0 = t_0, \dots, t_S$ be equidistant time points, meaning

$$\Delta t := t_S/S = \Delta t_s$$

holds for all s . Let the claim be a combination of an Asian and a Spread Call option, formally

$$G(\mathbb{S}_0, \dots, \mathbb{S}_S) := (\bar{\mathbb{S}}^1 - \bar{\mathbb{S}}^2 - K)^+$$

where $K > 0$ is the strike price and $\bar{\mathbb{S}}^i := \frac{1}{S} \sum_{s=1}^S \mathbb{S}_s^i$ is the Asian option; see [27]. Thus, f can be written as

$$f(X_1, \dots, X_S) := e^{-rts} \left(\frac{1}{S} \sum_{s=1}^S \mathbb{S}_s^1(X_s) - \frac{1}{S} \sum_{s=1}^S \mathbb{S}_s^2(X_s) - K \right)^+$$

where for given $\mathbb{S}_{s-1}^i$

$$X_s \mapsto \mathbb{S}_s^i(X_s) := \mathbb{S}_{s-1}^i \exp \left(\left(r - \frac{1}{2} \sum_{j=1}^2 |\sigma^{ij}|^2 \right) \Delta t + \sqrt{\Delta t} \sum_{j=1}^2 \sigma^{ij} X_{s,j} \right)$$

holds according to (28).

The following code example shows how one could implement this model practically for

$$\mathbb{S}_0 = \begin{pmatrix} 0.7 \\ 1.1 \end{pmatrix}, \Sigma = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, r = 0.05, K = 0.1 \text{ and } S = 20.$$

```

8 SS_0 = np.array([0.7, 1.1])
9 Sigma = np.array([[1, 1], [0, 1]])
10 S = 20
11 d = 2
12 t_S = 1
13 r = 0.05
14 K = 0.1
15
16 def BlackScholes(normal, ss_0=SS_0, r=r, sigma=Sigma, t_S=
    t_S, S=S):
17     SS1_array = np.zeros(S + 1)
18     SS2_array = np.zeros(S + 1)
19     SS1_array[0], SS2_array[0] = ss_0
20
21     del_t = t_S / S
22     sig1_quad = sigma[0, 0] ** 2 + sigma[0, 1] ** 2
23     sig2_quad = sigma[1, 0] ** 2 + sigma[1, 1] ** 2
24
25     for s in range(S):
26         Z1 = sigma[0, 0] * normal[s, 0] + sigma[0, 1] *
            normal[s, 1]
27         Z2 = sigma[1, 0] * normal[s, 0] + sigma[1, 1] *
            normal[s, 1]
28         SS1_array[s + 1] = SS1_array[s] * np.exp(
29             (r - 0.5 * sig1_quad) * del_t + np.sqrt(del_t) *
                Z1

```

```

30         )
31         SS2_array[s + 1] = SS2_array[s] * np.exp(
32             (r - 0.5 * sig2_quad) * del_t + np.sqrt(del_t) *
33             Z2
34         )
35     return SS1_array, SS2_array
36 def Asian_Spread_Call(normal, K=K, ss_0=SS_0, r=r, sigma=
37     Sigma, t_S=t_S, S=S):
38     SS1_array, SS2_array = BlackScholes(normal, ss_0=ss_0, r
39     =r, sigma=sigma, t_S=t_S, S=S)
40     AsianSS1 = (np.sum(SS1_array) - SS1_array[0]) / S
41     AsianSS2 = (np.sum(SS2_array) - SS2_array[0]) / S
42     return np.maximum(0, AsianSS1 - AsianSS2 - K)
43 def f(XX):
44     return np.exp(-r * t_S) * Asian_Spread_Call(XX)

```

Python Code 13: Example Asian Spread Call Option part 1

Code 13 shows that the function `f` is simply a combination of the Black-Scholes asset formula for fixed $d = 2$ and the Asian spread option, multiplied with the risk-less asset. Although $d = 2$ cannot easily be changed, the proposed code allows for flexible S , however, it will stay equal to 20 throughout testing.

Using rules 4.2 this example requires to choose a multivariate normal distribution as exponential family. Hence, the natural parameter must be defined by using method `create_z_normal` from the class `natural_parameter_definer`.

```

57 list_of_z = natural_parameter_definer(typ="stdNormal", beta
    =5).create_z_normal(d, S, combination_typ="mean_in_typ")

```

Python Code 14: Example Asian Spread Call Option part 2

Due to the utilization of `list_of_z` is, in this case with a clipped standard normal distribution from example 4.7 as type function within the mean, the natural parameter is fully defined inside the third program. Note that `beta` in this environment serves the same purpose as β throughout chapter 4.3.

In order to train the optimizer with a "large enough" sample set one can use already established sampler from `numpy`.

```

60 samples_train = np.random.multivariate_normal(mean, cov,
    size=(training_size, S))

```

Python Code 15: Example Asian Spread Call Option part 3

Code 15 shows one way to sample from the multivariate normal distribution using `numpy`. In this case `training_size` has been set to 1000

whereas mean and cov are set to `np.zeros(d)`, `np.eye(d)` respectively. Choosing this setup ensures that each sample $\mathbb{X}^{(m)}$ is independently sampled from $\bar{p}(x) := \prod_{s=1}^S p^{(s)}(x_s)$ where $p^{(s)}$ is the density from the standard multivariate normal distribution $\mathcal{N}(0, I_d)$. Setting $\bar{p} = \text{p_joint}$ accordingly within the code, one can initiate the class `IS_optimizer` by plugging in all necessary inputs and consequently solving the optimization problem with the method `normal`.

```

62 bounds = lambda x: []
63 opti = IS_optimizer(list_of_z, samples_train, p_joint, f)
64 q_sampler, q_alpha, alpha_opt, list_of_z_alpha = opti.normal
    (bounds=bounds)

```

Python Code 16: Example Asian Spread Call Option part 4

The excerpt of code listed above demonstrates the simplicity of obtaining all necessary components to perform IS with the new exponential family $q(\cdot|z_{\alpha_{opt}})$. Now IS can be easily conducted using the formulae in chapter 2, calculating the MCE or the confidence interval. As example, for any desired $M \in \mathbb{N}$ the MCE for the IS $\mu_{f,I}^{(M)}$ is given via

$$\mu_{f,I}^{(M)} := \frac{1}{M} \sum_{m=1}^M f(\mathbb{X}^{(m)}) \frac{\bar{p}(\mathbb{X}^{(m)})}{q(\mathbb{X}^{(m)}|z_{\alpha_{opt}})}$$

where $\mathbb{X}^{(m)}$ is now sampled from $q(\cdot|z_{\alpha_{opt}})$ or practically using `q_sampler` from code 16. Code implementation for such calculations is not shown within this thesis as it is very simple in nature and would be redundant to demonstrate.

Note that in code 16 `bounds = lambda x: []` does not give any constraints on α_{opt} and simply implement for demonstrating purposes. Removing `bounds = lambda x: []` from the code would yield the exact same result. Indeed, any other constraint for example

$$\text{bounds} = \text{lambda } x: [x >= 0, x <= 1].$$

would have an humongous impact on the final α_{opt} as the search space is being restricted.

6.1.2 Evaluating IS for Model 1

In order to evaluate the efficiency of the IS procedure under various settings this thesis tries to compare the IS with the regular Monte Carlo procedure. Furthermore, with respect to the code which has been partially addressed in chapter 6.1.1, this thesis tries to compare various types of natural parameters to the regular MCE. The types of natural parameters are given with the `typ` function and `combination_typ` within the class `natural_parameter_definer`.

To allow for a consistent analysis the seed for the evaluation has been set to `np.random.seed(42)`. Although strictly speaking, the experiments cannot be considered truly random anymore, this setup has other advantages such as enabling the comparison of various type functions directly on the same data set. It is worth noting that there is no particular reason for the choice of this seed.

The simplest way to evaluate how efficient the IS procedure is would be to compare the results to the objective real expectation. However, within the scope of this thesis the real expectation of $f(\mathbb{X})$ is not calculable as no reasonable way to estimate it via practical or theoretical integral estimations could be found. To mitigate this, the real expectation has been set to regular MCE with 1,000,000 samples as input.

```

73 f_true = [f(x) for x in np.random.multivariate_normal(mean,
74               cov, size=(1000000, S))]
75 true_expect = np.mean(f_true)
76 true_std_err = np.std(f_true, ddof=1) / np.sqrt(len(f_true))
77 true_ci_lower = true_expect - 1.96 * true_std_err
78 true_ci_upper = true_expect + 1.96 * true_std_err

```

Python Code 17: Example Asian Spread Call Option part 5

The result of this estimation was $\mu_f := \mu_f^{(1,000,000)} = 0.06048743233528901$ with an 95% confidence interval

$$[0.05969921131020916, 0.061275653360368866]$$

giving a width of 0.001576442050159707.

The model described in section 6.1.1 can thus be evaluated using the description from the introduction of section 6. In order to find the final optimal α , α_{opt} , the training size M_{train} , meaning the amount of samples used to solve problem (15), has been set to 1000. Additionally, during testing no bounds were given as the accuracy of the final α_{opt} would suffer, hence cancelling the upside of faster computational time.

Moreso, the only change throughout testing is the natural parameter, mainly the type function. As all type functions, seen in code 2, were tested with `combination_typ = "mean_in_typ"` where the explanations for each scenario do not specifically state that "mean_in_typ" was used. Only the "stdNormal" was tested using `combination_typ = "mean_out_of_typ"` which is highlighted accordingly. The reason behind this, is that "mean_in_typ" was substantially faster whilst calculating relative to its counterpart. For simplicity reasons beta was also kept consistently at 5.

At the end new $M_{eval} = 7000$ samples where drawn from their respective density, allowing for comparison of the traditional MCE with the one under IS according to the introduction of section 6. Furthermore, for image generation this thesis uses $M_{start} = 5$ while for the second level of evaluation $\mathcal{M}$ consists of values 100, 500, 700, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000, 5500, 6000, 6500 and 7000. Throughout the third level of evaluation M_{min} will be set consistently to 1000 while $M_{max} = 2000, 3000, \dots, 6000, 7000$.

cappedReLU and ReLU

Lvl 1: Fig. 5, Lvl 2: Tab. 9, Lvl 3: Tab. 10

Although the confidence intervals seem to be improving in contrast to the regular MCE, the chosen type functions are strongly influencing the final result. Whereas the "cappedReLU", "ReLU" and "abs" experienced a small upward jump between 5000 and 6000 samples, the rest of the approximations seem to be more stable.

The "cappedReLU" and "ReLU" scenario yields identical results throughout, due to the relative large β . Here, the absolute error at 5000 samples jumped from 0.006513 to 0.004009 at 6000 samples, a reduction of about 38.45%. Note that the initial error $|\mu_{f,I}^{(M)} - \mu_f|$ remained fairly stable around 0.0065 between 1500,...,5000 samples, whereas after the upward jump the error decreased to around 0.0043. Moreover, after this change the width of the confidence interval increased from 0.010652 to 0.013631 which results in a change of about 27.97%, whilst it was experiencing a decrease in width of the ci before the jump. One can also see that the width was even higher at 5500 samples with 0.014703.

In contrast the regular MCE yielded a much more stable approximation, although, with a wider confidence interval before 5000 samples. The absolute error at 5000 samples was 0.001325 and stayed at 0.001371 at 6000 samples. The width of the confidence interval at 6000 samples was 0.016308, in contrast to the IS case with a width of 0.013631. This means IS improved the confidence interval about 16.42%.

Looking at the averages, one can see that the width of the ci was reduced by about 38% for all approximations by applying IS instead of the regular MCE. On the other hand, it can be observed that the regular MCE beats the IS when calculating the average error between 1000 and 7000 samples. Applying the IS results in an average error of about 0.00578 a 171.5% worsening from the 0.002129 average error in the MCE case.

abs

Lvl 1: Fig. 1, Lvl 2: Tab. 1, Lvl 3: Tab. 2

Although, "abs" scenario behaved somewhat similar to the "cappedReLU" and "ReLU", the final values differ. As the width of the confidence interval and absolute error are 0.010286 and 0.005354 respectively at 6000 samples. Resulting in an better confidence interval with a worse error than the "cappedReLU" and "ReLU" cases. This can also be confirmed visually in figure 1 and in the average ci width. After generating 7000 samples the average ci width for samples 1000 to 7000 reduces about 50% in contrast to the regular MCE. However, the average error worsens about 215.7% in the same interval of samples.

LReLU

Lvl 1: Fig. 4, Lvl 2: Tab. 7, Lvl 3: Tab. 8

Looking at figure 4, visually, the "LReLU" scenario is the most similar to the regular MCE in comparison to the other cases. Yielding a only slightly smaller confidence interval throughout after 2000 samples. In contrast to the previous mentioned cases which had no significant overlap with the regular MCE, "LReLU" overlapped with the regular MCE on several occasions. Although "LReLU" seems to be very similar to the MCE the absolute error only seems to be consistently improving after 5500 samples, whereas before the MCE had a slight edge concerning the absolute error but not the width of the confidence interval.

When looking at the average ci width the from 1000 to 7000 samples, IS yields an about 20% better confidence interval. However, as the $\mu_{f,I}^{(M)}$ underestimates μ_f in the first 5000 samples the MCE averages better results in the same interval of integers by 63.7%. This average can be deceiving as $\mu_{f,I}^{(7000)} \approx 0.059066$ and $\mu_f^{(7000)} \approx 0.05833$ hold, meaning the IS does yield better results at the end.

id

Lvl 1: Fig. 3, Lvl 2: Tab. 5, Lvl 3: Tab. 6

Also very similar to the "LReLU" is the "id" scenario. Although the average ci width gets reduced about 29% the average approximation between 1000 and 3000 samples is slightly better for IS than MCE. Similarly to the "LReLU", "id" underestimates the expectation and additionally, shares similarities with regular MCE in the sense that 2000 and 3000 samples $\mu_{f,I}^{(M)}$ and

$\mu_f^{(M)}$ are roughly comparable. Moreover, "id" has a upward jump between 5000 and 6000 samples (see Fig. 3), although here the change is more subtle than in the "ReLU"/"cappedReLU" scenario.

Although the error of IS over samples 1000 to 7000 averages an about 28.55% worse result than the regular MCE, this percentage can again be misinterpreted as the MCE's accuracy benefits a lot from an upward drift between 3000 and 5000 samples. After 5000 samples the IS recovers and generates results of comparable and even higher quality as the MCE with an additional tighter ci. For example at 7000 samples the ci of the IS is about 23% better than the ci of the MCE, very similiar to the "LReLU" case.

stdExp_dist

Lvl 1: Fig. 6, Lvl 2: Tab. 11, Lvl 3: Tab. 12

The type function "stdExp_dist" is a version of the clipped density for Exp(1) from example 4.7. Visually, the "stdExp_dist" case is the first scenario where the approximations are fairly stable throughout samples 2000 to 7000 (see Fig. 6). Additionally, it is the first case where $\mu_{f,I}^{(M)}$ overestimates the expectation rather than underestimating it like "id" or "LReLU".

On average between 1000 and 7000 samples "stdExp_dist" yields a 0.0014 error rate resulting in a 33.37% better result than the MCE case. Between 1000 and 3000 samples the average error is further reduced by 51.57%. These enhanced results also exhibit an even better ci. On average one can expect a 40-45% tighter ci when using IS in contrast to regular MCE case. For example at 7000 samples IS yields a ci width of about 0.006558 which is about 56% better than the MCE ci width of about 0.014914.

shiftReLU and constant

Lvl 1: Fig. 2, Lvl 2: Tab. 3, Lvl 3: Tab. 4

Due to the relative large β which creates a somewhat large shift for "shiftReLU", "shiftReLU" and "constant" are practically identical for most feasible $X_s \sim \mathcal{N}(0, I_d)$. Therefore, both of these cases can be considered to be the only practically trivial scenarios, meaning each $q^{(s)}$ is measurable w.r.t. the trivial σ -algebra.

Visually this assumption holds as both cases seem to have the same approximation. Although the exponential family here is trivial, reducing the adapted IS procedure back down to the traditional IS, the results are objectively better than the ones conducted with more advanced adapted

exponential families. The approximation $\mu_{f,I}^{(M)}$ begins to stable even before $M = 1000$. With the beginning of 1500 samples $\mu_{f,I}^{(M)}$ stays nearly constant, oscillating minimally around 0.059.

Between 1000 and 2000 samples the error for the MCE averages roughly 0.003655 whereas the IS averages around 0.002007, a -45.08% difference. Similar numbers are also averaged for other intervals of samples. Next to that the confidence intervals are also reduced. Between 1000 and 7000 samples the average width of the ci is reduced by -78.9% from the MCE to the IS case. At 2000 samples $\mu_{f,I}^{(2000)} \approx 0.059956$ with an ci width of 0.006132, whereas $\mu_{f,I}^{(7000)}$ only changes slightly in contrast, the ci width halves down to 0.003221. In comparison at 7000 samples the MCE yields a worse approximation with a ci width of 0.014914, meaning from MCE to IS a -78.4% difference. When comparing the $\mu_f = \mu_f^{(1,000,000)}$ to $\mu_{f,I}^{(7000)}$ one can see that with 142.86 times more samples the MCE can only produce a ci with half the width as IS with 7000 samples.

stdNormal

Lvl 1: Fig. 7, Lvl 2: Tab. 13, Lvl 3: Tab. 14

Arguably the only type function that performed slightly better than the traditional IS procedure with "constant" was "stdNormal" which is a version of the clipped density of $\mathcal{N}(0, I_d)$ from example 4.7. Figure 7 shows that the approximation also begins to stabilize with 1500 samples, oscillating slightly around 0.059. Between 1000 and 2000 samples the error for the IS with "stdNormal" performed slightly worse than the "constant" case at around 0.002179. However, with more samples the "stdNormal" beats the "constant" scenario, averaging an error of 0.001124 between 1000 and 7000 samples. In comparison "constant" averaged around 0.001259. However, the confidence interval for "stdNormal" remains slightly worse on average throughout experimentation, e.g., between 1000 and 7000 samples the average ci width for the "stdNormal" rounded 0.004763 whereas "constant" only averaged around 0.004666.

Looking at some M in tables 3 and 13 one can also see that "constant" and "stdNormal" produce very similar data. "stdNormal" seems to be slightly better at lower M , however, at scale both procedures for IS work sufficiently well.

stdNormal with mean_out_of_typ

Lvl 1: Fig. 8, Lvl 2: Tab. 15, Lvl 3: Tab. 16

This scenario is the only case where the mean was calculated after each $X_{s,i}$ was transformed using the type function "stdNormal". The underlying reason for this is that combination_typ = "mean_out_of_typ" performs relatively slow to its counterpart with only marginal upside as shown by the following results.

Overall, on average the "stdNormal" with "mean_out_of_typ" performed worse than "stdNormal" with "mean_in_typ" in error estimation and width of the confidence interval. For example between 1000 and 7000 samples the error and ci width averaged about 0.001901 and 0.005032 respectively for "mean_out_of_typ".

With no regards to computational time it has to be noted that "stdNormal" with "mean_out_of_typ" still performs very good in comparison to the regular MCE. Figure 8 also shows that the approximation remains very stable across all samples. Furthermore, it can be argued that even though "stdNormal" with "mean_out_of_typ" has shown better results throughout than most scenarios, it does not perform to the same standards when applied to other latest two cases.

6.1.3 Summary of the Evaluation

Although "shiftReLU" produced the same results as "constant" it is apparent that the relatively large β reduced the type function back to the "constant" type function. However, when conducting independent tests with a smaller β the "shiftReLU" behaved very similar to the "ReLU" case. A similar argument can be made for "cappedReLU". In theory it would be possible to find the best appropriate β for each of these cases, however, one would then also have to optimize over β making the problem potentially not convex anymore. Experiments in such direction were not made within the scope of this thesis.

Throughout testing all cases of type function produced a tighter confidence interval than the regular MCE counterpart. In most of the cases the confidence interval was still large enough so that the assumed real expectation μ_f was within the borders which is arguably the most important aspect. Only "abs", "ReLU" and "cappedReLU" showed some weaknesses in this aspect. Looking at figure 1 and 5 one can see that for some M the ci interval for $\mu_{f,I}^{(M)}$ does not include μ_f , thus not including μ_f . Most notably some M before the jump between 5000 and 6000 samples fit this description.

Considering this and application purposes the rest of type functions are recommendable when conducting IS with adapted exponential families for this specific model, especially "constant" and "stdNormal" with "mean_in_typ". Although, small differences may occur both scenarios both work equally good and perform much better than the regular MCE

Computational time throughout testing remained nearly constant and depended majorly on the size of d , S and M_{eval} . Testing was conducted on an IdeaPad Slim 5 from Lenovo with 16 Gigabyte RAM and AMD Ryzen 5 7530U with AMD Radeon Graphics. However, while running the code only CPU and RAM were used, so no usage of the GPU. Given this information, defining the natural parameter takes basically no time. However, given 1000 training samples the optimization process roughly takes 10 seconds for the combination "mean_in_typ" and 35 seconds for "mean_out_of_typ". Sampling 7000 samples from the resulting optimal exponential family `q_sampler` does take about 35 seconds in the case of "mean_in_typ" and up to 3 minutes for "mean_out_of_typ". This also shows the difference of running time for different combination functions, underlining on why only one test was conducted using "mean_out_of_typ".

Once more it necessary to highlight that the time frames are highly dependent on d , S , M_{eval} and the hardware utilized to run the experiments. For example using a stronger device with GPU acceleration yields a much faster computational time even with a much larger d and S .

6.2 Model 2: Poisson Process with Exponential Distribution

The following model was inspired by [21]. Furthermore, the premise of pages 64-66 in [21] is that the summation of i.i.d. exponential distributed random variables creates a counting process. Formally, let $\mathbb{X} := (X_s)_{s=1}^S$ have i.i.d. $\text{Exp}(\theta)$ components, then the random variable defined by

$$\mathbb{S}_S := \sum_{s=1}^S X_s$$

is the waiting time until the S -th event of the Poisson process with rate θ ; see [21]. Now the aim is to find $\mu_f := E[\mathbb{S}_S]$. Theoretically, this model is very simple as

$$E[\mathbb{S}_S] = \sum_{s=1}^S E[X_s] = \sum_{s=1}^S \frac{1}{\theta} = \frac{S}{\theta}$$

holds. This thesis attempts to explore different sizes for S and how this affects the stability of adapted exponential families under these scenarios in the coming experiments.

6.2.1 Practical Implementation of Model 2

To keep model 2 as simple as possible θ will be set equal to 1. Thus, $\mu_f = S$ holds indefinitely. As $\mathbb{S}_S$ has a very simple structure and $\mathbb{X} = \mathbf{XX}$ is practically a numpy array, one can very easily define the function f via

```
12 def f(XX):
13     return float(np.sum(XX))
```

Python Code 18: Example Poisson Process part 1

As the underlying density for each X_s is the exponential distribution, the chosen exponential family also has to be of exponential distribution origin as stated in rules 4.2. Consider that the normal distribution does theoretically fulfill rules 2.3, therefore IS could also be conducted using a normal distributed exponential family. However, this thesis will not explore this possibility and therefore will act in accordance to rules 4.2. This does *not* mean that the classes introduced in chapter 5 are obsolete, however, one could use the methods `create_z_normal` and `normal` with `d=1` respectively and conduct IS accordingly.

Similar to chapter 6.1.1 the natural parameter `list_of_z` is easily defined using the class `natural_parameter_definer`.

```
21 samples_train = np.random.exponential(scale=scale, size=(
    training_size, S, d))
```

Python Code 19: Example Poisson Process part 2

Using the a exponential distribution sampler from numpy one can sample the components of $\mathbb{X}$ from `Exp(1)` accordingly. Also using `scipy` one can define the pdf $\bar{p} = p_joint$. The acquired training sample, `list_of_z`, `p_joint` and `f` can then be used to solve the optimization problem.

```
23 opti = IS_optimizer(list_of_z, samples_train, p_joint, f)
24 q_sampler, q_alpha, alpha_opt, list_of_z_alpha = opti.exp_dist()
```

Python Code 20: Example Poisson Process part 3

The resulting `q_sampler` and `q_alpha` can then be used to conduct IS accordingly.

6.2.2 Evaluating IS for Model 2

The analysis of the models' efficiency as well as their respective notation is nearly identical, hence as described in the introduction of section 6 the notation used in model 1 will be continued analogously throughout both models. Note that all tests are conducted with a fixed seed `np.random.seed(34)` which was chosen for no particular reason.

In order to evaluate the IS procedure this thesis will only compare the type functions "constant", "shiftReLU" and "stdExp_dist" as the natural parameter components need to be negative, thus the type functions must be positive. Likewise, as elaborated in section 6.1 explained combination_typ="mean_in_typ" yields a much faster computational time, therefore the experiments will only be conducted using this setting. In contrast to the previous model 1, model 2 will explore different $S = 5, 50, 100$ and analyze the efficiency for each S , using again $M_{train} = 1000$ training samples but only $M_{eval} = 3500$ evaluation samples while $M_{start} = 5$.

Furthermore, for the second level of evaluation $\mathcal{M}$ consists of values 100, 500, 700, 1000, 1500, 2000, 2500, 2700, 3000 and 3500 while for the third level M_{min} will again remain at 1000 and M_{max} takes 1500, 2000, 2500, 2700, 3000 and 3500.

constant with $\beta = 1$

Lvl 1: Fig. 9, Lvl 2: Tab. 17, Lvl 3: Tab. 18 for $S = 5$;

Lvl 1: Fig. 10, Lvl 2: Tab. 19, Lvl 3: Tab. 20 for $S = 50$;

Lvl 1: Fig. 11, Lvl 2: Tab. 21, Lvl 3: Tab. 22 for $S = 100$

Looking at figure 9 one can see that for the type function "constant" with beta=1 performs well for $S = 5$. The error and width of the ci was greatly reduced by applying IS as opposed to the regular MCE. For example, at 3500 samples $\mu_{f,I}^{3500} \approx 5.01269$ and $\mu_f^{3500} \approx 4.98411$ hold which is equivalent to an 18.6% error reduction from MCE compared to IS. Moreover, the width at this sample size also reduced by about 69.9% after applying IS.

These results are further reinforced by the averages. Between 1000 and 1500 samples the IS averages an error and width of 0.030777 and 0.075069, equivalent to percentage changes -59.13% and -68.79 % respectively in contrast to the MCE cases. Between 1000 and 3500 samples the average error and width of the ci do recover, however, conducting IS still yields comparatively better results.

For $S = 50$ the results worsen for IS (see Fig. 10). The width of the ci reduces "only" by 25.57% on average between 1000 and 7000 samples. Due to the relatively large discrepancies before 2000 samples the IS averages an error of about 0.196532 between 1000 and 3500 samples, a 331.17% change after using IS in contrast to the regular MCE. However, $\mu_{f,I}^{3500} \approx 49.909497$ and $\mu_f^{3500} \approx 49.965185$ do show that the IS is not so far of the real expectation of 50, although the regular MCE still produces improved results.

Looking at figure 11 one can see that for $S = 100$ the trend of worse results continue with larger S . Even without direct consultation of the underlying data sets one can deduce that the width of the ci is tighter for the regular MCE compared to the IS case. The approximation $\mu_{f,I}^{(M)}$ does underestimate the real expectation of 100 nearly throughout all $M \in \{5, 6, \dots, 3500\}$, whilst the regular MCE suffers from identical shortcomings, however, it still performs better than the IS. Although the IS performs worse nearly across all samples, it is important to note that the exponential family q with the type function "constant" seemingly tries to mimic the base density $\bar{p}$. Furthermore, this is shown through the components of the optimal α , α_{opt} . Each $\alpha_{opt}^{(s)}$ was oscillating around 1, for example for $s = 1, 2, 3, 4$ and 5 , $\alpha_{opt}^{(s)}$ equals 1.00607716, 1.00378975, 0.97368747, 0.96705466 and 0.99916115 respectively. Combining this with the constant type function $\beta = 1$, one can see that each $q^{(s)}(\cdot | z_{\alpha_{opt}}^{(s)})$ tries to imitate $\text{Exp}(1)$, meaning that the entire exponential family $q(\cdot | z_{\alpha_{opt}})$ tries to be equal to $\bar{p}$. This hypothesis is reinforced by the fact that α_{opt} trained on 10,000 samples follows the exact same trend, for example for $s = 1, 2, 3, 4$ and 5 , $\alpha_{opt}^{(s)}$ equals the 0.98815767, 0.99561996, 0.98296864, 0.97987375 and 0.99063407 respectively.

This thesis could not find an answer on why IS does not improve the model in such cases. However, the fact that $q(\cdot | z_{\alpha_{opt}})$ tries to imitate $\bar{p}$ does underline the point made shortly after rule 4.2 that $I = \bar{p}/q$ would cancel to 1 making the result *theoretically* not worse.

shiftReLU with $\beta = 0.5$ and stdExp_dist with $\beta = 1$

shiftReLU:

Lvl 1: Fig. 12, Lvl 2: Tab. 23, Lvl 3: Tab. 24 for $S = 5$;

Lvl 1: Fig. 13, Lvl 2: Tab. 25, Lvl 3: Tab. 26 for $S = 50$;

Lvl 1: Fig. 14, Lvl 2: Tab. 27, Lvl 3: Tab. 28 for $S = 100$

stdExp_dist:

Lvl 1: Fig. 15, Lvl 2: Tab. 29, Lvl 3: Tab. 30 for $S = 5$;

Lvl 1: Fig. 16, Lvl 2: Tab. 31, Lvl 3: Tab. 32 for $S = 50$;

Lvl 1: Fig. 17, Lvl 2: Tab. 33, Lvl 3: Tab. 34 for $S = 100$

In this part "shiftReLU" and "stdExp_dist" will be analyzed combined as both cases performed below expectations. In contrast to the "constant" case, "shiftReLU" and "stdExp_dist" start to produce subpar results at $S = 5$ (see Fig. 12 and 15). Although, "stdExp_dist" averages a better error between 1000 and 3500 samples, the width of the ci worsens at about 117.58% from MCE to IS. "shiftReLU" performs even more suboptimal, changing the width of the ci by about 693.042% on average on the same sample interval.

Whilst looking at figures 13 and 16 the results for $S = 50$ are so striking that the need for a numerical analysis vanishes. Although the results show that `"stdExp_dist"` outperforms `"shiftReLU"`, both cases are negligible when compared to the results produced by regular MCE.

Moreover, this additionally applies to the scenario $S = 100$ (see 14 and 17) where the quality of results is comparatively worse than in the previous scenario. For example for $S = 50$ the ci width changed about at 2669.45% from MCE to IS on average with a sample sizes ranging from 1000 to 3500. In comparison, with $S = 100$ and all other parameters remaining constant the ci width experiences an even more significant change of 7430.97%.

6.2.3 Summary of Evaluation

Hence, a clear trend can be observed, as with a larger S the overall quality of results worsens when conducting IS. Also all non-trivial adapted exponential families, meaning adapted exponential families with non-constant type functions, performed at inferior rates compared to the regular MCE. T Moreso, these results pose a drastic change to model 1 in section 6.1 where nearly all cases outperformed the regular MCE.

This thesis speculates that the reason for this subpar performance is that IS heavily dependent on the model and argues that the reason behind the correlation between a larger S and a considerable worsening of results is of mathematical origin as code corrections have not changed nature of the outcome. In order to fully discover the fundamental connections and relations, more testing is required which is not part of the scope of this thesis due to reasons related to complexity. However, for future reference this study proposes to test other functions for f with the same underlying density $\bar{p}$ to find out if the problem is related to the exponential distribution. Small experiments conducted outside the presented models indicate similar results as model 2. Yet a change to a more complex type function could possibly resolve this problem.

7 Conclusion

Over the last few chapters this study introduced adapted exponential families into the Importance Sampling process via a practical Python code. The classes introduced in section 5 were then tested in the previous chapter. Although the latest results in model 2 depict some inefficiencies, model 1 showed promising findings.

Firstly, the code is user-friendly independent of the chosen model, whereas the only major noticeable drawback of the code were long computing times in some instances, for example sampling from the adapted exponential family. Secondly, for the Black-Scholes model the final approximation of the expectation was greatly improved almost for all type and combination functions. Even though the constant type function yielded the most consistent outcomes, type functions like `"stdNormal"` demonstrated similar results, meaning that the Importance Sampling process with non-trivial adapted exponential families can be sufficiently conducted.

To further explore the possibilities of adapted exponential families, more parameters similar to β can be introduced. Additionally, one could test the optimization process by implementing both trivial and non-trivial type functions into the natural parameter. For example looking at the exponent, via doubling the size of the natural parameter one could set

$$(\alpha^{(s)} \odot z^{(s)})^T T^{(s)}(x_s) =: \left(\bar{\alpha}^{(s)} \odot \bar{z}^{(s)} + \tilde{\alpha}^{(s)} \odot \tilde{z}^{(s)} \right)^T T^{(s)}(x_s)$$

where $\bar{z}^{(s)}$ would be the trivial part and $\tilde{z}^{(s)}$ would be the non-trivial part of the natural parameter while both of them share the same dimension. Constructing the optimization process in such a way would yield weights $\bar{\alpha}^{(s)}$ and $\tilde{\alpha}^{(s)}$ which would indicate the relevancy of $\bar{z}^{(s)}$ and $\tilde{z}^{(s)}$, without changing the kind of exponential family and the dimension of X_s .

Furthermore, with the introduction of more complex type and combination functions or the optimization over the matrix part in the normal distribution case, the adapted exponential family can be improved greatly for the Importance Sampling process. Problems such as the inaccuracies for larger S can then be mitigated by, for example, making $q^{(s)}$ "less adapted" over time points s .

If certain problems were resolved and further research was conducted, Importance Sampling with adapted exponential families shows promise to be an upgrade from the traditional Importance Sampling and Monte Carlo process.

A Appendix

A.1 Mathematical Preliminaries

This section was created to be an extension to the theoretical part of this thesis. Thus, presented below are the most important notations, definitions, theorems, propositions and lemmas which were used without further explanation throughout this work.

For the following paragraphs let Q and P be probability measures in measurable space $(\Omega, \mathcal{F})$, unless it is expressed otherwise.

Proposition A.1 (Property of Variance under independency [15]).

Let $E[\cdot]$ be the expectation, $\text{Var}(\cdot)$ be the variance under P and $(X_i)_{i=1}^I$ be a sequence of $I \in \mathbb{N}$ independent random variables. Then

$$\text{Var} \left(\sum_{i=1}^I X_i \right) = E \left[\left(\sum_{i=1}^I X_i \right)^2 \right] - \left(E \left[\sum_{i=1}^I X_i \right] \right)^2 = I\sigma^2$$

holds.

Definition A.2 (Probability Density Function [15]).

A function $f : \mathbb{R}^d \rightarrow [0, \infty)$ is called probability density function, pdf for short, for a d -dimensional random vector X if

$$P(X \in A) = \int_A f(x) dx \text{ for all } A \subset \mathbb{R}^d \text{ Borel.}$$

Most notably $\int_{\mathbb{R}^d} f(x) dx = 1$ holds.

Definition A.3 (Convergence in distribution [15]).

A sequence of random variables $(X^{(m)})_{m=1}^{\infty}$ converges in distribution with respect to probability measure P to the random variable X if

$$P(X^{(m)} \leq x) \xrightarrow{m \rightarrow \infty} P(X \leq x)$$

$\forall x \in \mathbb{R}$ such that $x \mapsto P(X \leq x)$ is continuous at x .

A random vector $(X_1^{(m)}, \dots, X_k^{(m)})$ converges in distribution to the random vector $(X_1, \dots, X_k)$ if

$$P((X_1^{(m)}, \dots, X_k^{(m)}) \leq x) \xrightarrow{m \rightarrow \infty} P((X_1, \dots, X_k) \leq x)$$

$\forall x \in \mathbb{R}^n$ such that $x \mapsto P((X_1, \dots, X_k) \leq x)$ is continuous. Where " $\leq$ " can be understood component-wise. In both cases this thesis will denote a convergence in distribution with $X^{(m)} \xrightarrow{d} X$ and analog $(X_1^{(m)}, \dots, X_k^{(m)}) \xrightarrow{d} (X_1, \dots, X_k)$.

Theorem A.4 (Central Limit Theorem [15]).

Let $X^{(m)} := \frac{1}{\sqrt{\sigma^2 m}} \sum_{k=1}^m (\xi_k - \mu)$ for $(\xi_k)_{k=1}^n$ i.i.d. random variables with $E[\xi_1] = \mu$ and $\text{Var}(\xi_1) = \sigma^2$. Then

$$X^{(m)} \xrightarrow{d} \eta$$

holds for some $\eta \sim \mathcal{N}(0, 1)$.

Theorem A.5 (Properties of the Conditional Expectation [15]).

Let $X, Y \in \mathcal{L}^1(\Omega, \mathcal{F}, P)$ and $\mathcal{F}_2 \subset \mathcal{F}_1 \subset \mathcal{F}$ be σ -algebras. Then the following properties hold of the conditional expectation.

- (i) (Linearity) $E[\alpha X + \beta Y] = \alpha E[X] + \beta E[Y]$.
- (ii) (Monotonicity) $X \geq Y$ a.s. $\Rightarrow E[X|\mathcal{F}] \geq E[Y|\mathcal{F}]$.
- (iii) If $E[|XY|] < \infty$ and Y $\mathcal{F}$ -measurable, then

$$E[XY|\mathcal{F}] = Y E[X|\mathcal{F}] \text{ and } E[XY|\mathcal{F}] = E[XY|\sigma(Y)] = Y.$$

- (iv) (Tower property) $E[E[X|\mathcal{F}_1]|\mathcal{F}_2] = E[X|\mathcal{F}_2]$.

- (v) (Triangle inequality) $|E[X|\mathcal{F}]| \leq E[|X||\mathcal{F}]$.

- (vi) (Independence) If X and $\mathcal{F}$ are independent, then

$$E[X|\mathcal{F}] = E[X].$$

Definition A.6 (Continuity between probability measures [9]).

Q is called absolutely continuous with respect to P on $\mathcal{F}$, if $\forall A \in \mathcal{F}$

$$P(A) = 0 \Rightarrow Q(A) = 0$$

holds. In such a case one can write $Q \ll P$. If additionally $P \ll Q$ holds, then P and Q are called equivalent, which gets denoted by $P \approx Q$.

Theorem A.7 (Radon-Nikodym; [9]).

The following statements are equivalent:

- $Q \ll P$ on $\mathcal{F}$
- there exists an $\mathcal{F}$ -measurable function $\phi \geq 0$ such that

$$E_Q[F] = E_P[F\phi] \quad \forall \mathcal{F}\text{-measurable functions } F \geq 0.$$

The ϕ for which theorem A.7 holds will be called the Radon-Nikodym derivative of Q with respect to P and will be denoted by $\frac{dQ}{dP}$. One can also show that $P \approx Q$ is equivalent to $\frac{dQ}{dP} > 0$ P -almost surely. Additionally $\frac{dP}{dQ}$ would then be given by $(\frac{dQ}{dP})^{-1}$ [9].

Theorem A.8 (Fubini [15]).

Let $(\Omega_i, \mathcal{F}_i, \mu_i)$ be σ -finite measurable spaces, $i = 1, 2$, and $f : \Omega_1 \times \Omega_2 \rightarrow \bar{\mathbb{R}}$ be measurable w.r.t. $\mathcal{F}_1 \otimes \mathcal{F}_2$. If $f \geq 0$ or $f \in \mathcal{L}^1(\mu_1 \otimes \mu_2)$ then

$$\omega_1 \mapsto \int f(\omega_1, \omega_2) \mu_2(d\omega_2) \text{ is } \mathcal{A}_1\text{-measurable}$$

$$\omega_2 \mapsto \int f(\omega_1, \omega_2) \mu_1(d\omega_1) \text{ is } \mathcal{A}_2\text{-measurable}$$

and

$$\begin{aligned} \int_{\Omega_1 \times \Omega_2} f d(\mu_1 \otimes \mu_2) &= \int_{\Omega_1} \left(\int_{\Omega_2} f(\omega_1, \omega_2) \mu_2(d\omega_2) \right) \mu_1(d\omega_1) \\ &= \int_{\Omega_2} \left(\int_{\Omega_1} f(\omega_1, \omega_2) \mu_1(d\omega_1) \right) \mu_2(d\omega_2). \end{aligned}$$

Lemma A.9 (Doobs Functional Representation Lemma [14]).

Let f, g be measurable functions from $(\Omega, \mathcal{F})$ to measurable spaces $(\Omega_f, \mathcal{F}_f)$ and $(\Omega_g, \mathcal{F}_g)$ respectively where $\mathcal{F}_f$ is Borel. Then

$$f \text{ is } g\text{-measurable} \Leftrightarrow \exists h : \Omega_g \rightarrow \Omega_f \text{ measurable function such that } f = h \circ g.$$

Theorem A.10 (Hölder inequality [10]).

Let $(\Omega, \mathcal{F}, \mu)$ be some measurable space, $f, g : \Omega \rightarrow \bar{\mathbb{R}}$ measurable functions and $a, b \in [1, \infty]$ with $\frac{1}{a} + \frac{1}{b} = 1$. Then the Hölder inequality is given by

$$\int_{\Omega} |fg| d\mu \leq \left(\int_{\Omega} |f|^a d\mu \right)^{\frac{1}{a}} \left(\int_{\Omega} |g|^b d\mu \right)^{\frac{1}{b}}.$$

Definition A.11 ((real-valued) Cholesky Decomposition [12]).

Let $A \in \mathbb{R}^{d \times d}$ be a symmetric positive definite matrix with $d \in \mathbb{N}$. Then the decomposition of

$$A = LL^T$$

where $L \in \mathbb{R}^{d \times d}$ is lower triangular matrix with positive diagonal entries is called the Cholesky decomposition of A .

Given the Cholesky decomposition LL^T for any proper matrix A one can find that by the product rule for the determinant for two square matrices

$$\det(A) = \det(L) \det(L^T) = \prod_{i=1}^d L_{i,i}^2 \Rightarrow \log(\det(A)) = 2 \sum_{i=1}^d \log(L_{i,i})$$

holds as L is a lower triangular matrix; see [3].

A.2 Tables and Graphics: Model 1

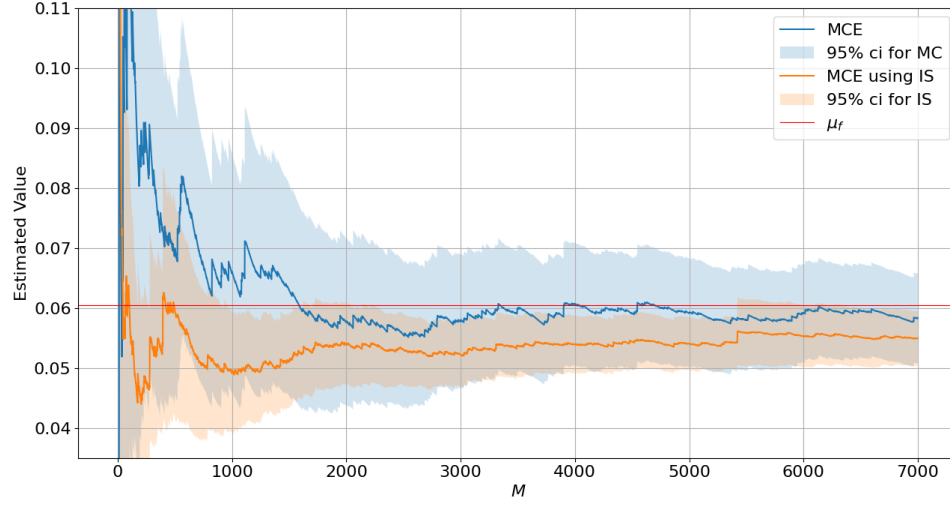


Figure 1: Model 1 - Lvl 1 - "abs" - "mean_in_typ"

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	0.060695	0.033839	0.087551	0.053712	0.112638	0.028406	0.196869	0.168462	0.000208	0.05215
500	0.059553	0.04164	0.077466	0.035826	0.069813	0.045561	0.094065	0.048505	0.000935	0.009326
700	0.051646	0.038458	0.064834	0.026376	0.070321	0.048789	0.091854	0.043065	0.008841	0.009834
1000	0.049369	0.039325	0.059413	0.020088	0.065997	0.046061	0.085932	0.039871	0.011119	0.005509
1500	0.051082	0.043323	0.05884	0.015517	0.062925	0.045309	0.080541	0.035232	0.009406	0.002437
2000	0.053933	0.047134	0.060733	0.013599	0.057625	0.043696	0.071554	0.027857	0.006554	0.002863
2500	0.053015	0.047082	0.058949	0.011867	0.05549	0.043299	0.067682	0.024383	0.007472	0.004997
3000	0.052187	0.046879	0.057495	0.010616	0.05768	0.04654	0.06882	0.02228	0.0083	0.002808
3500	0.053725	0.048538	0.058913	0.010375	0.059925	0.049422	0.070428	0.021006	0.006762	0.000562
4000	0.05389	0.049115	0.058665	0.009551	0.060696	0.050477	0.070914	0.020436	0.006597	0.000208
4500	0.054337	0.049696	0.058977	0.009281	0.059131	0.049761	0.0685	0.018739	0.006151	0.001357
5000	0.054032	0.049614	0.05845	0.008836	0.059163	0.050109	0.068217	0.018108	0.006456	0.001325
5500	0.055856	0.050318	0.061393	0.011076	0.057968	0.049526	0.06641	0.016884	0.004632	0.00252
6000	0.055274	0.05013	0.060417	0.010286	0.059117	0.050963	0.067271	0.016308	0.005214	0.001371
6500	0.055133	0.050294	0.059972	0.009678	0.05955	0.051733	0.067368	0.015635	0.005354	0.000937
7000	0.05494	0.050378	0.059502	0.009124	0.05833	0.050873	0.065787	0.014914	0.005547	0.002157

Table 1: Model 1 - Lvl 2 - "abs" - "mean_in_typ"

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 2000	0.008769	0.003655	0.015992	0.035081	-0.544145	1.399495
1000 - 3000	0.008144	0.003556	0.01397	0.029968	-0.533821	1.290546
1000 - 4000	0.007728	0.002845	0.012718	0.02697	-0.528425	1.716627
1000 - 5000	0.007394	0.00231	0.011829	0.025021	-0.52722	2.201046
1000 - 6000	0.007011	0.002289	0.011435	0.02341	-0.511564	2.062306
1000 - 7000	0.006722	0.002129	0.011145	0.022111	-0.495982	2.157424

Table 2: Model 1 - Lvl 3 - "abs" - "mean_in_typ"

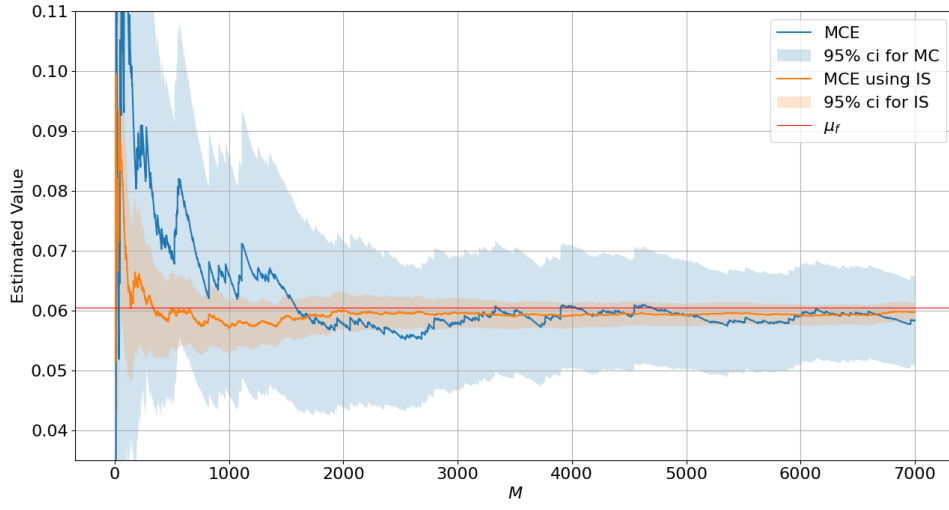


Figure 2: Model 1 - Lvl 1 - "constant" - "mean_in_typ"

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	0.069041	0.051701	0.086382	0.034681	0.112638	0.028406	0.196869	0.168462	0.008554	0.05215
500	0.060283	0.053908	0.066657	0.012749	0.069813	0.045561	0.094065	0.048505	0.000205	0.009326
700	0.058593	0.053411	0.063776	0.010365	0.070321	0.048789	0.091854	0.043065	0.001894	0.009834
1000	0.057229	0.052841	0.061617	0.008776	0.065997	0.046061	0.085932	0.039871	0.003259	0.005509
1500	0.059074	0.055565	0.062583	0.007018	0.062925	0.045309	0.080541	0.035232	0.001414	0.002437
2000	0.059956	0.05689	0.063022	0.006132	0.057625	0.043696	0.071554	0.027857	0.000531	0.002863
2500	0.059646	0.056938	0.062355	0.005417	0.05549	0.043299	0.067682	0.024383	0.000841	0.004997
3000	0.059501	0.057021	0.061982	0.004962	0.05768	0.04654	0.06882	0.02228	0.000986	0.002808
3500	0.058966	0.056721	0.061212	0.004491	0.059925	0.049422	0.070428	0.021006	0.001521	0.000562
4000	0.059153	0.057042	0.061264	0.004222	0.060696	0.050477	0.070914	0.020436	0.001334	0.000208
4500	0.059451	0.057449	0.061454	0.004005	0.059131	0.049761	0.0685	0.018739	0.001036	0.001357
5000	0.0592	0.057319	0.06108	0.00376	0.059163	0.050109	0.068217	0.018108	0.001288	0.001325
5500	0.059474	0.057666	0.061282	0.003615	0.057968	0.049526	0.06641	0.016884	0.001014	0.00252
6000	0.059227	0.057509	0.060944	0.003435	0.059117	0.050963	0.067271	0.016308	0.001261	0.001371
6500	0.05925	0.057596	0.060905	0.003308	0.05955	0.051733	0.067368	0.015635	0.001237	0.000937
7000	0.059708	0.058098	0.061319	0.003221	0.05833	0.050873	0.065787	0.014914	0.000779	0.002157

Table 3: Model 1 - Lvl 2 - "constant" - "mean_in_typ"

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 2000	0.002007	0.003655	0.007076	0.035081	-0.798305	-0.450817
1000 - 3000	0.001482	0.003556	0.006266	0.029968	-0.790924	-0.583154
1000 - 4000	0.001389	0.002845	0.005692	0.02697	-0.788961	-0.511558
1000 - 5000	0.001333	0.00231	0.005267	0.025021	-0.789502	-0.422794
1000 - 6000	0.001287	0.002289	0.004935	0.02341	-0.789186	-0.438004
1000 - 7000	0.001259	0.002129	0.004666	0.022111	-0.78897	-0.408626

Table 4: Model 1 - Lvl 3 - "constant" - "mean_in_typ"

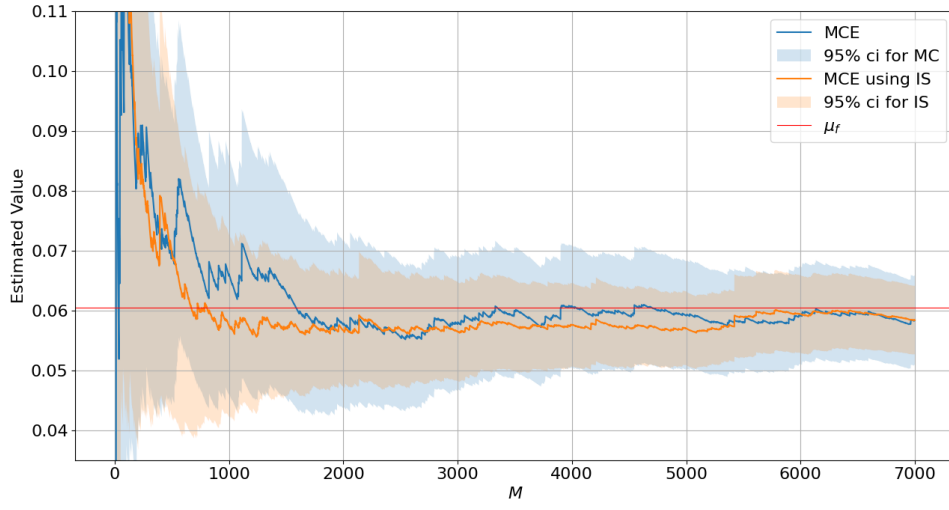


Figure 3: Model 1 - Lvl 1 - "id" - "mean_in_typ"

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	0.130614	0.044485	0.216742	0.172256	0.112638	0.028406	0.196869	0.168462	0.070126	0.05215
500	0.069549	0.042159	0.09694	0.054781	0.069813	0.045561	0.094065	0.048505	0.009062	0.009326
700	0.05945	0.039245	0.079654	0.040409	0.070321	0.048789	0.091854	0.043065	0.001038	0.009834
1000	0.057379	0.042186	0.072571	0.030384	0.065997	0.046061	0.085932	0.039871	0.003109	0.005509
1500	0.057074	0.045698	0.068449	0.022751	0.062925	0.045309	0.080541	0.035232	0.003414	0.002437
2000	0.056596	0.047224	0.065967	0.018743	0.057625	0.043696	0.071554	0.027857	0.003892	0.002863
2500	0.056831	0.047499	0.066163	0.018664	0.05549	0.043299	0.067682	0.024383	0.003656	0.004997
3000	0.056449	0.048181	0.064717	0.016535	0.05768	0.04654	0.06882	0.02228	0.004039	0.002808
3500	0.057421	0.049863	0.064978	0.015115	0.059925	0.049422	0.070428	0.021006	0.003067	0.000562
4000	0.057539	0.050566	0.064512	0.013947	0.060696	0.050477	0.070914	0.020436	0.002948	0.000208
4500	0.057111	0.050497	0.063726	0.013229	0.059131	0.049761	0.0685	0.018739	0.003376	0.001357
5000	0.05699	0.050833	0.063146	0.012313	0.059163	0.050109	0.068217	0.018108	0.003498	0.001325
5500	0.0589	0.052114	0.065686	0.013572	0.057968	0.049526	0.06641	0.016884	0.001588	0.00252
6000	0.059357	0.052876	0.065837	0.012961	0.059117	0.050963	0.067271	0.016308	0.001131	0.001371
6500	0.059677	0.053563	0.065792	0.012229	0.05955	0.051733	0.067368	0.015635	0.00081	0.000937
7000	0.058382	0.052657	0.064107	0.01145	0.05833	0.050873	0.065787	0.014914	0.002105	0.002157

Table 5: Model 1 - Lvl 2 - "id" - "mean_in_typ"

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 2000	0.003307	0.003655	0.023298	0.035081	-0.335882	-0.095076
1000 - 3000	0.003391	0.003556	0.020889	0.029968	-0.30294	-0.046388
1000 - 4000	0.003296	0.002845	0.018984	0.02697	-0.296109	0.158573
1000 - 5000	0.003282	0.00231	0.017534	0.025021	-0.29924	0.420837
1000 - 6000	0.003055	0.002289	0.016584	0.02341	-0.291614	0.334528
1000 - 7000	0.002737	0.002129	0.015856	0.022111	-0.282924	0.285481

Table 6: Model 1 - Lvl 3 - "id" - "mean_in_typ"

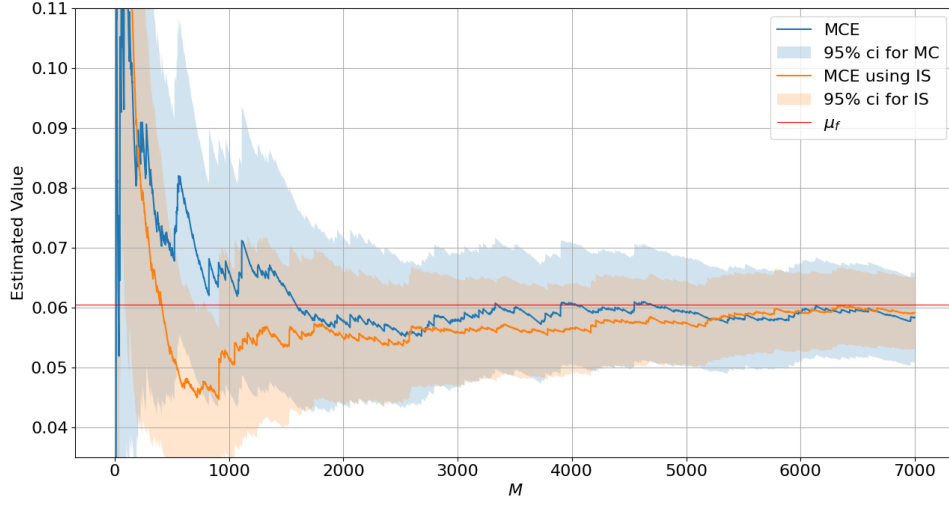


Figure 4: Model 1 - Lvl 1 - "LReLU" - "mean_in_typ"

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	0.14457	0.058064	0.231076	0.173012	0.112638	0.028406	0.196869	0.168462	0.084083	0.05215
500	0.052102	0.032525	0.071678	0.039153	0.069813	0.045561	0.094065	0.048505	0.008386	0.009326
700	0.045914	0.031475	0.060353	0.028878	0.070321	0.048789	0.091854	0.043065	0.014573	0.009834
1000	0.051379	0.034913	0.067844	0.032931	0.065997	0.046061	0.085932	0.039871	0.009109	0.005509
1500	0.053648	0.040287	0.067009	0.026722	0.062925	0.045309	0.080541	0.035232	0.006839	0.002437
2000	0.054879	0.043756	0.066002	0.022245	0.057625	0.043696	0.071554	0.027857	0.005608	0.002863
2500	0.053913	0.044165	0.06366	0.019495	0.05549	0.043299	0.067682	0.024383	0.006575	0.004997
3000	0.05593	0.04704	0.06482	0.01778	0.05768	0.04654	0.06882	0.02228	0.004558	0.002808
3500	0.056449	0.04844	0.064458	0.016018	0.059925	0.049422	0.070428	0.021006	0.004038	0.000562
4000	0.056435	0.048939	0.063931	0.014992	0.060696	0.050477	0.070914	0.020436	0.004052	0.000208
4500	0.057463	0.050074	0.064851	0.014777	0.059131	0.049761	0.0685	0.018739	0.003025	0.001357
5000	0.05744	0.050507	0.064373	0.013867	0.059163	0.050109	0.068217	0.018108	0.003047	0.001325
5500	0.058715	0.051936	0.065494	0.013558	0.057968	0.049526	0.06641	0.016884	0.001772	0.00252
6000	0.059278	0.052736	0.065819	0.013083	0.059117	0.050963	0.067271	0.016308	0.00121	0.001371
6500	0.059879	0.05361	0.066149	0.01254	0.05955	0.051733	0.067368	0.015635	0.000608	0.000937
7000	0.059066	0.053078	0.065055	0.011977	0.05833	0.050873	0.065787	0.014914	0.001421	0.002157

Table 7: Model 1 - Lvl 2 - "LReLU" - "mean_in_typ"

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 2000	0.005377	0.003655	0.027685	0.035081	-0.210837	0.471341
1000 - 3000	0.005296	0.003556	0.023772	0.029968	-0.206738	0.489311
1000 - 4000	0.004948	0.002845	0.02122	0.02697	-0.213201	0.739485
1000 - 5000	0.004493	0.00231	0.019578	0.025021	-0.217525	0.945258
1000 - 6000	0.003986	0.002289	0.018362	0.02341	-0.215635	0.741162
1000 - 7000	0.003486	0.002129	0.017392	0.022111	-0.21346	0.637277

Table 8: Model 1 - Lvl 3 - "LReLU" - "mean_in_typ"

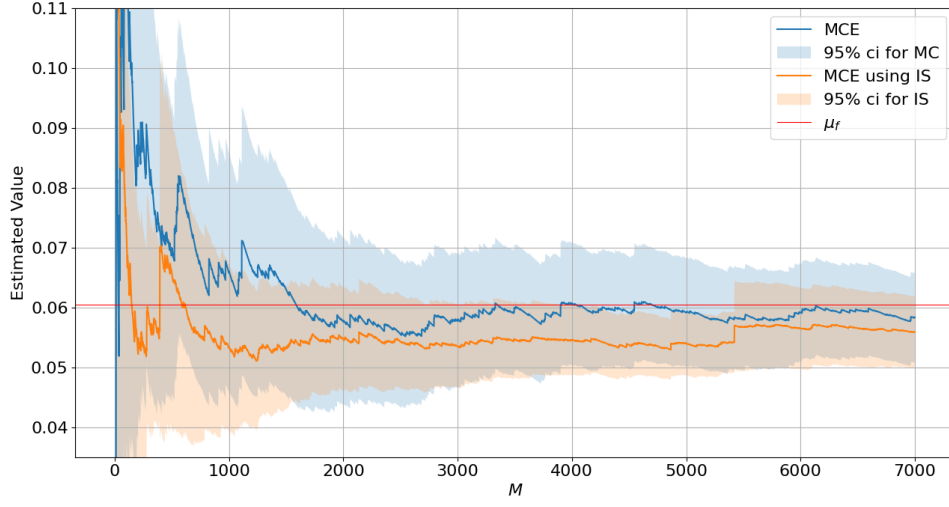


Figure 5: Model 1 - Lvl 1 - "ReLU" - "mean_in_typ"

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	0.073309	0.033884	0.112734	0.078849	0.112638	0.028406	0.196869	0.168462	0.012822	0.05215
500	0.065497	0.039707	0.091287	0.05158	0.069813	0.045561	0.094065	0.048505	0.00501	0.009326
700	0.056438	0.037622	0.075253	0.037631	0.070321	0.048789	0.091854	0.043065	0.00405	0.009834
1000	0.053277	0.039381	0.067172	0.027791	0.065997	0.046061	0.085932	0.039871	0.007211	0.005509
1500	0.053166	0.042955	0.063377	0.020422	0.062925	0.045309	0.080541	0.035232	0.007321	0.002437
2000	0.054622	0.046019	0.063225	0.017205	0.057625	0.043696	0.071554	0.027857	0.005865	0.002863
2500	0.054266	0.046442	0.062091	0.015649	0.05549	0.043299	0.067682	0.024383	0.006221	0.004997
3000	0.053465	0.046562	0.060369	0.013807	0.05768	0.04654	0.06882	0.02228	0.007022	0.002808
3500	0.054653	0.048284	0.061023	0.01274	0.059925	0.049422	0.070428	0.021006	0.005834	0.000562
4000	0.054495	0.048714	0.060277	0.011564	0.060696	0.050477	0.070914	0.020436	0.005992	0.000208
4500	0.053694	0.048342	0.059047	0.010706	0.059131	0.049761	0.0685	0.018739	0.006793	0.001357
5000	0.053975	0.048649	0.059301	0.010652	0.059163	0.050109	0.068217	0.018108	0.006513	0.001325
5500	0.056732	0.049381	0.064084	0.014703	0.057968	0.049526	0.06641	0.016884	0.003755	0.00252
6000	0.056478	0.049663	0.063294	0.013631	0.059117	0.050963	0.067271	0.016308	0.004009	0.001371
6500	0.05653	0.050135	0.062926	0.012792	0.05955	0.051733	0.067368	0.015635	0.003957	0.000937
7000	0.055868	0.049884	0.061852	0.011968	0.05833	0.050873	0.065787	0.014914	0.004619	0.002157

Table 9: Model 1 - Lvl 2 - "ReLU" - "mean_in_typ"

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 2000	0.006851	0.003655	0.02111	0.035081	-0.398235	0.87448
1000 - 3000	0.006561	0.003556	0.018394	0.029968	-0.386224	0.845101
1000 - 4000	0.006383	0.002845	0.016493	0.02697	-0.388453	1.243868
1000 - 5000	0.006434	0.00231	0.015067	0.025021	-0.397816	1.785524
1000 - 6000	0.006135	0.002289	0.014575	0.02341	-0.377409	1.679669
1000 - 7000	0.00578	0.002129	0.014279	0.022111	-0.354226	1.71528

Table 10: Model 1 - Lvl 3 - "ReLU" - "mean_in_typ"

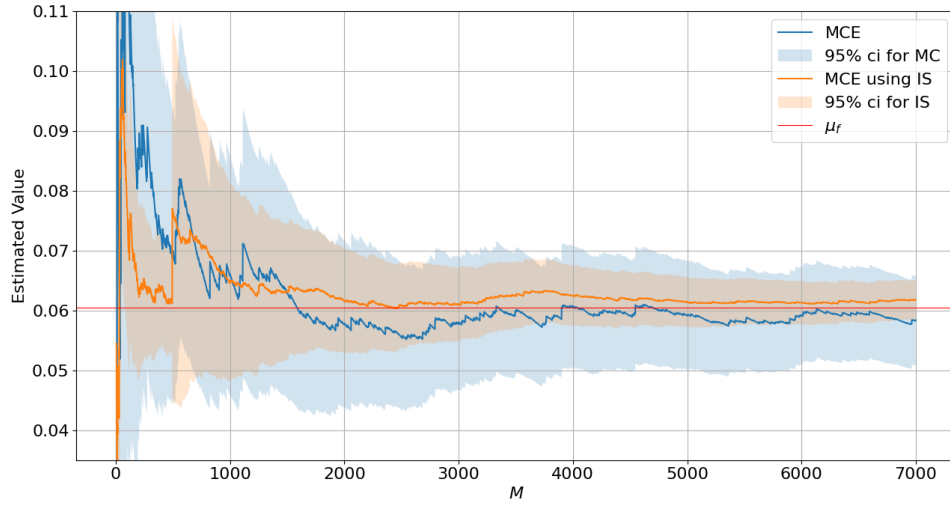


Figure 6: Model 1 - Lvl 1 - "stdExp_dist" - "mean_in_typ"

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	0.076652	0.055275	0.098028	0.042753	0.112638	0.028406	0.196869	0.168462	0.016164	0.05215
500	0.076476	0.044748	0.108205	0.063457	0.069813	0.045561	0.094065	0.048505	0.015989	0.009326
700	0.072353	0.049165	0.095542	0.046377	0.070321	0.048789	0.091854	0.043065	0.011866	0.009834
1000	0.064985	0.048569	0.081401	0.032832	0.065997	0.046061	0.085932	0.039871	0.004498	0.005509
1500	0.063106	0.0518	0.074412	0.022612	0.062925	0.045309	0.080541	0.035232	0.002618	0.002437
2000	0.061563	0.052848	0.070279	0.017432	0.057625	0.043696	0.071554	0.027857	0.001076	0.002863
2500	0.0608	0.053631	0.067968	0.014337	0.05549	0.043299	0.067682	0.024383	0.000312	0.004997
3000	0.060982	0.054864	0.067099	0.012235	0.05768	0.04654	0.06882	0.02228	0.000495	0.002808
3500	0.06241	0.056898	0.067923	0.011025	0.059925	0.049422	0.070428	0.021006	0.001923	0.000562
4000	0.062425	0.057407	0.067444	0.010037	0.060696	0.050477	0.070914	0.020436	0.001938	0.000208
4500	0.061818	0.05728	0.066355	0.009075	0.059131	0.049761	0.0685	0.018739	0.00133	0.001357
5000	0.061411	0.057259	0.065564	0.008305	0.059163	0.050109	0.068217	0.018108	0.000924	0.001325
5500	0.061421	0.057559	0.065283	0.007724	0.057968	0.049526	0.06641	0.016884	0.000934	0.00252
6000	0.061282	0.057688	0.064876	0.007188	0.059117	0.050963	0.067271	0.016308	0.000794	0.001371
6500	0.061368	0.057946	0.064789	0.006843	0.05955	0.051733	0.067368	0.015635	0.00088	0.000937
7000	0.061764	0.058486	0.065043	0.006558	0.05833	0.050873	0.065787	0.014914	0.001277	0.002157

Table 11: Model 1 - Lvl 2 - "stdExp_dist" - "mean_in_typ"

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 2000	0.002863	0.003655	0.023442	0.035081	-0.331775	-0.216498
1000 - 3000	0.001722	0.003556	0.018971	0.029968	-0.366969	-0.515692
1000 - 4000	0.001773	0.002845	0.016356	0.02697	-0.393528	-0.376877
1000 - 5000	0.001683	0.00231	0.014544	0.025021	-0.418723	-0.271228
1000 - 6000	0.001514	0.002289	0.013179	0.02341	-0.437067	-0.338722
1000 - 7000	0.001418	0.002129	0.012124	0.022111	-0.451664	-0.333726

Table 12: Model 1 - Lvl 3 - "stdExp_dist" - "mean_in_typ"

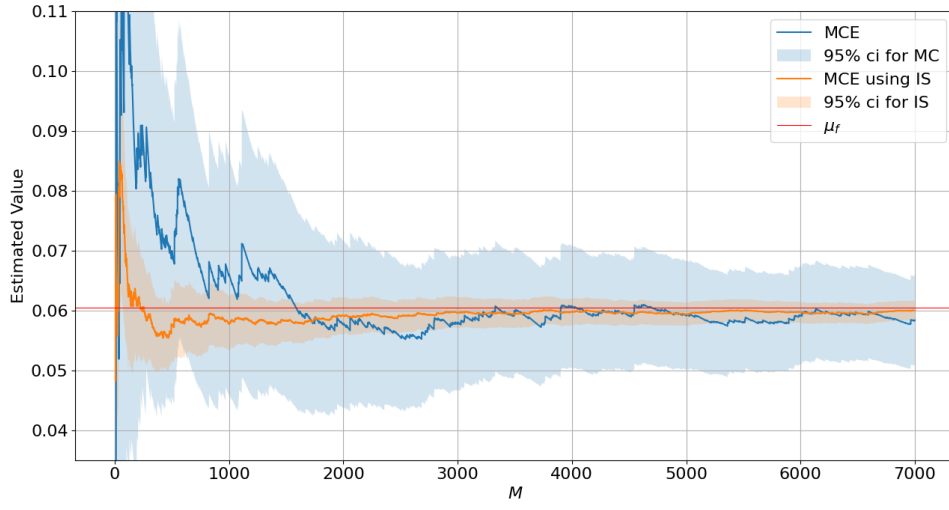


Figure 7: Model 1 - Lvl 1 - "stdNormal" - "mean_in_typ"

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	0.067599	0.054043	0.081156	0.027113	0.112638	0.028406	0.196869	0.168462	0.007112	0.05215
500	0.057642	0.051553	0.063731	0.012178	0.069813	0.045561	0.094065	0.048505	0.002846	0.009326
700	0.058161	0.052774	0.063548	0.010774	0.070321	0.048789	0.091854	0.043065	0.002327	0.009834
1000	0.057966	0.053474	0.062459	0.008984	0.065997	0.046061	0.085932	0.039871	0.002521	0.005509
1500	0.058241	0.05472	0.061762	0.007042	0.062925	0.045309	0.080541	0.035232	0.002246	0.002437
2000	0.059125	0.056082	0.062168	0.006086	0.057625	0.043696	0.071554	0.027857	0.001363	0.002863
2500	0.059024	0.056272	0.061776	0.005503	0.05549	0.043299	0.067682	0.024383	0.001463	0.004997
3000	0.05959	0.057042	0.062137	0.005095	0.05768	0.04654	0.06882	0.02228	0.000898	0.002808
3500	0.059296	0.056975	0.061618	0.004643	0.059925	0.049422	0.070428	0.021006	0.001191	0.000562
4000	0.059731	0.057524	0.061937	0.004413	0.060696	0.050477	0.070914	0.020436	0.000757	0.000208
4500	0.059516	0.057464	0.061568	0.004104	0.059131	0.049761	0.0685	0.018739	0.000972	0.001357
5000	0.059364	0.057433	0.061294	0.003861	0.059163	0.050109	0.068217	0.018108	0.001124	0.001325
5500	0.059942	0.05809	0.061793	0.003703	0.057968	0.049526	0.06641	0.016884	0.000546	0.00252
6000	0.059671	0.057925	0.061418	0.003493	0.059117	0.050963	0.067271	0.016308	0.000816	0.001371
6500	0.059505	0.057843	0.061167	0.003325	0.05955	0.051733	0.067368	0.015635	0.000982	0.000937
7000	0.059968	0.058343	0.061593	0.00325	0.05833	0.050873	0.065787	0.014914	0.00052	0.002157

Table 13: Model 1 - Lvl 2 - "stdNormal" - "mean_in_typ"

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 2000	0.002179	0.003655	0.007163	0.035081	-0.795815	-0.403644
1000 - 3000	0.001733	0.003556	0.006345	0.029968	-0.788288	-0.512509
1000 - 4000	0.00144	0.002845	0.005803	0.02697	-0.784839	-0.493682
1000 - 5000	0.001295	0.00231	0.005384	0.025021	-0.784837	-0.439316
1000 - 6000	0.001179	0.002289	0.005046	0.02341	-0.784476	-0.484825
1000 - 7000	0.001124	0.002129	0.004763	0.022111	-0.784588	-0.472136

Table 14: Model 1 - Lvl 3 - "stdNormal" - "mean_in_typ"

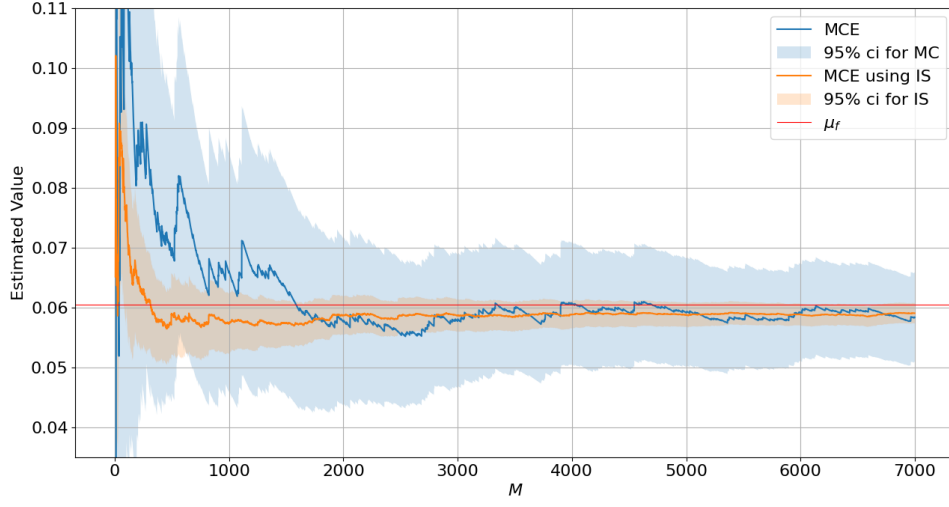


Figure 8: Model 1 - Lvl 1 - "stdNormal" - "mean_out_of_typ"

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	0.07501	0.058202	0.091819	0.033618	0.112638	0.028406	0.196869	0.168462	0.014523	0.05215
500	0.058829	0.05209	0.065568	0.013479	0.069813	0.045561	0.094065	0.048505	0.001658	0.009326
700	0.057885	0.051978	0.063792	0.011813	0.070321	0.048789	0.091854	0.043065	0.002602	0.009834
1000	0.057449	0.052427	0.062471	0.010045	0.065997	0.046061	0.085932	0.039871	0.003038	0.005509
1500	0.057382	0.053604	0.06116	0.007556	0.062925	0.045309	0.080541	0.035232	0.003106	0.002437
2000	0.058719	0.055492	0.061946	0.006453	0.057625	0.043696	0.071554	0.027857	0.001768	0.002863
2500	0.058575	0.055616	0.061534	0.005918	0.05549	0.043299	0.067682	0.024383	0.001912	0.004997
3000	0.058651	0.055972	0.061331	0.005359	0.05768	0.04654	0.06882	0.02228	0.001836	0.002808
3500	0.058369	0.055937	0.060801	0.004863	0.059925	0.049422	0.070428	0.021006	0.002118	0.000562
4000	0.05881	0.056513	0.061107	0.004594	0.060696	0.050477	0.070914	0.020436	0.001678	0.000208
4500	0.058883	0.056746	0.061021	0.004275	0.059131	0.049761	0.0685	0.018739	0.001604	0.001357
5000	0.058738	0.056724	0.060751	0.004027	0.059163	0.050109	0.068217	0.018108	0.00175	0.001325
5500	0.058979	0.057063	0.060895	0.003832	0.057968	0.049526	0.06641	0.016884	0.001508	0.00252
6000	0.058803	0.056986	0.060621	0.003634	0.059117	0.050963	0.067271	0.016308	0.001684	0.001371
6500	0.058672	0.056941	0.060402	0.003461	0.05955	0.051733	0.067368	0.015635	0.001816	0.000937
7000	0.059018	0.057332	0.060704	0.003372	0.05833	0.050873	0.065787	0.014914	0.001469	0.002157

Table 15: Model 1 - Lvl 2 - "stdNormal" - "mean_out_of_typ"

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 2000	0.002849	0.003655	0.007773	0.035081	-0.778435	-0.220545
1000 - 3000	0.002356	0.003556	0.006826	0.029968	-0.772211	-0.337372
1000 - 4000	0.002174	0.002845	0.006195	0.02697	-0.770286	-0.235801
1000 - 5000	0.002023	0.00231	0.005721	0.025021	-0.77136	-0.123958
1000 - 6000	0.00194	0.002289	0.005342	0.02341	-0.771804	-0.152593
1000 - 7000	0.001901	0.002129	0.005032	0.022111	-0.772417	-0.107073

Table 16: Model 1 - Lvl 3 - "stdNormal" - "mean_out_of_typ"

A.3 Tables and Graphics: Model 2

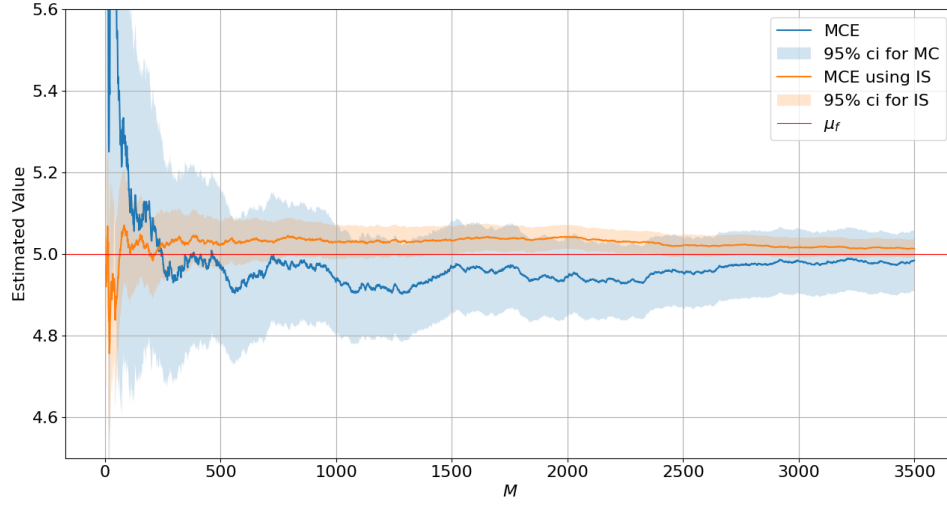


Figure 9: Model 2 - Lvl 1 - "constant" - $S = 5$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	5.02541	4.895344	5.155476	0.260132	5.218423	4.723818	5.713029	0.989211	0.02541	0.218423
500	5.021399	4.963482	5.079316	0.115834	4.970098	4.78101	5.159186	0.378176	0.021399	0.029902
700	5.031207	4.981529	5.080885	0.099355	4.965272	4.806307	5.124237	0.31793	0.031207	0.034728
1000	5.029499	4.988136	5.070862	0.082726	4.935417	4.80244	5.068394	0.265954	0.029499	0.064583
1500	5.031882	4.997439	5.066324	0.068885	4.968106	4.856035	5.080177	0.224142	0.031882	0.031894
2000	5.04128	5.012147	5.070413	0.058266	4.942582	4.845828	5.039336	0.193508	0.04128	0.057418
2500	5.020941	4.994463	5.04742	0.052957	4.954682	4.868604	5.04076	0.172155	0.020941	0.045318
2700	5.020889	4.995547	5.046231	0.050684	4.971701	4.888536	5.054866	0.16633	0.020889	0.028299
3000	5.016225	4.992366	5.040084	0.047718	4.982476	4.90302	5.061931	0.158911	0.016225	0.017524
3500	5.01269	4.990382	5.034998	0.044616	4.984411	4.910352	5.05847	0.148118	0.01269	0.015589

Table 17: Model 2 - Lvl 2 - "constant" - $S = 5$

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	0.030777	0.075295	0.075069	0.24056	-0.687939	-0.591254
1000 - 2000	0.034387	0.060515	0.069053	0.22392	-0.69162	-0.431762
1000 - 2500	0.033013	0.059613	0.064488	0.210109	-0.693075	-0.446206
1000 - 2700	0.03155	0.057381	0.062988	0.205282	-0.693163	-0.450159
1000 - 3000	0.029718	0.052288	0.060896	0.198841	-0.693745	-0.431656
1000 - 3500	0.026835	0.045928	0.057931	0.189677	-0.694581	-0.415707

Table 18: Model 2 - Lvl 3 - "constant" - $S = 5$

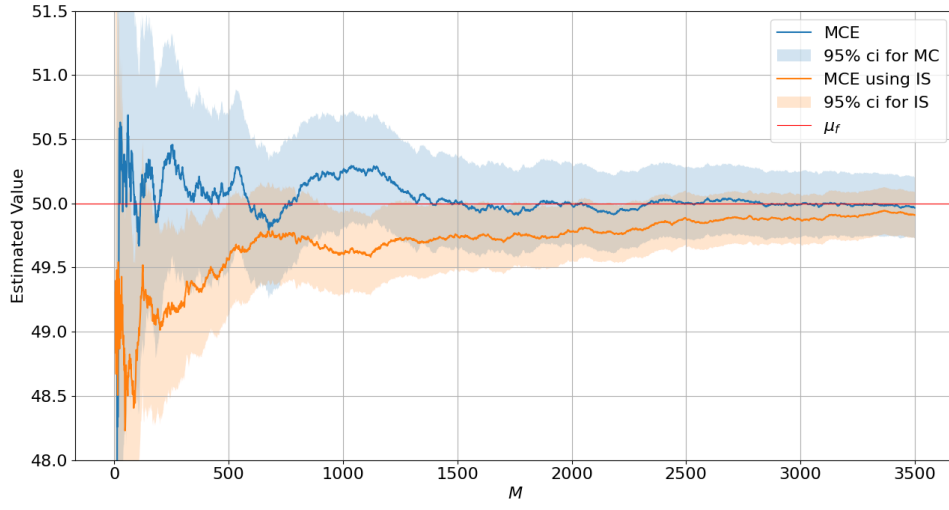


Figure 10: Model 2 - Lvl 1 - "constant" - $S = 50$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	48.777105	47.728902	49.825308	2.096407	49.80718	48.448086	51.166274	2.718189	1.222895	0.19282
500	49.561763	49.122939	50.000587	0.877648	50.120843	49.50112	50.740566	1.239447	0.438237	0.120843
700	49.735416	49.359458	50.111374	0.751917	49.876955	49.341349	50.412561	1.071212	0.264584	0.123045
1000	49.608478	49.287286	49.92967	0.642384	50.257017	49.815401	50.698633	0.883232	0.391522	0.257017
1500	49.750071	49.482795	50.017348	0.534553	50.011985	49.65246	50.371509	0.719049	0.249929	0.011985
2000	49.779429	49.546245	50.012612	0.466367	49.991577	49.67428	50.308875	0.634594	0.220571	0.008423
2500	49.882423	49.670468	50.094378	0.42391	50.004666	49.722997	50.286334	0.563337	0.117577	0.004666
2700	49.886486	49.682774	50.090197	0.407422	50.030213	49.759003	50.301424	0.542421	0.113514	0.030213
3000	49.871797	49.678141	50.065454	0.387313	49.99426	49.738339	50.25018	0.511842	0.128203	0.00574
3500	49.909497	49.730188	50.088806	0.358618	49.965185	49.72756	50.202809	0.475249	0.090503	0.034815

Table 19: Model 2 - Lvl 2 - "constant" - $S = 50$

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	0.315282	0.136506	0.585227	0.791994	-0.261072	1.309659
1000 - 2000	0.286457	0.08407	0.541653	0.732791	-0.260835	2.407379
1000 - 2500	0.255136	0.066972	0.508786	0.687959	-0.260442	2.809574
1000 - 2700	0.240991	0.061203	0.497773	0.67195	-0.259211	2.937598
1000 - 3000	0.222404	0.054024	0.482584	0.6502	-0.257791	3.116782
1000 - 3500	0.196532	0.045581	0.460634	0.61888	-0.255697	3.311715

Table 20: Model 2 - Lvl 3 - "constant" - $S = 50$

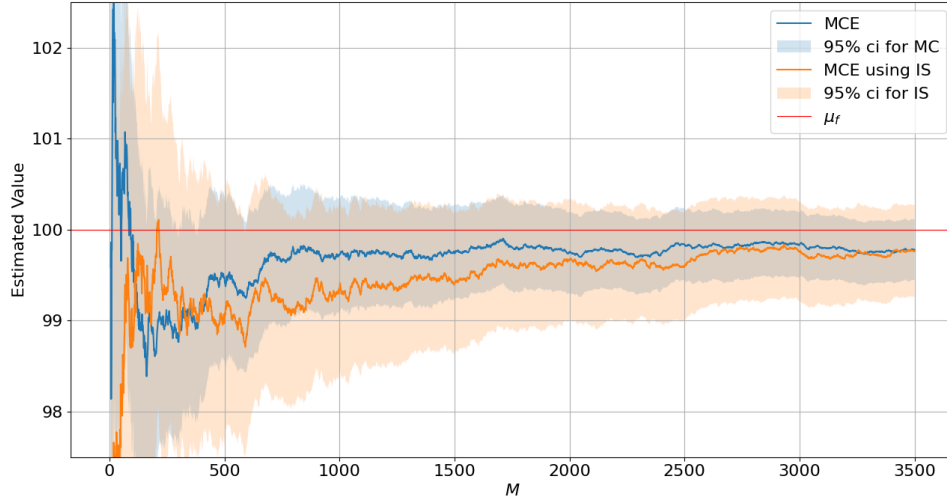


Figure 11: Model 2 - Lvl 1 - "constant" - $S = 100$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	98.928596	96.19168	101.665511	5.473832	99.721464	97.75206	101.690867	3.938807	1.071404	0.278536
500	98.987035	97.650646	100.323424	2.672778	99.452276	98.547366	100.357186	1.80982	1.012965	0.547724
700	99.290463	98.176029	100.404898	2.228869	99.74796	98.994768	100.501153	1.506385	0.709537	0.25204
1000	99.36144	98.428279	100.294601	1.866322	99.725762	99.100524	100.350999	1.250475	0.63856	0.274238
1500	99.408528	98.638706	100.17835	1.539644	99.741665	99.227112	100.256218	1.029107	0.591472	0.258335
2000	99.666179	99.001881	100.330477	1.328596	99.753064	99.306635	100.199492	0.892857	0.333821	0.246936
2500	99.598361	99.002486	100.194236	1.19175	99.836236	99.440982	100.231489	0.790507	0.401639	0.163764
2700	99.761798	99.189473	100.334124	1.144651	99.803235	99.423591	100.182878	0.759288	0.238202	0.196765
3000	99.753167	99.209209	100.297124	1.087915	99.837627	99.478044	100.19721	0.719166	0.246833	0.162373
3500	99.760736	99.257452	100.264019	1.006567	99.777616	99.444392	100.11084	0.666448	0.239264	0.222384

Table 21: Model 2 - Lvl 2 - "constant" - $S = 100$

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	0.592017	0.271038	1.699366	1.131428	0.501966	1.184258
1000 - 2000	0.505635	0.234996	1.561112	1.044279	0.494919	1.151673
1000 - 2500	0.465405	0.237286	1.459992	0.974665	0.497942	0.961366
1000 - 2700	0.443209	0.23099	1.425769	0.951179	0.49895	0.918743
1000 - 3000	0.408996	0.220171	1.379429	0.919258	0.50059	0.857624
1000 - 3500	0.382418	0.220584	1.31298	0.87407	0.502145	0.733661

Table 22: Model 2 - Lvl 3 - "constant" - $S = 100$

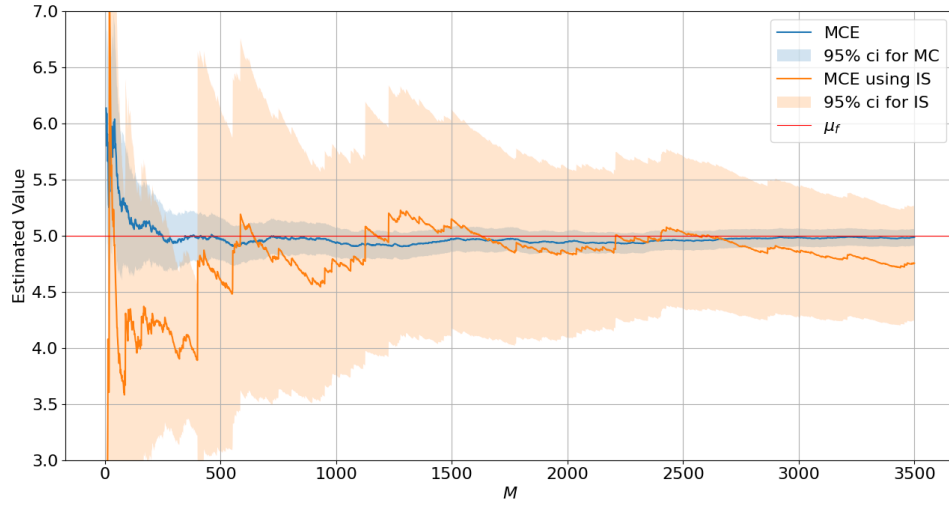


Figure 12: Model 2 - Lvl 1 - "shiftReLU" - $S = 5$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	4.144236	2.276081	6.012392	3.73631	5.218423	4.723818	5.713029	0.989211	0.855764	0.218423
500	4.605677	3.08091	6.130444	3.049534	4.970098	4.78101	5.159186	0.378176	0.394323	0.029902
700	4.865308	3.545412	6.185205	2.639792	4.965272	4.806307	5.124237	0.31793	0.134692	0.034728
1000	4.758233	3.751041	5.765425	2.014384	4.935417	4.80244	5.068394	0.265954	0.241767	0.064583
1500	5.146163	4.173177	6.119149	1.945973	4.968106	4.856035	5.080177	0.224142	0.146163	0.031894
2000	4.844228	4.09025	5.598206	1.507956	4.942582	4.845828	5.039336	0.193508	0.155772	0.057418
2500	5.027944	4.349334	5.706554	1.35722	4.954682	4.868604	5.04076	0.172155	0.027944	0.045318
2700	4.946041	4.313201	5.578881	1.26568	4.971701	4.888536	5.054866	0.16633	0.053959	0.028299
3000	4.853974	4.273428	5.43452	1.161091	4.982476	4.90302	5.061931	0.158911	0.146026	0.017524
3500	4.753046	4.242953	5.263138	1.020185	4.984411	4.910352	5.05847	0.148118	0.246954	0.015589

Table 23: Model 2 - Lvl 2 - "shiftReLU" - $S = 5$

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	0.135164	0.075295	2.055847	0.24056	7.546096	0.79511
1000 - 2000	0.114939	0.060515	1.877895	0.22392	7.386442	0.899346
1000 - 2500	0.102189	0.059613	1.728406	0.210109	7.22623	0.714224
1000 - 2700	0.092153	0.057381	1.679194	0.205282	7.179945	0.605992
1000 - 3000	0.09541	0.052288	1.608821	0.198841	7.090998	0.824699
1000 - 3500	0.11852	0.045928	1.504216	0.189677	6.930425	1.580583

Table 24: Model 2 - Lvl 3 - "shiftReLU" - $S = 5$

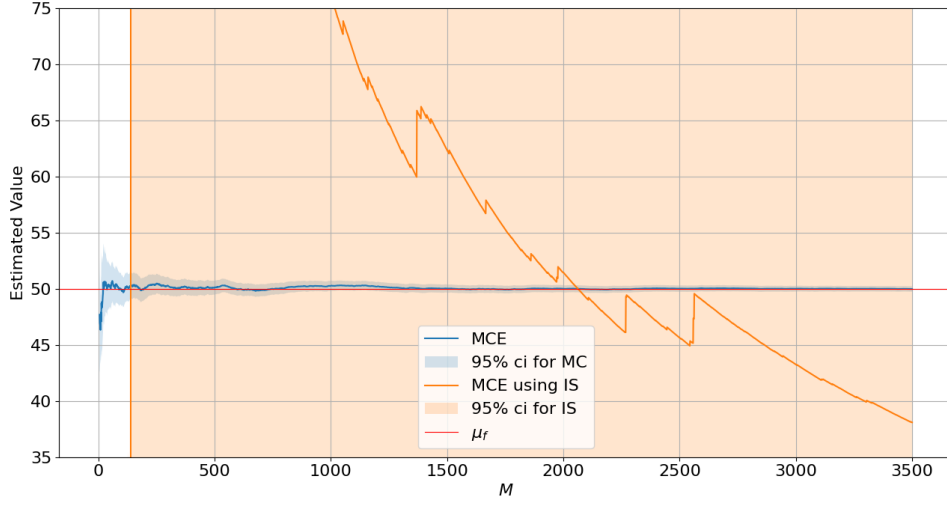


Figure 13: Model 2 - Lvl 1 - "shiftReLU" - $S = 50$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	9.966319	-1.345098	21.277736	22.622834	49.80718	48.448086	51.166274	2.718189	40.033681	0.19282
500	143.985841	-112.589594	400.561277	513.150871	50.120843	49.50112	50.740566	1.239447	93.985841	0.120843
700	104.877036	-78.403499	288.15757	366.56107	49.876955	49.341349	50.412561	1.071212	54.877036	0.123045
1000	76.095818	-52.208902	204.400539	256.609441	50.257017	49.815401	50.698633	0.883232	26.095818	0.257017
1500	62.321257	-23.937546	148.58006	172.517606	50.011985	49.65246	50.371509	0.719049	12.321257	0.011985
2000	51.434893	-13.340466	116.210253	129.550719	49.991577	49.67428	50.308875	0.634594	1.434893	0.008423
2500	45.608154	-6.529582	97.745891	104.275473	50.004666	49.722997	50.286334	0.563337	4.391846	0.004666
2700	47.338267	-1.300471	95.977005	97.277475	50.030213	49.759003	50.301424	0.542421	2.661733	0.030213
3000	43.239654	-0.537379	87.016687	87.554066	49.99426	49.738339	50.25018	0.511842	6.760346	0.00574
3500	38.079406	0.552822	75.605989	75.053167	49.965185	49.72756	50.202809	0.475249	11.920594	0.034815

Table 25: Model 2 - Lvl 2 - "shiftReLU" - $S = 50$

M _{min} - M _{max}	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	16.695928	0.136506	208.536573	0.791994	262.305717	121.309285
1000 - 2000	11.155612	0.08407	178.751032	0.732791	242.931853	131.694723
1000 - 2500	8.136141	0.066972	157.819473	0.687959	228.40238	120.485013
1000 - 2700	7.475109	0.061203	151.117057	0.67195	223.893261	121.137015
1000 - 3000	7.070608	0.054024	142.287974	0.6502	217.837181	129.879779
1000 - 3500	7.547294	0.045581	130.029061	0.61888	209.103863	164.580148

Table 26: Model 2 - Lvl 3 - "shiftReLU" - $S = 50$

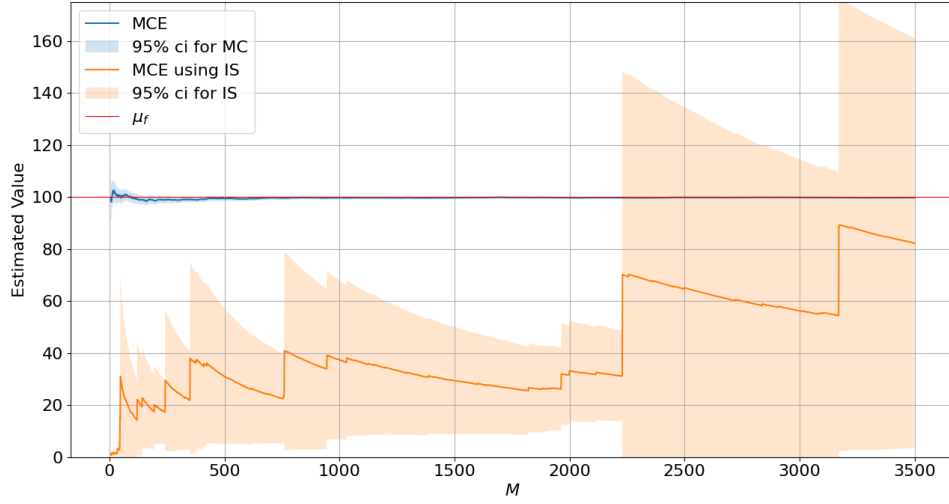


Figure 14: Model 2 - Lvl 1 - "shiftReLU" - $S = 100$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	16.556606	-1.742076	34.855288	36.597364	99.721464	97.75206	101.690867	3.938807	83.443394	0.278536
500	30.935837	4.900493	56.97118	52.070687	99.452276	98.547366	100.357186	1.80982	69.064163	0.547724
700	23.972703	5.34897	42.596436	37.247466	99.74796	98.994768	100.501153	1.506385	76.027297	0.25204
1000	37.335383	6.733572	67.937195	61.203623	99.725762	99.100524	100.350999	1.250475	62.664617	0.274238
1500	29.350759	8.754946	49.946571	41.191625	99.741665	99.227112	100.256218	1.029107	70.649241	0.258335
2000	33.068455	13.722702	52.414207	38.691505	99.753064	99.306635	100.199492	0.892857	66.931545	0.246936
2500	65.076561	-4.705397	134.85852	139.563918	99.836236	99.440982	100.231489	0.790507	34.923439	0.163764
2700	60.540589	-4.074335	125.155513	129.229848	99.803235	99.423591	100.182878	0.759288	39.459411	0.196765
3000	56.231958	-1.942542	114.406459	116.349001	99.837627	99.478044	100.19721	0.719166	43.768042	0.162373
3500	82.218605	3.347613	161.089596	157.741982	99.777616	99.444392	100.11084	0.666448	17.781395	0.222384

Table 27: Model 2 - Lvl 2 - "shiftReLU" - $S = 100$

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	66.561102	0.271038	50.003969	1.131428	43.195433	244.578319
1000 - 2000	69.563743	0.234996	43.111228	1.044279	40.283254	295.020608
1000 - 2500	62.611166	0.237286	61.005324	0.974665	61.591062	262.863672
1000 - 2700	59.631433	0.23099	69.615648	0.951179	72.188816	257.156431
1000 - 3000	56.919745	0.220171	77.551139	0.919258	83.362756	257.524718
1000 - 3500	50.466284	0.220584	91.595849	0.87407	103.792341	227.78505

Table 28: Model 2 - Lvl 3 - "shiftReLU" - $S = 100$

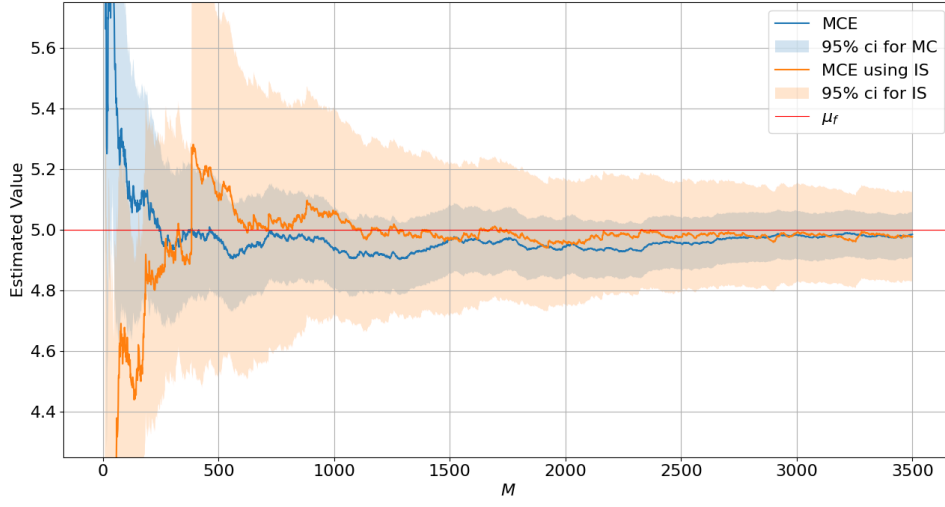


Figure 15: Model 2 - Lvl 1 - "stdExp_dist" - $S = 5$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	4.587527	4.045047	5.130007	1.084961	5.218423	4.723818	5.713029	0.989211	0.412473	0.218423
500	5.095375	4.508282	5.682468	1.174186	4.970098	4.78101	5.159186	0.378176	0.095375	0.029902
700	5.009683	4.567673	5.451692	0.884019	4.965272	4.806307	5.124237	0.31793	0.009683	0.034728
1000	5.054593	4.714005	5.395181	0.681176	4.935417	4.80244	5.068394	0.265954	0.054593	0.064583
1500	4.966703	4.717854	5.215552	0.497698	4.968106	4.856035	5.080177	0.224142	0.033297	0.031894
2000	4.959341	4.752579	5.166104	0.413526	4.942582	4.845828	5.039336	0.193508	0.040659	0.057418
2500	4.975117	4.794186	5.156048	0.361862	4.954682	4.868604	5.04076	0.172155	0.024883	0.045318
2700	4.986353	4.813703	5.159002	0.345299	4.971701	4.888536	5.054866	0.16633	0.013647	0.028299
3000	4.989319	4.82734	5.151297	0.323957	4.982476	4.90302	5.061931	0.158911	0.010681	0.017524
3500	4.976116	4.82891	5.123322	0.294412	4.984411	4.910352	5.05847	0.148118	0.023884	0.015589

Table 29: Model 2 - Lvl 2 - "stdExp_dist" - $S = 5$

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	0.016881	0.075295	0.575103	0.24056	1.390686	-0.775805
1000 - 2000	0.02077	0.060515	0.513394	0.22392	1.292755	-0.656787
1000 - 2500	0.021433	0.059613	0.471366	0.210109	1.243433	-0.640455
1000 - 2700	0.020852	0.057381	0.457526	0.205282	1.22877	-0.636597
1000 - 3000	0.020843	0.052288	0.438976	0.198841	1.207673	-0.601388
1000 - 3500	0.020704	0.045928	0.41269	0.189677	1.175756	-0.549198

Table 30: Model 2 - Lvl 3 - "stdExp_dist" - $S = 5$

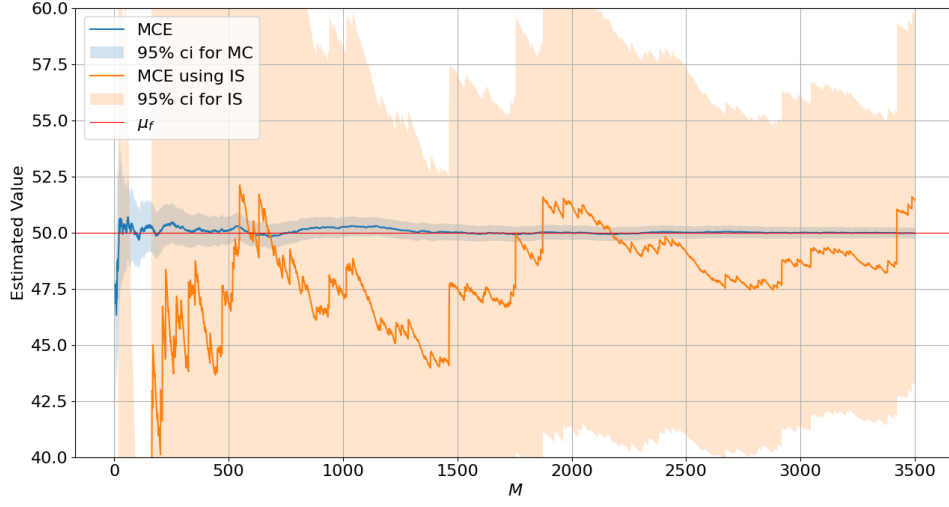


Figure 16: Model 2 - Lvl 1 - "stdExp_dist" - $S = 50$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	24.746663	12.126151	37.367174	25.241023	49.80718	48.448086	51.166274	2.718189	25.253337	0.19282
500	46.910847	30.026952	63.794742	33.76779	50.120843	49.50112	50.740566	1.239447	3.089153	0.120843
700	49.31761	35.084695	63.550525	28.46583	49.876955	49.341349	50.412561	1.071212	0.68239	0.123045
1000	47.254595	36.246427	58.262762	22.016335	50.257017	49.815401	50.698633	0.883232	2.745405	0.257017
1500	47.574506	38.009889	57.139124	19.129235	50.011985	49.65246	50.371509	0.719049	2.425494	0.011985
2000	51.21894	41.293797	61.144084	19.850286	49.991577	49.67428	50.308875	0.634594	1.21894	0.008423
2500	49.13983	40.961983	57.317677	16.355694	50.004666	49.722997	50.286334	0.563337	0.86017	0.004666
2700	47.879377	40.196801	55.561953	15.365152	50.030213	49.759003	50.301424	0.542421	2.120623	0.030213
3000	48.782564	41.416922	56.148207	14.731285	49.99426	49.738339	50.25018	0.511842	1.217436	0.00574
3500	51.449804	43.227978	59.67163	16.443652	49.965185	49.72756	50.202809	0.475249	1.449804	0.034815

Table 31: Model 2 - Lvl 2 - "stdExp_dist" - $S = 50$

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	3.982473	0.136506	19.246107	0.791994	23.300822	28.174386
1000 - 2000	2.960919	0.08407	18.992259	0.732791	24.917707	34.219787
1000 - 2500	2.176225	0.066972	18.64574	0.687959	26.102974	31.494359
1000 - 2700	2.13061	0.061203	18.308476	0.67195	26.246777	33.812375
1000 - 3000	2.114479	0.054024	17.793327	0.6502	26.365922	38.139849
1000 - 3500	1.923464	0.045581	17.139569	0.61888	26.694499	41.198896

Table 32: Model 2 - Lvl 3 - "stdExp_dist" - $S = 50$

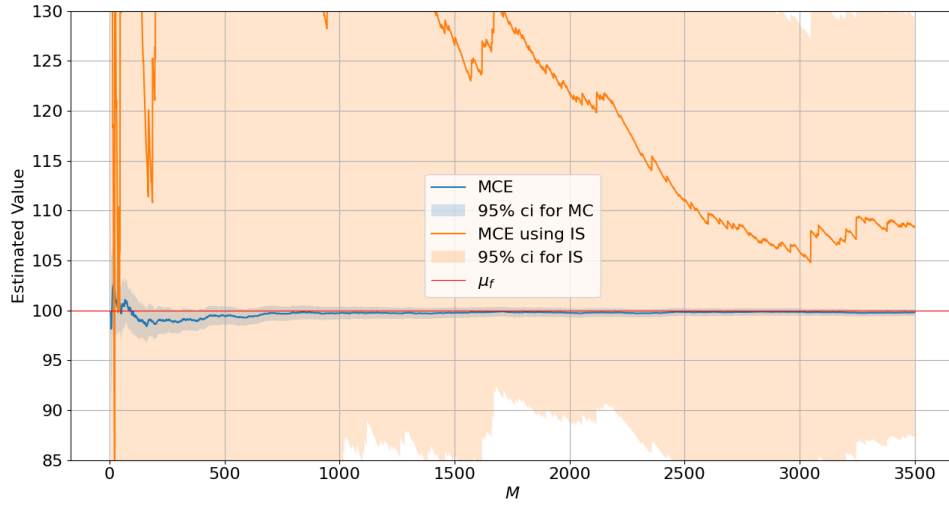


Figure 17: Model 2 - Lvl 1 - "stdExp_dist" - $S = 100$

M	IS	low. IS	up. IS	width ci IS	MCE	low. MCE	up. MCE	width ci MCE	abs. err. IS	abs. err. MCE
100	171.905916	76.138977	267.672855	191.533879	99.721464	97.75206	101.690867	3.938807	71.905916	0.278536
500	169.221579	63.9795	274.463658	210.484158	99.452276	98.547366	100.357186	1.80982	69.221579	0.547724
700	153.433732	76.429154	230.438309	154.009155	99.74796	98.994768	100.501153	1.506385	53.433732	0.25204
1000	143.567671	82.414846	204.720495	122.305649	99.725762	99.100524	100.350999	1.250475	43.567671	0.274238
1500	126.642198	84.642688	168.641708	83.99902	99.741665	99.227112	100.256218	1.029107	26.642198	0.258335
2000	121.711363	88.787745	154.63498	65.847235	99.753064	99.306635	100.199492	0.892857	21.711363	0.246936
2500	110.947929	84.022536	137.873322	53.850786	99.836236	99.440982	100.231489	0.790507	10.947929	0.163764
2700	108.527383	83.384804	133.669963	50.285159	99.803235	99.423591	100.182878	0.759288	8.527383	0.196765
3000	105.890507	83.106647	128.674367	45.56772	99.837627	99.478044	100.19721	0.719166	5.890507	0.162373
3500	108.330643	87.301937	129.359348	42.057411	99.777616	99.444392	100.11084	0.666448	8.330643	0.222384

Table 33: Model 2 - Lvl 2 - "stdExp_dist" - $S = 100$

M_min - M_max	av. abs. diff. IS	av. abs. diff. MCE	av. ci width IS	av. ci width MCE	rel. diff. width	rel. diff. err.
1000 - 1500	36.630218	0.271038	101.060568	1.131428	88.32122	134.147813
1000 - 2000	31.273223	0.234996	87.82872	1.044279	83.104663	132.07965
1000 - 2500	26.684152	0.237286	78.407774	0.974665	79.445861	111.455632
1000 - 2700	24.661943	0.23099	75.300647	0.951179	78.16561	105.766496
1000 - 3000	22.030344	0.220171	71.180564	0.919258	76.432629	99.059979
1000 - 3500	19.209426	0.220584	65.82598	0.87407	74.309729	86.084469

Table 34: Model 2 - Lvl 3 - "stdExp_dist" - $S = 100$

A.4 Full Code

A.4.1 Code class_natural_parameter_components.py

```
1 import numpy as np
2 import random
3 from scipy.stats import norm
4 from scipy.stats import expon
5
6 def ReLU(x):
7     return np.maximum(0, x)
8
9 def LReLU(x, beta):
10     if x < 0:
11         return x * beta
12     else:
13         return x
14
15 def shiftReLU(x, beta):
16     if x < beta:
17         return beta
18     else:
19         return x
20
21 def cappedReLU(x, beta):
22     if x < beta:
23         return np.maximum(0, x)
24     else:
25         return beta
26
27 def standardNormal(x, beta):
28     if x >= -beta and x <= beta:
29         return norm.pdf(x)
30     else:
31         return norm.pdf(beta)
32
33 def standardExp_dist(x, beta):
34     if x >= -beta and x <= beta:
35         return expon(scale=1).pdf(x)
36     else:
37         return expon(scale=1).pdf(beta)
38
39
40 class natural_parameter_definer(object):
41     def __init__(self, typ=None, beta=None):
42         def shiftReLUbeta(x): return shiftReLU(x, beta)
43         def LReLUbeta(x): return LReLU(x, beta)
44         def id(x): return x
45         def const(x): return beta
46         def cappedReLUbeta(x): return cappedReLU(x, beta)
47         def stdNormal(x): return standardNormal(x, beta)
48         def stdExp_dist(x): return standardExp_dist(x, beta)
49
50         if typ == "ReLU":
51             self.typ = ReLU
52
53         elif typ == "LReLU":
54             self.typ = LReLUbeta
```

```

55         elif typ == "shiftReLU":
56             self.typ = shiftReLUbeta
57
58         elif typ == "id":
59             self.typ = id
60
61         elif typ == "constant":
62             self.typ = const
63
64         elif typ == "cappedReLU":
65             self.typ = cappedReLUbeta
66
67         elif typ == "stdNormal":
68             self.typ = stdNormal
69
70         elif typ == "abs":
71             self.typ = abs
72
73         elif typ == "stdExp_dist":
74             self.typ = stdExp_dist
75
76         else:
77             self.typ = random.choice([ReLU, const,
78                                     shiftReLUbeta, LReLUbeta, cappedReLUbeta, id,
79                                     abs])
80
81     def create_z_exp_dist(self, S, lamb = None,
82                          combination_typ="mean_out_of_typ"):
83
84         typ = self.typ
85
86         if lamb == None:
87             lamb = 1
88
89         def z0(list_empty):
90             return -lamb
91
92         list_of_z = [z0]
93
94         if combination_typ == "mean_out_of_typ":
95             def combination(list_of_xs):
96                 list_of_xs = np.array(list_of_xs)
97                 sm1 = len(list_of_xs)
98                 typed_values = np.zeros(sm1)
99
100                 for s in range(sm1):
101                     typed_values[s] = typ(list_of_xs[s])
102
103                 output = np.mean(typed_values)
104
105                 return -output
106
107             combination_typ_for_USE = combination
108
109

```

```

110         elif combination_typ == "mean_in_typ":
111
112             def combination(list_of_xs):
113                 list_of_xs = np.array(list_of_xs)
114                 this_is_mean = np.mean(list_of_xs)
115                 output = typ(this_is_mean)
116
117                 return -output
118
119             combination_typ_for_USE = combination
120
121         else:
122             raise ValueError(f"Invalid combination_typ '{combination_typ}'. Use 'mean_out_of_typ' or 'mean_in_typ'.")
123
124         def make_z_s(comb_typ):
125             def z_s(list_of_xs):
126                 return comb_typ(list_of_xs)
127             return z_s
128
129         for _ in range(S - 1):
130             list_of_z.append(make_z_s(
131                 combination_typ_for_USE))
132
133         return list_of_z
134
135     def create_z_normal(self, d, S, mu=None, SIG=None,
136         combination_typ="mean_out_of_typ"):
137
138         typ = self.typ
139
140         if mu is None:
141             mu = np.ones(d)
142         if SIG is None:
143             SIG = np.eye(d)
144         SIG_inv = np.linalg.inv(SIG)
145
146         def z0(list_empty):
147             return SIG_inv @ mu, -0.5 * SIG_inv
148
149         list_of_z = [z0]
150
151         if combination_typ == "mean_out_of_typ":
152
153             def combination(list_of_xs, sig_mat):
154                 list_of_xs = np.array(list_of_xs)
155                 sm1, d = list_of_xs.shape
156                 output_array = np.zeros(d)
157                 typed_values = np.zeros((sm1, d))
158
159                 for i in range(d):
160                     for l in range(sm1):
161                         typed_values[l, i] = typ(list_of_xs[l, i])

```

```

163         output_array[i] = np.mean(typed_values
164                                  [:, i])
165     return output_array, -0.5 * sig_mat
166
167     combination_typ_for_USE = combination
168
169     elif combination_typ == "mean_in_typ":
170
171         def combination(list_of_xs, sig_mat):
172             list_of_xs = np.array(list_of_xs)
173             sm1, d = list_of_xs.shape
174             output_array = np.zeros(d)
175
176             for i in range(d):
177                 make_this_mean = np.zeros(sm1)
178
179                 for l in range(sm1):
180                     make_this_mean[l] = list_of_xs[l, i]
181
182                 this_is_mean = np.mean(make_this_mean)
183                 output_array[i] = typ(this_is_mean)
184
185             return output_array, -0.5 * sig_mat
186
187     combination_typ_for_USE = combination
188
189     else:
190
191         raise ValueError(f"Invalid combination_typ '{
192             combination_typ}'. Use 'mean_out_of_typ' or
193             'mean_in_typ'.")
194
195     def make_z_s(SIG_local, comb_typ):
196         def z_s(list_of_xs):
197             lam_s, Lam_s = comb_typ(list_of_xs,
198                                     SIG_local)
199             return lam_s, Lam_s
200         return z_s
201
202     for _ in range(S - 1):
203         list_of_z.append(make_z_s(SIG_inv,
204                                   combination_typ_for_USE))
205
206     return list_of_z

```

Python Code 21: class_natural_parameter_components Full Code

A.4.2 Code class_IS_optimizer.py

```

1 import numpy as np
2 import cvxpy as cp
3
4 def A_normal(lam_s, Lam_s, eps=1e-6, penalty=1e6):
5     Lam_reg = Lam_s + eps * np.eye(Lam_s.shape[0])
6     try:
7         sol = np.linalg.solve(Lam_reg, lam_s)
8     except np.linalg.LinAlgError:
9         return penalty
10    quad = -0.25 * np.dot(lam_s, sol)
11    try:
12        sign, logdet = np.linalg.slogdet(-2.0 * Lam_reg)
13        if sign <= 0 or not np.isfinite(logdet):
14            return penalty
15    except np.linalg.LinAlgError:
16        return penalty
17    return quad - 0.5 * logdet
18
19 def block_diag(mats):
20     sizes = [m.shape[0] for m in mats]
21     total = sum(sizes)
22     out = np.zeros((total, total), dtype=np.result_type(*
23         mats))
24     k = 0
25     for m in mats:
26         n = m.shape[0]
27         out[k:k+n, k:k+n] = m
28         k += n
29     return out
30
31 class IS_optimizer(object):
32     def __init__(self, list_of_z, samples, p_joint, f):
33         self.list_of_z = list_of_z
34         self.samples = samples
35         self.p_joint = p_joint
36         self.f = f
37
38         sample0 = self.samples[0]
39         assert sample0.ndim == 2, "Each sample must be a 2D
40             array of shape (S, d)"
41         self.S, self.d = sample0.shape
42
43     def exp_dist(self, bounds=None, cvxpy_solver=cp.SCS,
44         cvxpy_verbose=False, cvxpy_max_iters=50000,
45         cvxpy_eps=1e-5, cvxpy_acceleration_lookback=10,
46         cvxpy_scale=1.0):
47
48         d, S = 1, self.S
49         f = self.f
50         p_joint = self.p_joint
51         list_of_z = self.list_of_z
52
53         samples = self.samples
54         samples = np.asarray(samples)
55         if samples.ndim == 1:

```

```

52         samples = samples[None, :]
53         assert samples.shape[1] == S, "Each sample must
54         be a vector of length S."
55     M = samples.shape[0]
56     samples = samples.reshape(M, S)
57
58     def helping_z(XX):
59         z_array = np.zeros(S)
60         for s in range(S):
61             z_array[s] = list_of_z[s](XX[:, s])
62         return z_array
63
64     if np.any(samples <= 0):
65         raise ValueError("Exponential distributions
66         take no samples with values smaller or
67         equal to 0")
68
69     alpha = cp.Variable(S*d)
70
71     cons = []
72     eps_pos = 1e-5
73     if bounds is not None:
74         cons += list(bounds(alpha))
75     else:
76         cons += [alpha >= eps_pos]
77
78     z_T_all_XX = [None]*M
79     z_all_XX = [None]*M
80     sum_log_z_all_XX = [None]*M
81
82     for i, XX in enumerate(samples):
83         z_all_XX[i] = helping_z(XX)
84         z_T_all_XX[i] = z_all_XX[i] * XX
85         sum_log_z_all_XX[i] = np.sum(np.log(-
86             z_all_XX[i]))
87
88     z_T_matrix = np.array(z_T_all_XX)
89
90     weight = np.array([p_joint(XX) * (f(XX)**2) * np
91         .exp(-sum_log_z_all_XX[i]) for XX in samples
92         ])
93
94     alpha_sum_log = cp.sum(cp.log(alpha))
95     log_weight = cp.Constant(np.log(np.maximum(
96         weight, 1e-300)))
97     exponent_vec = -(z_T_matrix @ alpha) +
98         log_weight
99     expression = -alpha_sum_log + cp.log_sum_exp(
100         exponent_vec)
101
102     obj = cp.Minimize(expression)
103     prob = cp.Problem(obj, cons)
104     prob.solve(solver=cvxpy_solver, verbose=
105         cvxpy_verbose, max_iters=cvxpy_max_iters,
106         eps=cvxpy_eps, acceleration_lookback=
107         cvxpy_acceleration_lookback, scale=

```

```

cvxpy_scale)
97
98     if alpha.value is None:
99         raise RuntimeError(f"Solver failed (status:
{prob.status}).")
100
101     alpha_opt = alpha.value.reshape(S, d)
102
103     def create_list_of_z_alpha(alpha):
104         alpha = alpha.reshape(S, d)
105         list_of_z_alpha = []
106         for s in range(S):
107             alpha_s = alpha[s]
108             z_s = list_of_z[s]
109             def z_alpha_s_factory(z_s=z_s, alpha_s=
alpha_s):
110                 def z_alpha_s(list_of_x_s):
111                     eval_z_s = z_s(list_of_x_s)
112                     return alpha_s * eval_z_s
113                 return z_alpha_s
114             list_of_z_alpha.append(z_alpha_s_factory
())
115         return list_of_z_alpha
116
117     list_of_z_alpha = create_list_of_z_alpha(
alpha_opt)
118
119     def q_sampler(sample_size):
120         sample_set = [None]*sample_size
121         eps = 1e-8
122         for i in range(sample_size):
123             XX = []
124             for s in range(S):
125                 z_alpha_s = list_of_z_alpha[s]
126                 past_xs = [] if len(XX) == 0 else XX
127                 lam_s = abs(z_alpha_s(past_xs))
128                 lam_s = float(max(lam_s, eps))
129                 x_s = np.random.exponential(scale
=1.0/lam_s, size=d)
130                 XX.append(x_s)
131             XX_matrix = np.stack(XX, axis=0)
132             sample_set[i] = XX_matrix
133         return sample_set
134
135     def q_alpha(XX):
136         if np.any(np.array(XX) <= 0):
137             raise ValueError("This exponential pdf
take no samples with values smaller
or equal to 0")
138         list_of_x_s = []
139         eval_z = np.zeros(S)
140         eval_T = np.zeros(S)
141         for s in range(S):
142             x_s = XX[s]
143             eval_z[s] = list_of_z_alpha[s](
list_of_x_s)
144             eval_T[s] = x_s

```

```

145         list_of_x_s.append(x_s)
146         return np.exp(np.clip(eval_z.T @ eval_T,
147                                -700, 700)) * np.prod(np.maximum(-eval_z
148                                , 1e-300))
147
148         return q_sampler, q_alpha, alpha_opt,
149                list_of_z_alpha
150
151     def normal(self, bounds=None, cvxpy_solver=cp.SCS,
152                cvxpy_verbose=False, cvxpy_max_iters=20000,
153                cvxpy_eps=1e-5, cvxpy_acceleration_lookback=10,
154                cvxpy_scale=1.0):
155
156         d, S = self.d, self.S
157         f = self.f
158         p_joint = self.p_joint
159         q_0_inv = ((2 * np.pi) ** (-d / 2)) ** S ** (-1)
160         list_of_z = self.list_of_z
161
162         def T(x):
163             return x, np.outer(x, x)
164
165         samples = self.samples
166         M = len(samples)
167
168         alpha = cp.Variable(S*d)
169
170         cons = []
171         if bounds is not None:
172             cons += list(bounds(alpha))
173
174         frobenius_all_XX = [None]*M
175         quad_term_matrix_all_XX = [None]*M
176         log_det_all_XX = [None]*M
177         lam_times_tau_all_XX = [None]*M
178         exprs = [None]*M
179
180         for i, XX in enumerate(samples):
181             frob_sum = 0.0
182             logdet_sum = 0.0
183             Diag_lam_blocks = []
184             Lam_inv_blocks = []
185             lam_tau_segments = []
186
187             for s in range(S):
188                 lam_s, Lam_s = list_of_z[s](list(XX[:s]))
189                 tau_s, Tau_s = T(XX[s])
190
191                 frob_sum += np.vdot(Lam_s, Tau_s)
192
193                 L = np.linalg.cholesky(-2.0 * Lam_s)
194                 logdet_sum += np.sum(np.log(np.diag(L)))
195
196                 Diag_lam_blocks.append(np.diag(lam_s))
197                 Lam_inv_blocks.append(np.linalg.inv(Lam_s))
198                 lam_tau_segments.append(lam_s * tau_s)

```

```

197     frobenius_all_XX[i] = frob_sum
198     log_det_all_XX[i] = logdet_sum
199
200     D_lambda = block_diag(Diag_lam_blocks)
201     Lambda_inv = block_diag(Lam_inv_blocks)
202     Q = (D_lambda @ Lambda_inv @ D_lambda)
203     quad_term_matrix_all_XX[i] = -0.125 * (Q + Q.T)
204     lam_times_tau_all_XX[i] = np.concatenate(
205         lam_tau_segments)
206
207     Q_i = quad_term_matrix_all_XX[i]
208     b_i = lam_times_tau_all_XX[i]
209     exprs[i] = cp.quad_form(alpha, Q_i) - (alpha @
210         b_i)
211
212     weight = np.array([p_joint(XX) * (f(XX)**2) *
213         q_0_inv * np.exp(-frobenius_all_XX[i] -
214         log_det_all_XX[i]) for i, XX in enumerate(
215         samples)])
216
217     log_w = np.log(weight + 1e-300)
218     maxlog = float(np.max(log_w))
219
220     exponent = cp.hstack(exprs)
221     shifted = exponent + log_w - maxlog
222     obj = cp.Minimize(cp.sum(cp.exp(shifted)))
223     prob = cp.Problem(obj, cons)
224     prob.solve(solver=cvxpy_solver, verbose=cvxpy_verbose,
225         max_iters=cvxpy_max_iters, eps=cvxpy_eps,
226         acceleration_lookback=
227         cvxpy_acceleration_lookback, scale=cvxpy_scale)
228
229     if alpha.value is None:
230         raise RuntimeError(f"Solver failed (status: {
231             prob.status}).")
232
233     alpha_opt = alpha.value.reshape(S, d)
234
235     def create_list_of_z_alpha(alpha):
236         alpha = alpha.reshape(S, d)
237         list_of_z_alpha = []
238         for s in range(S):
239             alpha_s = alpha[s]
240             z_s = list_of_z[s]
241             def z_alpha_s_factory(z_s=z_s, alpha_s=
242                 alpha_s):
243                 def z_alpha_s(list_of_x_s):
244                     lam_s, Lam_s = z_s(list_of_x_s)
245                     return alpha_s * lam_s, Lam_s
246                 return z_alpha_s
247             list_of_z_alpha.append(z_alpha_s_factory())
248         return list_of_z_alpha
249
250     list_of_z_alpha = create_list_of_z_alpha(alpha_opt)
251
252     def q_sampler(sample_size):

```

```

244         sample_set = []
245         eps = 1e-6
246         for _ in range(sample_size):
247             XX = []
248             for s in range(S):
249                 z_alpha_s = list_of_z_alpha[s]
250                 past_xs = [] if s == 0 else XX
251                 lam_s, Lam_s = z_alpha_s(past_xs)
252                 Lam_reg = -2 * Lam_s + eps * np.eye(d)
253                 Sigma_s = np.linalg.inv(Lam_reg)
254                 mu_s = Sigma_s @ lam_s
255                 x_s = np.random.multivariate_normal(mu_s
256                                                         , Sigma_s)
257                 XX.append(x_s)
258             XX_matrix = np.stack(XX, axis=0)
259             sample_set.append(XX_matrix)
260         return sample_set
261
262     def q_alpha(XX):
263         q_0 = q_0_inv**(-1)
264         list_of_x_s = []
265         exp_XX = 0.0
266         for s in range(S):
267             x_s = XX[s]
268             lam_s, Lam_s = list_of_z_alpha[s](
269                 list_of_x_s)
270             tau_x_s, Tau_x_s = T(x_s)
271             exp_XX += (np.dot(lam_s, tau_x_s) + np.vdot(
272                 Lam_s, Tau_x_s) - A_normal(lam_s, Lam_s)
273             )
274             list_of_x_s.append(x_s)
275         return q_0 * np.exp(np.clip(exp_XX, -700, 700))
276
277     return q_sampler, q_alpha, alpha_opt,
278         list_of_z_alpha

```

Python Code 22: class_IS_optimizer Full Code

A.4.3 Example Code: Model 1

```

1 from class_IS_optimizer import IS_optimizer
2 from class_natural_parameter_components import
   natural_parameter_definer
3 import numpy as np
4 from scipy.stats import multivariate_normal
5
6 np.random.seed(42)
7
8 SS_0 = np.array([0.7, 1.1])
9 Sigma = np.array([[1, 1], [0, 1]])
10 S = 20
11 d = 2
12 t_S = 1
13 r = 0.05
14 K = 0.1
15
16 def BlackScholes(normal, ss_0=SS_0, r=r, sigma=Sigma, t_S=
   t_S, S=S):
17     SS1_array = np.zeros(S + 1)
18     SS2_array = np.zeros(S + 1)
19     SS1_array[0], SS2_array[0] = ss_0
20
21     del_t = t_S / S
22     sig1_quad = sigma[0, 0] ** 2 + sigma[0, 1] ** 2
23     sig2_quad = sigma[1, 0] ** 2 + sigma[1, 1] ** 2
24
25     for s in range(S):
26         Z1 = sigma[0, 0] * normal[s, 0] + sigma[0, 1] *
           normal[s, 1]
27         Z2 = sigma[1, 0] * normal[s, 0] + sigma[1, 1] *
           normal[s, 1]
28         SS1_array[s + 1] = SS1_array[s] * np.exp(
29             (r - 0.5 * sig1_quad) * del_t + np.sqrt(del_t) *
           Z1
30         )
31         SS2_array[s + 1] = SS2_array[s] * np.exp(
32             (r - 0.5 * sig2_quad) * del_t + np.sqrt(del_t) *
           Z2
33         )
34     return SS1_array, SS2_array
35
36 def Asian_Spread_Call(normal, K=K, ss_0=SS_0, r=r, sigma=
   Sigma, t_S=t_S, S=S):
37     SS1_array, SS2_array = BlackScholes(normal, ss_0=ss_0, r
   =r, sigma=sigma, t_S=t_S, S=S)
38
39     AsianSS1 = (np.sum(SS1_array) - SS1_array[0]) / S
40     AsianSS2 = (np.sum(SS2_array) - SS2_array[0]) / S
41     return np.maximum(0, AsianSS1 - AsianSS2 - K)
42
43 def f(XX):
44     return np.exp(-r * t_S) * Asian_Spread_Call(XX)
45
46 mean, cov = np.zeros(d), np.eye(d)
47

```

```

48 def p(X):
49     return multivariate_normal.pdf(X, mean=mean, cov=cov)
50
51 def p_joint(XX):
52     p_j = 1.0
53     for s in range(S):
54         p_j *= p(XX[s])
55     return p_j
56
57 list_of_z = natural_parameter_definer(typ="stdNormal", beta
    =5).create_z_normal(d, S, combination_typ="mean_in_typ")
58
59 training_size = 1000
60 samples_train = np.random.multivariate_normal(mean, cov,
    size=(training_size, S))
61
62 bounds = lambda x: []
63 opti = IS_optimizer(list_of_z, samples_train, p_joint, f)
64 q_sampler, q_alpha, alpha_opt, list_of_z_alpha = opti.normal
    (bounds=bounds)
65
66 M_eval = 7000
67 samples_eval = [x for x in np.random.multivariate_normal(
    mean, cov, size=(M_eval, S))]
68 samples_evalIS = q_sampler(M_eval)
69
70 ##—The user proceeds with their own calculations—##
71
72 ##—"True" Expectation estimation for reference—#
73 f_true = [f(x) for x in np.random.multivariate_normal(mean,
    cov, size=(1000000, S))]
74 true_expect = np.mean(f_true)
75
76 true_std_err = np.std(f_true, ddof=1) / np.sqrt(len(f_true))
77 true_ci_lower = true_expect - 1.96 * true_std_err
78 true_ci_upper = true_expect + 1.96 * true_std_err

```

Python Code 23: Example Asian Spread Call Option Full Example Code

A.4.4 Example Code: Model 2

```
1 from class_IS_optimizer import IS_optimizer
2 from class_natural_parameter_components import
   natural_parameter_definer
3 import numpy as np
4 from scipy.stats import expon
5
6 np.random.seed(34)
7
8 S = 5
9 d=1
10 scale =1
11
12 def f(XX):
13     return float(np.sum(XX))
14
15 def p_joint(XX):
16     return float(np.prod(expon.pdf(XX, scale=scale)))
17
18 list_of_z = natural_parameter_definer(typ="shiftReLU", beta
   =0.5).create_z_exp_dist(S=S,lamb = 1,combination_typ= "
   mean_in_typ")
19
20 training_size = 1000
21 samples_train = np.random.exponential(scale=scale, size=(
   training_size, S, d))
22
23 opti = IS_optimizer(list_of_z, samples_train, p_joint, f)
24 q_sampler, q_alpha, alpha_opt, list_of_z_alpha = opti.
   exp_dist()
25
26 M_eval = 3500
27 samples_eval = np.random.exponential(scale=scale, size=(
   M_eval, S, d))
28 samples_evalIS = q_sampler(M_eval)
29
30 ##——The user proceeds with their own calculations——##
```

Python Code 24: Example Poisson Process Full Example Code

A.4.5 Example Code: Monte Carlo Circle

```
1 import numpy as np
2
3 np.random.seed(42)
4
5 sample_size = 10000
6 samples = np.random.uniform(-1, 1, size=(sample_size, 2))
7
8 def f(XX):
9     if XX[0]**2 + XX[1]**2 <= 1:
10         return 1
11     else:
12         return 0
13
14 f_val = [f(XX) for XX in samples]
15 f_arr = np.asarray(f_val)
16
17 mu_f = np.mean(f_arr)
18 MCE = 4*mu_f
19
20 c = 1.96
21
22 std = np.std(f_arr)
23 se_pi = 4 * std / np.sqrt(sample_size)
24 low_pi = MCE - c * se_pi
25 up_pi = MCE + c * se_pi
26
27 print(MCE)
28 print(f"95% CI: [{low_pi:.6f}, {up_pi:.6f}]")
```

Python Code 25: Monte Carlo Circle

References

- [1] Akshay Agrawal et al. “A rewriting system for convex optimization problems”. In: *Journal of Control and Decision* 5.1 (2018), pp. 42–60.
- [2] Rabi Bhattacharya, Lizhen Lin, and Victor Patrangenaru. *A course in mathematical statistics and large sample theory*. Springer, 2016.
- [3] Siegfried Bosch. *Lineare algebra*. Springer, 2008.
- [4] Stephen P Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.
- [5] Nicolas Chopin and Omiros Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer Series in Statistics. Cham: Springer, 2020. ISBN: 978-3-030-47844-5. DOI: 10.1007/978-3-030-47845-2. URL: <https://doi.org/10.1007/978-3-030-47845-2>.
- [6] Anirban DasGupta. *Probability for statistics and machine learning: fundamentals and advanced topics*. Springer, 2011.
- [7] Steven Diamond and Stephen Boyd. “CVXPY: A Python-embedded modeling language for convex optimization”. In: *Journal of Machine Learning Research* 17.83 (2016), pp. 1–5.
- [8] Bradley Efron. *Exponential families in theory and practice*. Cambridge University Press, 2022.
- [9] Hans Föllmer and Alexander Schied. *Stochastic finance: an introduction in discrete time*. Walter de Gruyter, 2011.
- [10] Otto Forster. *Analysis 3: Maß-und Integrationstheorie, Integralsätze im $\mathbb{R}^n$ und Anwendungen*. Springer, 2011.
- [11] Python Software Foundation. *Python*. Accessed: 2025-06-10. 2025. URL: <https://www.python.org/>.
- [12] Martin Hanke-Bourgeois. *Grundlagen der numerischen Mathematik und des wissenschaftlichen Rechnens*. Springer, 2006.
- [13] J. D. Hunter. “Matplotlib: A 2D graphics environment”. In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: 10.1109/MCSE.2007.55.
- [14] Olav Kallenberg. *Foundations of Modern Probability*. 3rd ed. Vol. 99. Probability Theory and Stochastic Modelling. Springer, 2021. ISBN: 978-3-030-61870-4. DOI: 10.1007/978-3-030-61871-1.
- [15] Achim Klenke. *Wahrscheinlichkeitstheorie*. Springer, 2006.
- [16] Christiane Lemieux. *Monte Carlo and Quasi-Monte Carlo Sampling*. Springer Series in Statistics. Springer, 2009. ISBN: 978-0-387-78164-8. DOI: 10.1007/978-0-387-78165-5.

- [17] Andreas Maniatopoulos and Nikolaos Mitianoudis. “Learnable leaky relu (LeLeLU): An alternative accuracy-optimized activation function”. In: *Information* 12.12 (2021), p. 513.
- [18] N. Metropolis and S. Ulam. “The Monte Carlo Method”. In: *Journal of the American Statistical Association* 44.247 (1949), pp. 335–341. DOI: 10.2307/2280232. URL: <https://www.jstor.org/stable/2280232>.
- [19] Frank Nielsen and Vincent Garcia. “Statistical exponential families: A digest with flash cards”. In: *arXiv preprint arXiv:0911.4863* (2009).
- [20] Art B. Owen and Yi Zhou. “Safe and Effective Importance Sampling”. In: *Journal of the American Statistical Association* 95.449 (2000), pp. 135–143.
- [21] Sheldon M Ross. *Stochastic processes*. John Wiley & Sons, 1995.
- [22] Reuven Y. Rubinstein and Dirk P. Kroese. *Simulation and the Monte Carlo Method*. 3rd ed. Hoboken, NJ: John Wiley & Sons, 2017. ISBN: 978-1-119-36695-9.
- [23] The NumPy Team. *NumPy*. Accessed: 2025-06-10. 2025. URL: <https://numpy.org/>.
- [24] The SciPy Team. *SciPy*. Accessed: 2025-06-10. 2025. URL: <https://scipy.org/>.
- [25] Surya T. Tokdar and Robert E. Kass. “Importance Sampling: A Review”. Department of Statistics, Carnegie Mellon University, Pittsburgh, PA, USA. URL: <https://www2.stat.duke.edu/~st118/Publication/impsamp.pdf>.
- [26] Paul Wilmott. *Paul Wilmott on Quantitative Finance*. 2nd ed. Chichester, UK: John Wiley & Sons, 2006. ISBN: 978-0-470-31920-3.
- [27] Paul Wilmott. *Paul Wilmott on Quantitative Finance*. 2nd ed. Chichester, England: John Wiley & Sons, 2006. ISBN: 978-0-470-01870-5.